An extended abstract of this article will appear in Vol. 2025, No. 4 of IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES 2025). This is the full version. The code associated with our paper can be found here.

ABE Cubed: Advanced Benchmarking Extensions for ABE Squared

Sven Argo[1], Marloes Venema[2], Doreen Riepel[3], Tim Güneysu[1,4], and Diego F. Aranha[5]

[1] Ruhr University Bochum, Bochum, Germany, firstname.lastname@rub.de
[2] University of Wuppertal, Wuppertal, Germany, venema@uni-wuppertal.de
[3] CISPA Helmholtz Center for Information Security, Saarbrücken, Germany, riepel@cispa.de
[4] Deutsches Forschungszentrum für Künstliche Intelligenz, Bremen, Germany
[5] Aarhus University, Aarhus, Denmark, dfaranha@cs.au.dk

Abstract. Since attribute-based encryption (ABE) was proposed in 2005, it has established itself as a valuable tool in the enforcement of access control. For practice, it is important that ABE satisfies many desirable properties such as multi-authority and negations support. Nowadays, we can attain these properties simultaneously, but none of these schemes have been implemented. Furthermore, although simpler schemes have been optimized extensively on a structural level, there is still much room for improvement for these more advanced schemes. However, even if we had schemes with such structural improvements, we would not have a way to benchmark and compare them fairly to measure the effect of such improvements. The only framework that aims to achieve this goal, ABE Squared (TCHES '22), was designed with simpler schemes in mind.

In this work, we propose the ABE Cubed framework, which provides advanced benchmarking extensions for ABE Squared. To motivate our framework, we first apply structural improvements to the decentralized ciphertext-policy ABE scheme supporting negations presented by Riepel, Venema and Verma (ACM CCS '24), which results in five new schemes with the same properties. We use these schemes to uncover and bridge the gaps in the ABE Squared framework. In particular, we observe that advanced schemes depend on more "variables" that affect the schemes' efficiency in different dimensions. Whereas ABE Squared only considered one dimension (as was sufficient for the schemes considered there), we devise a benchmarking strategy that allows us to analyze the schemes in multiple dimensions. As a result, we obtain a more complete overview on the computational efficiency of the schemes, and ultimately, this allows us to make better-founded choices about which schemes provide the best efficiency trade-offs for practice.

Keywords: attribute-based encryption, multi-authority ABE, implementation

1 Introduction

Attribute-based encryption (ABE) [SW05] is a powerful type of public-key encryption that can be used to enforce access control to data [GPSW06a, KL10, SRGS12, WMZV16, VAH23]. For this reason, it has been considered by companies such as Cloudflare [LVV+23] and standardization institutes such as NIST [Nat23] and ETSI [ETS18]. To obtain the most benefits from ABE, practitioners typically choose schemes with many desirable, advanced features, including the support of negations [LSW10, YAHK14, Att19, TKN20] and multiple authorities [Cha07, LW11a, RW15, Ven23, AG23]. Furthermore, for many applications such as that of Cloudflare [LVV+23], it is important to provide fast decryption. However,

although some implementations exist [AGM+13, FEN, Zeu20, dlPVA22, LVV+23], most schemes satisfy only a limited number of properties. Specifically, none of the schemes support both multiple authorities and negations.

Recently, a scheme that satisfies many such attractive practical features simultaneously was proposed as part of the ISABELLA framework (RVV24-dCP-NEG) [RVV24a]. Due to its novelty and more complex nature than ABE schemes with fewer advanced properties, its computational efficiency is not well-understood. Furthermore, it uses almost no structural improvements that could significantly increase the computational performance, e.g., of decryption. For example, the Waters (Wat11) [Wat11] scheme was structurally improved with "randomness re-use" techniques by Agrawal and Chase (AC17) [AC17] to reduce the decryption costs. As ABE Squared [dlPVA22] shows, these techniques are indeed very powerful: AC17 decryption is an order of magnitude faster than Wat11 decryption.

In this paper, we apply similar structural improvements to RVV24-dCP-NEG to increase the decryption performance. Additionally, we use the principles posed by the ABE Squared framework to compare the efficiency of our structural optimizations with the starting point scheme: for all the schemes we use the same implementation strategies. Unfortunately, the benchmarking methodology (and subsequent evaluation of the benchmarks) provided by ABE Squared are insufficient to fairly and systematically compare the implemented schemes. The reason for this is that their methodology was applied to schemes with much simpler features and fewer "variables". In contrast, the schemes we consider are more advanced and have more variables, e.g., authorities and labels, that directly affect the efficiency of the schemes. Crucially, these variables affect the computational costs differently among the various schemes. Therefore, it is important to devise a more systematic approach.

To address these gaps, we present our framework: ABE Cubed, which provides **a**dvanced **b**enchmarking **e**xtensions for ABE Squared, essentially adding a new dimension to it. These extensions target schemes with advanced practical features, whose computational costs depend heavily on multiple variables. Our methodology aims to systematically analyze the influence of each variable on the computational costs. This allows us to obtain a more complete overview on the efficiency and, potentially, the different trade-offs posed by the schemes. Ultimately, it enables us to choose a scheme with the best trade-offs, motivated by some particular practical application.

1.1 Our contributions

Our main contributions are threefold.

- We first systematize commonly used structural improvements, as well as introducing a novel technique that enables even better improvements. We then apply these to RVV24-dCP-NEG, which yields five structural optimizations of the scheme. In this way, we obtain speedups in decryption by a factor up to 21.
- To benchmark and compare the schemes more accurately and fairly, we devise advanced benchmarking extensions for ABE Squared. These allow us to benchmark schemes that depend on multiple variables that influence the computational costs.
- To demonstrate ABE Cubed, we implement all schemes in Rust, and then apply our benchmarking strategies and evaluate the schemes.

1.2 Technical overview of our contributions

We give a technical overview, in which we will also address several side contributions.

1.2.1 The schemes

We briefly explain the properties of the schemes considered, optimized and implemented. These are *ciphertext-policy* attribute-based encryption (CP-ABE) [SW05, BSW07] schemes,

which is a type of public-key encryption that associates the keys and ciphertexts with predicates. In this type of ABE, users encrypt messages under policies. The resulting ciphertext can only be decrypted by a secret key for an attribute set that satisfies the associated policy. While most ABE schemes rely on a single trusted authority to issue the secret keys, *multi-authority* ABE [Cha07, CC09] allows to distribute this single point of trust across multiple parties. Some multi-authority schemes [LW11a, RW15] are fully decentralized in that they do not require any interaction among authorities.

Currently, the only existing unbounded CP-ABE schemes supporting negations are based on *pairings*, which is a map $e\colon \mathbb{G} \times \mathbb{H} \to \mathbb{G}_T$ over three groups $\mathbb{G}, \mathbb{H}$ and $\mathbb{G}_T$ of prime order p, with generators $g \in \mathbb{G}$, $h \in \mathbb{H}$, such that for all $x, y \in \mathbb{Z}_p$, we have $e(g^x, h^y) = e(g, h)^{xy}$. Using pairings, we can support *negations* in a completely unbounded fashion [LSW10, YAHK14, Att19], meaning that we can use an unlimited number of arbitrary strings as attributes, and the set and policy sizes are unbounded.

Recently, the ISABELLA [RVV24a] framework was proposed to simplify the design of ABE schemes with advanced practical features, such as multi-authority and negation support. As part of the framework, they introduced the first decentralized CP-ABE scheme supporting negations (RVV24-dCP-NEG) in a completely unbounded way. We use this scheme as the starting point for our structural optimization efforts. Furthermore, we prove all the modified versions of the scheme secure in the ISABELLA framework, enjoying the same strong security guarantees as the original scheme.

1.2.2 Randomness re-use

The first technique to optimize the structure of the starting point scheme is *randomness re-use*. To illustrate how randomness re-use works and that randomness re-use provides a significant advantage in practice, we consider the Wat11 [Wat11] and AC17 [AC17] schemes, as implemented and compared in ABE Squared [dlPVA22], as an example. The master public key, secret keys and ciphertexts of Wat11 are of the following form:

$$\begin{aligned}
\mathsf{MPK} &= \big(A = e(g,h)^{\alpha}, B = g^{b}, \{B_{\mathsf{att}} = g^{b_{\mathsf{att}}}\}_{\mathsf{att}\in\mathcal{U}}\big),\\
\mathsf{SK}_{\mathcal{S}} &= \big(\mathcal{S}, K = h^{\alpha+rb}, K' = h^{r}, \{K_{\mathsf{att}} = h^{rb_{\mathsf{att}}}\}_{\mathsf{att}\in\mathcal{S}}\big).\\
\mathsf{CT}_{\mathbb{A}} &= \big(\mathbb{A}, C = M\cdot A^{s}, C' = g^{s}, \{C_{1,j} = B^{\lambda_j} B_{\rho(j)}^{s_j}, C_{2,j} = g^{s_j}\}_{j\in[n_1]}\big),
\end{aligned}$$

where $\alpha, b, b_{\mathsf{att}}, r, s, s_j \in \mathbb{Z}_p$, $\mathcal{U}$ is the attribute universe, $\mathcal{S}$ an attribute set, $\mathbb{A}$ is an access policy represented as a linear secret sharing matrix $(\mathbf{A}, \rho)$, where λ_j is the j-th share of the secret s with respect to the policy $\mathbb{A}$, and ρ maps the rows of the policy matrix to attributes. If the set satisfies the policy, the key can decrypt the ciphertext as follows:

$$C \Big/ \left(e(C', K) \Big/ \prod_{j\in\Upsilon} \big(e(C_{1,j}, K')/e(C_{2,j}, K_{\rho(j)})\big)^{\varepsilon_j} \right),$$

where $\Upsilon = \{j \in [n_1] \mid \rho(j) \in \mathcal{S}\}$ is the set of literals in the policy that have a matching attribute in the set and $\{\varepsilon_j\}_{j\in\Upsilon}$ is defined such that $\sum_{j\in\Upsilon} \varepsilon_j \mathbf{A}_j = (1, 0, ..., 0)$. This can be computed more efficiently as

$$C \cdot e(C'^{-1}, K) \cdot e\left(\prod_{j\in\Upsilon} C_{1,j}^{\varepsilon_j}, K' \right) \cdot \prod_{j\in\Upsilon} e(C_{2,j}^{-\varepsilon_j}, K_{\rho(j)}).$$

Because the $C_{1,j}$ components are paired with the same "randomizer" K', the product can also be computed in $\mathbb{G}$ before pairing it with K', instead of pairing each separate $C_{1,j}$ component with K' and then taking the product of the pairing outputs.

Compared to Wat11, the AC17 scheme uses less randomness. Instead of using a fresh randomizer s_j for each row j of the policy matrix, AC17 re-uses the randomness more

efficiently. In the most extreme case, where each attribute occurs at most once in the policy, it only uses one randomizer for all rows. More specifically, for each occurrence of the *same* attribute, it needs to use a fresh randomizer, but it can use the same set of randomizers for each unique attribute. Hence, if any attribute occurs at most m times, then there are only m randomizers needed. Let $\tau\colon [n_1] \to [m]$ be a mapping that maps each row associated with the same attribute to a fresh randomizer, i.e., τ is injective on each subset of $[n_1]$ that is mapped to the same attribute with ρ. Then, AC17 has a master public key, secret keys and ciphertexts of the following form:

$$\begin{aligned}\mathsf{MPK} &= (A = e(g,h)^{\alpha}, B = g^{b}, \{B_{\mathsf{att}} = g^{b_{\mathsf{att}}}\}_{\mathsf{att}\in\mathcal{U}}),\\ \mathsf{SK}_{\mathcal{S}} &= (\mathcal{S}, K = h^{\alpha+rb}, K' = h^{r}, \{K_{\mathsf{att}} = h^{rb_{\mathsf{att}}}\}_{\mathsf{att}\in\mathcal{S}}).\\ \mathsf{CT}_{\mathbb{A}} &= (\mathbb{A}, C = M\cdot A^{s}, C' = g^{s}, \{C_{1,j} = B^{\lambda_j} B_{\rho(j)}^{s_{\tau(j)}}\}_{j\in[n_1]}, \{C_{2,\ell} = g^{s_\ell}\}_{\ell\in[m]}).\end{aligned}$$

Instead of requiring $|\Upsilon| + 2$ pairing operations to decrypt, like in the Wat11 scheme, we may require only 3, because

$$C / \left(e(C', K) / \prod_{j\in\Upsilon} \left(e(C_{1,j}, K') / e(C_{2,\tau(j)}, K_{\rho(j)}) \right)^{\varepsilon_j} \right)$$

can be computed more efficiently as

$$C \cdot e(C'^{-1}, K) \cdot e\left(\prod_{j\in\Upsilon} C_{1,j}^{\varepsilon_j}, K' \right) \cdot \prod_{\ell\in\tau(\Upsilon)} e\left(C_{2,\ell}, \prod_{j\in\Upsilon:\tau(j)=\ell} K_{\rho(j)}^{-\varepsilon_j} \right),$$

which costs $|\tau(\Upsilon)|+2$ pairing operations. Here, in the best case, the number of randomizers $|\tau(\Upsilon)|$ used in the decryption is 1. Hence, re-using randomness improves the decryption efficiency. In addition, it also optimizes the encryption efficiency, because we need to generate fewer g^{s_ℓ} components than before.

1.2.3 Randomness splitting

Our second structural optimization technique is a technique that we call *randomness splitting*, which allows us to re-use more randomness in RVV24-dCP-NEG than the structure currently allows. To illustrate how randomness splitting works, we consider a slightly modified version of the decentralized CP-ABE scheme by Rouselakis and Waters [RW15]. To simplify the explanation, we have placed the scheme in the single-authority setting. Note that we now also use a full-domain hash function $\mathcal{H}_1\colon \{0,1\}^* \to \mathbb{G}$ to generate $B_{\mathsf{att}} = \mathcal{H}_1(\mathsf{att})$, and a hash function $\mathcal{H}_2\colon \{0,1\}^* \to \mathbb{G}$ to map some global identifier GID into the group $\mathbb{G}$. We can then describe the scheme as follows:

$$\begin{aligned}\mathsf{MPK} &= \left(A = e(g,h)^{\alpha}, B = g^{b}\right),\\ \mathsf{SK}_{\mathsf{GID},\mathcal{S}} &= \left(\mathcal{S}, \{K_{\mathsf{att}} = g^{\alpha}\cdot\mathcal{H}_2(\mathsf{GID})^{b}\cdot\mathcal{H}_1(\mathsf{att})^{r}\}_{\mathsf{att}\in\mathcal{S}}, K' = h^{r}\right).\\ \mathsf{CT}_{\mathbb{A}} &= \big(\mathbb{A}, C = M\cdot e(g,h)^{\tilde{s}}, \{C_{1,j} = e(g,h)^{\lambda_j}\cdot A^{s_j}, C_{2,j} = g^{\mu_j}\cdot B^{s_j},\\ &\qquad C_{3,j} = \mathcal{H}_1(\rho(j))^{s_j}, C_{4,j} = h^{s_j}\}_{j\in[n_1]}\big),\end{aligned}$$

where λ_j is now a sharing of $\tilde{s}$ and μ_j is a sharing of 0. To decrypt, we compute

$$C\cdot\prod_{j\in\Upsilon} C_{1,j}^{-\varepsilon_j}\cdot e\left(\prod_{j\in\Upsilon} C_{2,j}^{\varepsilon_j}, \mathcal{H}_2(\mathsf{GID})\right)\cdot e\left(\prod_{j\in\Upsilon} C_{3,j}^{\varepsilon_j}, K'\right)\cdot\prod_{j\in\Upsilon} e(K_{\rho(j)}^{-\varepsilon_j}, C_{4,j}),$$

which costs $|\Upsilon| + 2$ pairing operations. Again, the number of pairing operations is linear in Υ because of the fresh randomizer s_j for each attribute in the policy. However, if we

try to use the same randomness re-use trick that AC17 applies to improve the efficiency of Wat11, i.e., replacing s_j by $s_{\tau(j)}$ in the $C_{3,j}$ term (where τ is the same as before), we run into problems with either the correctness or the security.

Specifically, if we keep using fresh randomizers s_j in the $C_{1,j}$ terms, the pairings in the third product would yield the components

$$e(K_{\rho(j)}, C_{4,\tau(j)}) = e(g,h)^{\alpha s_{\tau(j)}} \cdot e(\mathcal{H}_2(\mathsf{GID}), h)^{s_{\tau(j)} b} \cdot e(\mathcal{H}_1(\mathsf{att}), h)^{r s_{\tau(j)}}.$$

To cancel out $e(\mathcal{H}_2(\mathsf{GID}), h)^{s_{\tau(j)} b}$, we need to pair $C_{2,\tau(j)}$ and $\mathcal{H}_2(\mathsf{GID})$, instead of $C_{2,j}$. Similarly, to cancel out $e(g,h)^{\alpha s_{\tau(j)}}$, we use $C_{1,\tau(j)}$ instead of $C_{1,j}$. This yields the wrong "shares" that we need to decrypt, i.e., we obtain:

$$C_{1,\tau(j)} \cdot e(C_{2,\tau(j)}^{-1}, \mathcal{H}_2(\mathsf{GID})) \cdot e(C_{2,\tau(j)}, K') = e(g,h)^{\lambda_{\tau(j)}} \cdot e(g, \mathcal{H}_2(\mathsf{GID}))^{\mu_{\tau(j)}},$$

instead of $e(g,h)^{\lambda_j} \cdot e(g, \mathcal{H}_2(\mathsf{GID}))^{\mu_j}$. To show why this is problematic, suppose that, for example, the policy uses each attribute only once, in which case τ maps every input to 1. Then, we can only recover the share $e(g,h)^{\lambda_1} \cdot e(g, \mathcal{H}_2(\mathsf{GID}))^{\mu_1}$, which is in general insufficient to reconstruct the secret. For example, for the policy $\mathsf{att}_1 \wedge \mathsf{att}_2$, we need to also obtain the share $e(g,h)^{\lambda_2} \cdot e(g, \mathcal{H}_2(\mathsf{GID}))^{\mu_2}$. Hence, we cannot recover $e(g,h)^{\tilde{s}}$, and ultimately, not decrypt.

On the other hand, if we replace s_j in $C_{1,j}$ and $C_{2,j}$ term by $s_{\tau(j)}$, then the scheme is insecure, because $A^{s_{\tau(j)}}$ does not provide enough randomness[1] to hide $e(g,h)^{\lambda_j}$, and similarly, $B^{s_{\tau(j)}}$ does not provide enough randomness to hide g^{μ_j}.

To solve this problem, we apply a technique that we call randomness splitting. The idea is that the key component K_{att} is split in two parts, which are "tied together" with some new randomness. We do this by replacing K_{att} by

$$K_1 = g^{\alpha} \cdot \mathcal{H}_2(\mathsf{GID})^b \cdot (B')^r, \qquad K_{2,\mathsf{att}} = \mathcal{H}_1(\mathsf{att})^r,$$

where $B' = g^{b'}$ is the new randomness that is included in the public key. These key components K_1 and $K_{2,\mathsf{att}}$ are tied implicitly together via B' and r on the ciphertext side, by replacing $C_{3,j} = \mathcal{H}_1(\rho(j))^{s_j}$ by:

$$C_{3,j} = (B')^{s_j} \cdot \mathcal{H}_1(\rho(j))^{s_{\tau(j)}}.$$

Note that we have applied the randomness re-use techniques to the hashed part now, while we still use different randomizers s_j in combination with B' for each j, which ensures security with a similar argument as for AC17. Moreover, we can decrypt more efficiently:

$$C \cdot \prod_{j \in \Upsilon} C_{1,j}^{-\varepsilon_j} \cdot e\left(\prod_{j \in \Upsilon} C_{2,j}^{\varepsilon_j}, \mathcal{H}_2(\mathsf{GID})\right) \cdot e\left(\prod_{j \in \Upsilon} C_{3,j}, K'\right) \cdot e\left(K_1, \prod_{j \in \Upsilon} C_{4,j}^{\varepsilon_j}\right) \cdot \prod_{\ell \in \tau(\Upsilon)} e\left(\prod_{j \in \Upsilon} K_{2,\rho(j)}^{-\varepsilon_j}, C_{4,\ell}\right),$$

which costs $|\tau(\Upsilon)| + 3$ pairing operations, and if each attribute occurs only once in the policy, costs only 4 pairing operations.

1.2.4 Structural improvements to RVV24-dCP-NEG

Although applying the randomness re-use and randomness-splitting techniques to our examples is reasonably straightforward and we obtain schemes that do not depend on

[1] We refer to [VA23] for an attack on similar schemes that do not provide sufficient randomness.

many variables, we aim to apply the techniques to a significantly more advanced scheme. In particular, the attributes of RVV24-dCP-NEG are not represented by a single string, but rather by triples of strings, consisting of an authority, a label and an attribute. Here, the label is included to support negations in an efficient yet unbounded way. Furthermore, the scheme supports positive and negative literals in the policies, both of which affect the schemes' efficiency in a different way. Although RVV24-dCP-NEG already has a much more complex dependency on different variables, it is further complicated in the schemes we obtain by applying randomness re-use.

To argue that randomness re-use affects the efficiency at all, we first show how the two example schemes are affected. As mentioned, the decryption costs depend on the size of $\tau(\Upsilon)$, i.e., the number of randomizers g^{s_ℓ} that are associated with the attributes that are matched with during decryption. If attributes are used only once in the policy, then only one such randomizer is used. However, if the same attribute occurs multiple times in the policy, then multiple randomizers are used, and thus, the decryption costs may scale up accordingly. Further, the number of occurrences of the same attribute may also impact the encryption efficiency, as it increases the number of randomizers that we require.

In our schemes, the randomness re-use techniques have other restrictions. In total, we have three such mappings with different restrictions. The first needs to map the triples in the attribute set to a fresh randomizer for each use of the same authority-label pair. The second needs to map the triples associated with the policy to a fresh randomizer for each use of the same authority, and the third maps those triples to fresh randomizers for each use of the same authority-label pair as well. Depending on the scheme, different combinations of these mappings are used, and subsequently, the scheme is impacted differently as well.We illustrate the difference between the three mappings with examples. For the difference between the second and third mappings, consider the policy $(\texttt{auth}_1, \texttt{profession}, \texttt{doctor}) \wedge (\texttt{auth}_1, \texttt{profession}, \texttt{nurse}) \wedge (\texttt{auth}_1, \texttt{department}, \texttt{neurology})$. The second mapping would map the attributes to $1, 2, 3$, requiring three randomizers, whereas the third mapping would map the attributes to $1, 2, 1$. For the difference between the first and third mapping, we note that the first is defined for attribute set, e.g., $\{(\texttt{auth}_1, \texttt{profession}, \texttt{doctor}), (\texttt{auth}_1, \texttt{department}, \texttt{cardiology})\}$, while the third is defined for policies.

The benchmarking strategy in ABE Squared contains a gap in this respect: it does not explain how to measure and analyze the use of attribute duplicates in the policy. Although for AC17 this might not be very problematic, as it is clear how it is related to Wat11, it is problematic for our schemes. In particular, the duplication of the same authority is a very likely scenario in practice: we might have very few authorities but long policies, consisting of multiple label-attribute pairs associated with the same authority. Assuming that such duplicates do not occur gives a distorted view on the schemes' efficiency. Therefore, we need to extend the ABE Squared benchmarking strategies to compare schemes more fairly.

1.2.5 ABE Cubed

To bridge this gap, we present ABE Cubed. The benchmarking strategy of ABE Cubed consists of two phases: one that is similar to that in ABE Squared, and a second phase, where we measure the influence of various types of duplication. Using this strategy, we can benchmark and evaluate in more dimensions than ABE Squared. More concretely, the first phase consists of two steps, where the first step applies the strategies from ABE Squared to establish a "baseline efficiency" for the schemes. This step poses a fixed structure on the policy (to ensure consistency among all schemes) and assumes that all literals consist of three unique strings. The benchmarking strategy then iterates over various policy lengths. We do this twice: once for policies with all-positive literals and once with all-negative literals. In the second step of the first phase, we analyze the influence of the set size on policies consisting of negative literals. In this way, we obtain a good overview of the dependency of the computational costs on the policy and set sizes.

In the second phase of our benchmarking strategy, we analyze the influence of various types of attribute duplicates. Because attributes consist of triples, we consider six types of duplications: for the singletons (e.g., authority duplicates) and the pairs (e.g., authority-label duplicates). Again, we consider the case with all-positive and all-negative literals. To ensure that we can iterate over the number of duplicates, yet make measurements that compare fairly, we fix the policy length to a large value with many small divisors, and iterate over those divisors. For those variables that we fix that do not have a linear dependence on the efficiency of the schemes, we take measurements for multiple choices.

After applying the strategy to the schemes, we use our benchmarks to compare the schemes. First, we analyze the regular benchmarks to obtain a preliminary view on how the schemes compare. Then, we carefully analyze the relationship between all variables and the efficiency. We do this by comparing the scheme for each variable with its "baseline" (i.e., the "regular" case, in which all attribute strings are unique). If we see significant differences, then the efficiency is dependent and thus relevant.

Finally, for all those relevant variables, we summarize the measured efficiency by taking ranges over the maximum slowdown or speedup factors compared to RVV24-dCP-NEG among all iterations. Depending on the importance we put on the algorithm, we choose whether we analyze the slowdown or speedup factors. For example, since we aim to optimize the decryption efficiency, we consider the speedup factors for decryption, and slowdown factors for key generation and encryption. By computing the maximum slowdown for key generation and encryption among all iterations, we obtain a worst-case view on the costs of those algorithms. By considering the maximum speedups for decryption, we obtain a good view on the advantages that the schemes provide. Lastly, we can combine these two overviews to discuss different trade-offs. For example, whereas one scheme may be a clear winner in the decryption efficiency, it may have a big trade-off in key generation or encryption efficiency. Another candidate may perform slightly slower in decryption, but may be faster in the key generation or encryption. In this way, we can obtain better-founded recommendations for schemes than with ABE Squared.

1.2.6 Implementations in Rust

To demonstrate our framework, we implement all schemes in Rust (using Arkworks [ac22]), and apply the benchmarking strategy. We use these benchmarks and subsequent analysis to show that our structural optimizations improve on the efficiency of the starting point scheme: RVV24-dCP-NEG. Therefore, we obtain the most efficient scheme of its kind (i.e., completely unbounded decentralized CP-ABE schemes that support negations), and in general, the first implementation of any such schemes. As part of our implementation, we also provide a policy parser. This makes our implementations fully functional and deployable. This is unlike the ABE Squared implementations, which do not have a working parser. Furthermore, compared to the Charm [AGM+13] and OpenABE [Zeu20] frameworks, ours implements the more efficient secret sharing scheme from [LW11b] (used in conjunction with the parser) as highlighted in ABE Squared.

2 Preliminaries

Notation. If an element x is chosen uniformly at random from a finite set S, then we denote this as $x \in_R S$. If an element x is produced by running algorithm Alg, then we denote this as $x \leftarrow \text{Alg}$. We use $\mathbb{Z}_p = \{x \in \mathbb{Z} \mid 0 \leq x < p\}$ for the set of integers modulo p. For integers $a < b$, we denote $[a, b] = \{a, a+1, ..., b-1, b\}$, $[b] = [1, b]$ and $\overline{[b]} = [0, b]$. We use boldfaced variables $\mathbf{A}$ and $\mathbf{v}$ for matrices and vectors, respectively, where $(\mathbf{A})_{i,j}$ denotes the entry of $\mathbf{A}$ in the i-th row and j-th column, and $(\mathbf{v})_i$ denotes the i-th entry of $\mathbf{v}$. We use $\mathbf{A}_i$ to denote the i-th row of matrix $\mathbf{A}$. We denote $a \ : \ \mathbf{A}$ to substitute variable

a by a matrix or vector $\mathbf{A}$. We define $\mathbf{1}_{i,j}^{d_1 \times d_2} \in \mathbb{Z}_p^{d_1 \times d_2}$ as the matrix with 1 in the i-th row and j-th column, and 0 everywhere else, and similarly $\mathbf{1}_i^{d_1}$ and $\overline{\mathbf{1}}_i^{d_2}$ as the row and column vectors with 1 in the i-th entry and 0 everywhere else. If some algorithm yields no output or outputs an error message, then we use $\perp$ to indicate this. We use $a \| b$ to indicate that two strings a and b are concatenated.

Access structures. We represent access policies $\mathbb{A}$ by linear secret sharing scheme (LSSS) matrices, which support monotone span programs [Bei96, GPSW06b].

Definition 1 (Access structures represented by LSSS [GPSW06b])**.** An access structure is represented as a pair $\mathbb{A} = (\mathbf{A}, \rho)$ such that $\mathbf{A} \in \mathbb{Z}_p^{n_1 \times n_2}$ is an LSSS matrix, where $n_1, n_2 \in \mathbb{N}$, and ρ is a function that maps its rows to attributes in the universe. Then, for some vector $\mathbf{v} = (s, v_2, ..., v_{n_2}) \in_R \mathbb{Z}_p^{n_2}$, the i-th share of secret s generated by this matrix is $\lambda_i = \mathbf{A}_i \mathbf{v}^\intercal$, where $\mathbf{A}_i$ denotes the i-th row of $\mathbf{A}$. In particular, if $\mathcal{S}$ satisfies $\mathbb{A}$, then there exist a set of rows $\Upsilon = \{i \in [n_1] \mid \rho(i) \in \mathcal{S}\}$ and coefficients $\varepsilon_i \in \mathbb{Z}_p$ for all $i \in \Upsilon$ such that $\sum_{i \in \Upsilon} \varepsilon_i \mathbf{A}_i = (1, 0, ..., 0)$, and by extension $\sum_{i \in \Upsilon} \varepsilon_i \lambda_i = s$, holds. If $\mathcal{S}$ does not satisfy $\mathbb{A}$, there exists $\mathbf{w} = (1, w_2, ..., w_{n_2}) \in \mathbb{Z}_p^{n_2}$ such that $\mathbf{A}_i \mathbf{w}^\intercal = 0$ for all $i \in \Upsilon$ [Bei96].

Pairings (or bilinear maps). We define a pairing to be an efficiently computable map e on three groups $\mathbb{G}, \mathbb{H}$ and $\mathbb{G}_T$ of prime order p, so that $e \colon \mathbb{G} \times \mathbb{H} \to \mathbb{G}_T$, with generators $g \in \mathbb{G}, h \in \mathbb{H}$ is such that (i) for all $a, b \in \mathbb{Z}_p$, it holds that $e(g^a, h^b) = e(g, h)^{ab}$ (bilinearity), and (ii) for $g^a \neq 1_{\mathbb{G}}, h^b \neq 1_{\mathbb{H}}$, it holds that $e(g^a, h^b) \neq 1_{\mathbb{G}_T}$, where $1_{\mathbb{G}'}$ denotes the unique identity element of the associated group $\mathbb{G}'$ (non-degeneracy). We use the implicit representation used for group elements [EHK+13]. In particular, we use $[x]_{\mathbb{G}'}$ to denote the element $(g')^x$, where $g' \in \mathbb{G}'$ is the generator of some group $\mathbb{G}' \in \{\mathbb{G}, \mathbb{H}, \mathbb{G}_T\}$.

2.1 Attribute-based encryption

We define ABE for predicates following [Att14, AC17]. Similar to recent work [ABGW17, RW22, RVV24a], we define ABE as a KEM rather than explicitly encrypting a message.

Predicate family. A predicate $P \colon \mathcal{X} \times \mathcal{Y} \to \{0, 1\}$ is a map over the ciphertext attribute space $\mathcal{X}$ and the key attribute space $\mathcal{Y}$ such that, for $x \in \mathcal{X}, y \in \mathcal{Y}$, $P(x, y) = 1$ if and only if the predicate evaluates to true for x and y.

Syntax. An attribute-based encryption scheme ABE for a predicate $P \colon \mathcal{X} \times \mathcal{Y} \to \{0, 1\}$ consists of four algorithms:

- $\mathsf{Setup} \to (\mathsf{MPK}, \mathsf{MSK})$: This probabilistic algorithm generates the master public key MPK and the master secret key MSK.
- $\mathsf{KeyGen}(\mathsf{MSK}, y) \to \mathsf{SK}_y$: On input the master secret key MSK and some $y \in \mathcal{Y}$, this probabilistic algorithm generates a secret key SK_y.
- $\mathsf{Encrypt}(\mathsf{MPK}, x) \to (\mathsf{CT}_x, K)$: On input the master public key MPK and some $x \in \mathcal{X}$, this probabilistic algorithm generates a ciphertext CT_x and an encapsulated key K.
- $\mathsf{Decrypt}(\mathsf{MPK}, \mathsf{SK}_y, \mathsf{CT}_x) \to K$: On input the master public key MPK, the secret key SK_y, and the ciphertext CT_x, if $P(x, y) = 1$, then it returns K; and otherwise, $\perp$.

Correctness. For all $x \in \mathcal{X}$, and $y \in \mathcal{Y}$ such that $P(x, y) = 1$,

$$\Pr[(\mathsf{MPK}, \mathsf{MSK}) \leftarrow \mathsf{Setup}; (\mathsf{CT}_x, K) \leftarrow \mathsf{Encrypt}(\mathsf{MPK}, x) : \mathsf{Decrypt}(\mathsf{MPK}, \mathsf{KeyGen}(\mathsf{MSK}, y), \mathsf{CT}_x) = K] = 1.$$

Multi-authority ABE syntax and security. In the multi-authority setting, the Setup is split in two algorithms, where one is run globally and the other one is run by each authority in the system. Furthermore, each authority executes the key generation with its own input. Multi-authority ABE typically provides security against corruption, i.e., if the corrupted authorities and the decryption capabilities of the generated keys do not yield sufficient knowledge to satisfy the predicate, then the scheme should still be secure. For example, if we consider the policy $\mathsf{att}_1 \wedge \mathsf{att}_2 \wedge \mathsf{att}_3$, where att_1 is mapped to authority $\mathcal{A}_1$, att_2 to $\mathcal{A}_2$ and att_3 to $\mathcal{A}_3$, authority $\mathcal{A}_1$ is corrupted and the adversary has a key for att_2, then the message should remain hidden.

Multi-authority ciphertext-policy ABE for Boolean formulas (supporting negations). In this paper, we consider multi-authority ciphertext-policy ABE for Boolean formulas that supports negations. In this type of ABE, the key predicate y is a set of attributes $\mathcal{S}$ and the ciphertext predicate x is a policy $\mathbb{A}$. We represent each attribute in the set $\mathcal{S}$ as a triple $(l, \mathsf{lab}, \mathsf{att})$, where l denotes an authority in the authority identifier universe $\mathcal{AID}$, lab a label in the label universe $\mathcal{L}$, and att is an attribute value in the attribute universe $\mathcal{U}$. Sometimes, we use $\mathcal{S}_l$ to denote the "subset" of $\mathcal{S}$ managed by authority l, where $\mathcal{S}_l = \{(\mathsf{lab}, \mathsf{att}) \mid (l, \mathsf{lab}, \mathsf{att}) \in \mathcal{S}\}$ consists only of the label-attribute pairs. We represent the policy $\mathbb{A}$ as a tuple $(\mathbf{A}, \rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho})$, where $\mathbf{A}$ is an LSSS matrix as in Definition 1, and $\rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho}$ map the rows of the matrix to the attributes and their literals. First, the mapping $\rho\colon [n_1] \to \mathcal{U}$ maps the rows to attribute values. The mapping $\rho'\colon [n_1] \to \{0,1\}$ maps the rows to 0 when the attribute is negated and 1 otherwise. The mapping $\rho_{\mathsf{lab}}\colon [n_1] \to \mathcal{L}$ represents the labeling of the attributes associated with the rows. Lastly, the mapping $\tilde{\rho}\colon [n_1] \to \mathcal{AID}$ maps the rows to authorities. The predicate is satisfied for some policy and set if, for $\Upsilon = \{j \in [n_1] \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S} \wedge \rho'(j) = 1\}$ and $\overline{\Upsilon} = \{j \in [n_1] \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S} \wedge \rho'(j) = 0 \wedge \exists \mathsf{att} : (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \mathsf{att}) \in \mathcal{S}\}$, there exist $\{\varepsilon_j\}_{j \in \Upsilon \cup \overline{\Upsilon}}$ with $\sum_{j \in \Upsilon \cup \overline{\Upsilon}} \varepsilon_j \mathbf{A}_j = (1, 0, ..., 0)$.

2.2 Pair encodings and their security

We construct our new schemes from pair encodings using the ISABELLA compiler. Roughly, the ISABELLA compiler abstracts schemes in which the master public key, the secret keys and the ciphertexts have the following form:

$$\mathsf{MPK} = (e(g,h)^{\boldsymbol{\alpha}}, (g')^{\mathbf{b}}, (g')^{\boldsymbol{\beta}}), \qquad \mathsf{SK}_y = ((g')^{\mathbf{r}}, (g')^{\mathbf{k}(\mathbf{r},\hat{\mathbf{r}},\boldsymbol{\alpha},\boldsymbol{\beta},\mathbf{b})}),$$
$$\mathsf{CT}_x = (M \cdot e(g,h)^{c_M}, (g')^{\mathbf{s}}, (g')^{\mathbf{c}(\mathbf{s},\hat{\mathbf{s}},\mathbf{b})}, e(g,h)^{\mathbf{c}'(\mathbf{s},\tilde{\mathbf{s}},\boldsymbol{\alpha},\boldsymbol{\beta})}, (g')^{\mathbf{c}''(\mathbf{s},\tilde{\mathbf{s}},\boldsymbol{\beta})}),$$

where g' indicates that either $g' = g$ or $g' = h$ for each entry of the vector in the exponent. The associated pair encoding scheme is denoted "in the exponent", i.e., $\boldsymbol{\alpha}, \boldsymbol{\beta}, \mathbf{b}$, etc. The ISABELLA class of pair encodings is defined as follows.

Definition 2 (ISABELLA class of pair encoding schemes (PES-ISA))**.** A pair encoding scheme $\Gamma_{\text{PES-ISA}}$ for the class of schemes covered by our compiler, and for a predicate $P\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ and a prime integer $p \in \mathbb{N}$, is given by four deterministic polynomial-time algorithms as described below.

- Param(par) $\to (n_\alpha, n_b, n_\beta, \boldsymbol{\alpha}, \mathbf{b}, \boldsymbol{\beta})$: On input parameters par, the algorithm outputs $n_\alpha, n_b, n_\beta \in \mathbb{N}$ that specify the number of master key variables, common variables and semi-common variables, respectively, which are denoted as $\boldsymbol{\alpha} = (\alpha_1, ..., \alpha_{n_\alpha})$, $\mathbf{b} = (b_1, ..., b_{n_b})$ and $\boldsymbol{\beta} = (\beta_1, ..., \beta_{n_\beta})$, respectively.
- EncKey(y) $\to (m_1, m_2, \mathbf{k}(\mathbf{r}, \hat{\mathbf{r}}, \boldsymbol{\alpha}, \boldsymbol{\beta}, \mathbf{b}))$: On input $y \in \mathcal{Y}$, this algorithm outputs a vector of polynomials $\mathbf{k} = (k_1, ..., k_{m_3})$ defined over non-lone variables $\mathbf{r} = (r_1, ..., r_{m_1})$

and lone variables $\hat{\mathbf{r}} = (\hat{r}_1, ..., \hat{r}_{m_2})$. Specifically, the polynomial k_i is expressed as

$$k_i = \sum_{j\in[n_\alpha]} \delta_{1,i,j}\alpha_j + \sum_{j\in[n_\beta]} \delta_{2,i,j}\beta_j + \sum_{j\in[m_2]} \hat{\delta}_{i,j}\hat{r}_j + \sum_{j\in[m_1],k\in[n_b]} \delta_{3,i,j,k}r_j b_k,$$

for all $i \in [m_3]$, where $\delta_{1,i,j}, \delta_{2,i,j}, \hat{\delta}_{i,j}, \delta_{3,i,j,k} \in \mathbb{Z}_p$.

- $\mathrm{EncCt}(x) \to (w_1, w_2, w_2', c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}), \mathbf{c}'(\mathbf{s}, \tilde{\mathbf{s}}, \boldsymbol{\alpha}, \boldsymbol{\beta}), \mathbf{c}''(\mathbf{s}, \tilde{\mathbf{s}}, \boldsymbol{\beta}))$: On input $x \in \mathcal{X}$, this algorithm outputs a blinding variable c_M and three vectors of polynomials $\mathbf{c} = (c_1, ..., c_{w_3})$, $\mathbf{c}' = (c_1', ..., c_{w_4}')$ and $\mathbf{c}'' = (c_1'', ..., c_{w_5}'')$ defined over non-lone variables $\mathbf{s} = (s = s_0, s_1, s_2, ..., s_{w_1})$, lone variables $\hat{\mathbf{s}} = (\hat{s}_1, ..., \hat{s}_{w_2})$ and special lone variables $\tilde{\mathbf{s}} = (\tilde{s}_1, ..., \tilde{s}_{w_2'})$. Specifically, the polynomial c_i is expressed as

$$c_i = \sum_{j\in\overline{[w_1]},k\in[n_b]} \eta_{1,i,j,k}s_j b_k + \sum_{j\in[w_2]} \hat{\eta}_{1,i,j}\hat{s}_j$$

for all $i \in [w_3]$, where $\eta_{1,i,j,k}, \hat{\eta}_{1,i,j} \in \mathbb{Z}_p$, the polynomial c_i' is expressed as

$$c_i' = \sum_{j\in[n_\alpha],j'\in\overline{[w_1]}} \eta_{1,i,j,j'}'\alpha_j s_{j'} + \sum_{j\in[n_\beta],j'\in\overline{[w_1]}} \eta_{2,i,j,j'}'\beta_j s_{j'} + \sum_{j\in[w_2']} \hat{\eta}_{i,j}'\tilde{s}_j,$$

for all $i \in [w_4]$, where $\eta_{1,i,j,j'}', \eta_{2,i,j,j'}', \hat{\eta}_{i,j}' \in \mathbb{Z}_p$, the polynomial c_i'' is expressed as

$$c_i'' = \sum_{j\in\overline{[w_1]},k\in[n_\beta]} \eta_{2,i,j,k}s_j\beta_k + \sum_{j\in[w_2']} \hat{\eta}_{2,i,j}\tilde{s}_j,$$

for all $i \in [w_5]$, where $\eta_{2,i,j,k}, \hat{\eta}_{2,i,j} \in \mathbb{Z}_p$, and the polynomial c_M is expressed as

$$c_M = \sum_{j\in[w_2']} \zeta_j\tilde{s}_j + \sum_{j\in[n_\alpha],j'\in\overline{[w_1]}} \zeta_{1,j,j'}\alpha_j s_{j'} + \sum_{j\in[n_\beta],j'\in\overline{[w_1]}} \zeta_{2,j,j'}\beta_j s_{j'},$$

where $\zeta_j, \zeta_{1,j,j'}, \zeta_{2,j,j'} \in \mathbb{Z}_p$.

- $\mathrm{Pair}(x, y) \to (\mathbf{e}', \mathbf{e}'', \mathbf{E}, \overline{\mathbf{E}})$: On input x, and y, this algorithm outputs two vectors $\mathbf{e}' \in \mathbb{Z}_p^{w_4}, \mathbf{e}'' \in \mathbb{Z}_p^{w_5}$ and two matrices $\mathbf{E}$ and $\overline{\mathbf{E}}$ of sizes $(w_1 + 1) \times m_3$ and $w_3 \times m_1$, respectively.

A PES-ISA is correct if, for every $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ such that $P(x, y) = 1$, it holds that $\mathbf{e}'\mathbf{c}'^{\intercal} + \mathbf{e}''\mathbf{c}''^{\intercal} + \mathbf{s}\mathbf{E}\mathbf{k}^{\intercal} + \mathbf{c}\overline{\mathbf{E}}\mathbf{r}^{\intercal} = c_M$.

Security definition. We also give the associated security definition, which we use to simplify proving security of our new schemes.

Corruptable variables and the predicate. The definition of the symbolic property supports the corruption of variables by listing a set of corruptable variables. Corruption of the variables often impacts the satisfiability of the predicate P. In particular, P_{cor}, with $\mathsf{cor} = (\Gamma_{\text{PES-ISA}}, \mathfrak{a}_1, \mathfrak{a}_2, \mathfrak{b})$, expands the predicate P for scheme $\Gamma_{\text{PES-ISA}}$ with respect to the corruptable variables.

Definition 3 (Special selective symbolic property (SSSP) for PES-ISA)**.** A scheme $\Gamma_{\text{PES-ISA}} = (\mathrm{Param}, \mathrm{EncKey}, \mathrm{EncCt}, \mathrm{Pair})$ for a predicate $P\colon \mathcal{X} \times \mathcal{Y} \to \{0, 1\}$ satisfies the (d_1, d_2)-selective symbolic property for positive integers d_1 and d_2 and indices of corruptable variables $\mathfrak{a}_1 \subsetneq [n_\alpha]$, $\mathfrak{a}_2 \subsetneq [n_\beta]$, $\mathfrak{b} \subsetneq [n_b]$ and the predicate P_{cor} that expands the predicate P with respect to the corruptable variables, if there exist deterministic polynomial-time algorithms EncB, EncS, and EncR such that for all $x \in \mathcal{X}$ and $y \in \mathcal{Y}$ with $P_{\mathsf{cor}}(x, y) = 0$, such that we have that

- $\mathrm{EncB}(x, \mathfrak{a}_1, \mathfrak{a}_2, \mathfrak{b}) \to \mathbf{a}_1, ..., \mathbf{a}_{n_\alpha}, \mathbf{b}_1, ..., \mathbf{b}_{n_\beta} \in \mathbb{Z}_p^{d_1}, \mathbf{B}_1, ..., \mathbf{B}_{n_b} \in \mathbb{Z}_p^{d_1 \times d_2}$;
- $\mathrm{EncR}(x, y) \to \mathbf{r}_1, ..., \mathbf{r}_{m_1} \in \mathbb{Z}_p^{d_2}, \hat{\mathbf{r}}_1, ..., \hat{\mathbf{r}}_{m_2} \in \mathbb{Z}_p^{d_1}$;
- $\mathrm{EncS}(x) \to \mathbf{s}_0, ..., \mathbf{s}_{w_1} \in \mathbb{Z}_p^{d_1}, \hat{\mathbf{s}}_1, ..., \hat{\mathbf{s}}_{w_2} \in \mathbb{Z}_p^{d_2}, \tilde{\mathbf{s}}_1, ..., \tilde{\mathbf{s}}_{w_2'} \in \mathbb{Z}_p$;

such that, if we substitute

$$\hat{s}_{i'} : \hat{\mathbf{s}}_{i'} \quad \tilde{s}_{i''} : \tilde{\mathbf{s}}_{i''} \quad s_i b_j : \mathbf{s}_i \mathbf{B}_j \quad \alpha_l : \mathbf{a}_l^\intercal \quad \beta_{l'} : \mathbf{b}_{l'}^\intercal$$
$$\alpha_l s_i : \mathbf{s}_i \mathbf{a}_l^\intercal \quad \beta_{l'} s_i : \mathbf{s}_i \mathbf{b}_{l'}^\intercal \quad \hat{r}_{k'} : \hat{\mathbf{r}}_{k'}^\intercal \quad r_k b_j : \mathbf{B}_j \mathbf{r}_k^\intercal,$$

for $i \in [w_1], i' \in [w_2], i'' \in [w_2'], j \in [n_b], k \in [m_1], k' \in [m_2], l \in [n_\alpha], l' \in [n_\beta]$ in all the polynomials of $\mathbf{k}$ and $\mathbf{c}, \mathbf{c}', \mathbf{c}''$ (output by EncKey and EncCt, respectively), they evaluate to $\mathbf{0}^{d_1}$, $\mathbf{0}^{d_2}$, 0 and 0, respectively, and the polynomial c_M to nonzero. Furthermore, for $j \in \mathfrak{a}_1, j' \in \mathfrak{a}_2$, we have $\mathbf{a}_j = \mathbf{b}_{j'} = \mathbf{0}^{d_1}$ and for $j \in \mathfrak{b}$, we have that $\mathbf{B}_j = \mathbf{0}^{d_1 \times d_2}$.

Remark 1. Because the substitution vectors in the SSSP proofs are the same for the $\mathbf{c}'$ and $\mathbf{c}''$ parts (as well as the $\boldsymbol{\alpha}$ and $\boldsymbol{\beta}$ parts), we can simply move all polynomials in $\mathbf{c}''$ to $\mathbf{c}'$ for the security proofs, and split the two before instantiation with the ISABELLA compiler.

2.3 The ISABELLA compiler

The ISABELLA compiler transforms any PES-ISA for which SSSP holds into a secure ABE scheme. It allows much freedom in the distribution of the key and ciphertext components (as long as correctness is guaranteed). Furthermore, it allows common variables and non-lone key variables to be generated using a full-domain hash. Lastly, it uses a split-predicate mold to ensure that the public keys and secret keys can be generated more adaptively, as is required in multi-authority ABE. Because all of the schemes in our paper are multi-authority CP-ABE schemes, we use the split-predicate mold for multi-authority CP-ABE in [RVV24b]. For simplicity, we apply the mold directly to the compiler, so that we can remove one level of complexity of the ISABELLA compiler.

Distribution of the encodings. The ISABELLA compiler takes as input explicit definitions for the distribution of the encodings over $\mathbb{G}$ and $\mathbb{H}$ when they are instantiated. This distribution is denoted with the mapping $\mathfrak{D}$. Such a distribution should ensure that the correctness of the PES is preserved, such that the correctness of the ABE scheme is also guaranteed. In particular, for the correctness of the decryption algorithm, we require that each pair of key and ciphertext encodings that needs to be paired has one input in $\mathbb{G}$ and one in $\mathbb{H}$. Furthermore, to ensure that encryption can be performed correctly, the master public keys required in computing a ciphertext encoding element need to be in the same group. We additionally also have some restrictions on the distributions of $\mathbf{s}$ and $\boldsymbol{\beta}$. In particular, these need to be in the same group when they occur in a product in $\mathbf{c}''$.

Full-domain hashes and random oracles. When variables are generated implicitly by a full-domain hash (FDH), specified with the mapping $\mathcal{F}$, the description of the scheme needs to specify the hash input strings. The ISABELLA compiler also allows for explicit domain separation. To ensure correctness of the scheme, we require the distribution over the $\mathbb{G}$ and $\mathbb{H}$ to be such that, for any common variable b_k that is provided implicitly by a hash, and each associated encoding k_i and c_i, it holds that they are placed in the same group. Similarly, we can define such a restriction for the other variables. Furthermore, if a non-lone variable and a common variable occur together in a product in one of the polynomials, then it cannot be the case that both are generated by an FDH.

Compatibility. We define the notion of compatibility in the context of our compiler, which ensures that the inputs to the compiler are compatible for some PES-ISA. This covers the correctness properties for the mappings $\mathfrak{D}$ and $\mathcal{F}$. Additionally, we require that the PES-ISA uses the same prime order p as the pairing group and that there exist full-domain hash functions into groups $\mathbb{G}$ and $\mathbb{H}$.

Notation. In the following, we will often run the algorithms of the PES-ISA and instantiate the polynomials in the groups, i.e., we sample uniform values from $\mathbb{Z}_p$ and compute group elements whose exponents correspond to the PES-ISA polynomials evaluated on these values. To simplify notation, we will use algorithms SampleParam, SampleKey, SampleCt to sample those values. We write, e.g., $(\boldsymbol{\alpha}, \boldsymbol{\beta}, \mathbf{b})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleParam}(\text{par})$, where $\mathcal{F}_1(\cdot) = 0$ indicates that we skip those variables that are instantiated with an FDH.

Definition 4 (The ISABELLA compiler for multi-authority CP-ABE molds)**.** Let $\Gamma_{\text{PES-ISA}}$ be a PES-ISA for predicate $P\colon \mathcal{X} \times \mathcal{Y} \to \{0,1\}$ as defined in Definition 2 with distributions $\mathfrak{D}$ and $\mathcal{F}$, and let $\mathcal{PG}$ be a pairing group compatible with $\Gamma_{\text{PES-ISA}}$. Further, we assume the multi-authority CP-ABE mold in [RVV24b]. We define the scheme $\mathsf{ABE\text{-}ISA} = \mathbf{Comp}[\mathcal{PG}, \Gamma_{\text{PES-ISA}}, \mathsf{SPM}[\Gamma_{\text{PES-ISA}}], \mathfrak{D}, \mathcal{F}]$ as follows:

- $\mathsf{GlobalSetup} \to \text{GP}$: This algorithm picks full-domain hash functions $\mathcal{H}_{\mathbb{G}} : \{0,1\}^* \to \mathbb{G}$ and $\mathcal{H}_{\mathbb{H}} : \{0,1\}^* \to \mathbb{H}$ modeled as random oracles. It outputs global parameters $\text{GP} = (\mathcal{PG}, \mathcal{H}_{\mathbb{G}}, \mathcal{H}_{\mathbb{H}})$.
- $\mathsf{AuthoritySetup}(\mathcal{PG}, \mathcal{A}_l) \to (\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}, \mathsf{MSK}_{\mathcal{A}_l})$: On input the group description $\mathcal{PG}$ and authority identifier $\mathcal{A}_l$, this algorithm runs $(\boldsymbol{\alpha}_l, \boldsymbol{\beta}_l, \mathbf{b}_l)_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleParam}_l(\text{par})$, sets $\mathsf{MSK} = (\boldsymbol{\alpha}_l, \boldsymbol{\beta}_l, \{b_j \mid b_j \in \mathbf{b}_l \wedge \mathcal{F}_1(b_j) = 0\})$ as the master secret key, and outputs MSK along with master public key

$$\mathsf{MPK}_{\mathcal{A}_l} = (\{[\alpha_j]_{\mathbb{G}_T}\}_{\alpha_j \in \boldsymbol{\alpha}_l}, \{[\beta_j]_{\mathfrak{D}(\beta_j)} \mid \beta_j \in \boldsymbol{\beta}_l \wedge \mathcal{F}_1(\beta_j) = 0\}, \{[b_j]_{\mathfrak{D}(b_j)} \mid b_j \in \mathbf{b}_l \wedge \mathcal{F}_1(b_j) = 0\}).$$

- $\mathsf{KeyGen}(\mathcal{A}_l, \mathsf{MSK}_{\mathcal{A}_l}, \mathsf{GID}, \mathcal{S}_{\mathcal{A}_l}) \to \mathsf{SK}_{\mathsf{GID},\mathcal{S}_{\mathcal{A}_l}}$: On input the authority identifier $\mathcal{A}_l$, the master secret key $\mathsf{MSK}_{\mathcal{A}_l}$, global identifier GID and a set $\mathcal{S}_{\mathcal{A}_l}$ containing attributes managed by the authority, this algorithm generates $(\mathbf{k}_l(\mathbf{r}_l, \hat{\mathbf{r}}_l, \boldsymbol{\alpha}_l, \boldsymbol{\beta}_l, \mathbf{b}_l)) \leftarrow \text{EncKey}_l(\mathcal{S}_{\mathcal{A}_l})$ and $(\mathbf{r}_l, \hat{\mathbf{r}}_l)_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleKey}(\mathcal{S}_{\mathcal{A}_l})$, and outputs the secret key

$$\mathsf{SK}_{\mathcal{S}_{\mathcal{A}_l}} = (\mathcal{S}_{\mathcal{A}_l}, \{[r_j]_{\mathfrak{D}(r_j)} \mid r_j \in \mathbf{r}_l \wedge \mathcal{F}_1(r_j) = 0\}, \{[k_i]_{\mathfrak{D}(k_i)}\}_{k_i \in \mathbf{k}_l}).$$

- $\mathsf{Encrypt}(\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}\}_{l \in \tilde{\rho}([n_1])}, x = (\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}})) \to (\mathsf{CT}_x, K)$: On input the authority identifiers and associated master public keys $\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}\}_{l \in \tilde{\rho}([n_1])}$ and some policy $x = (\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}))$, this algorithm first generates $(w_1, w_2, w_2', c_M, \mathbf{c}(\mathbf{s}, \hat{\mathbf{s}}, \mathbf{b}), \mathbf{c}'(\mathbf{s}, \tilde{\mathbf{s}}, \boldsymbol{\alpha}, \boldsymbol{\beta}), \mathbf{c}''(\mathbf{s}, \tilde{\mathbf{s}}, \boldsymbol{\beta}) \leftarrow \text{EncCt}(x)$ and $(\mathbf{s}, \hat{\mathbf{s}}, \tilde{\mathbf{s}})_{\mathcal{F}_1(\cdot)=0} \leftarrow \text{SampleCt}(x)$, and outputs the key $K = e(g,h)^{c_M}$ and the ciphertext

$$\mathsf{CT}_x = (x, [s]_{\mathfrak{D}(s)}, \{[s_j]_{\mathfrak{D}(s_j)} \mid j \in [w_1] \wedge \mathcal{F}_1(s_j) = 0\}, \{[c_i]_{\mathfrak{D}(c_i)}\}_{i \in [w_3]}, \{[c_i']_{\mathbb{G}_T}\}_{i \in [w_4]}, \{[c_i'']_{\mathfrak{D}(c_i'')}\}_{i \in [w_5]}).$$

- $\mathsf{Decrypt}(\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}, \mathsf{SK}_{\mathcal{A}_l,\mathsf{GID}}\}_l, \mathsf{CT}) \to K$: On input the master public keys $\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}$, a secret key $\mathsf{SK}_{\mathcal{A}_l,\mathsf{GID}}$ and a ciphertext CT for policy $x = (\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}})$, if $x \models y$, where $y = \bigcup_l \mathcal{S}_{\mathcal{A}_l}$, this algorithm first obtains $(\mathbf{e}', \mathbf{e}'', \mathbf{E}, \overline{\mathbf{E}}) \leftarrow \text{Pair}(x, y)$, sets

$$\mathfrak{P} = \{(s_j, k_i, \mathbf{E}_{j,i}) \mid i \in [m_3], j \in \overline{[w_1]}, \mathbf{E}_{j,i} \neq 0 \wedge \mathfrak{D}(s_j) = \mathbb{G}\} \cup \{(k_i, s_j, \mathbf{E}_{j,i}) \mid i \in [m_3], j \in \overline{[w_1]}, \mathbf{E}_{j,i} \neq 0 \wedge \mathfrak{D}(s_j) = \mathbb{H}\}$$

$$\cup \{(r_j, c_i, \overline{\mathbf{E}}_{i,j}) \mid i \in [w_3], j \in [m_1], \overline{\mathbf{E}}_{i,j} \neq 0 \wedge \mathfrak{D}(r_j) = \mathbb{G}\}$$
$$\cup \{(c_i, r_j, \overline{\mathbf{E}}_{i,j}) \mid i \in [w_3], j \in [m_1], \overline{\mathbf{E}}_{i,j} \neq 0 \wedge \mathfrak{D}(r_j) = \mathbb{H}\},$$
$$\mathcal{I}_{\mathbb{G}} = \{i \in [w_5] \mid \mathfrak{D}(c_i'') = \mathbb{G}\}, \qquad \mathcal{I}_{\mathbb{H}} = \{i \in [w_5] \mid \mathfrak{D}(c_i'') = \mathbb{H}\}$$

and then retrieves

$$\prod_{i\in[w_4]} [c_i']_{\mathbb{G}_T}^{\mathbf{e}_i'} \prod_{i\in\mathcal{I}_{\mathbb{G}}} e([c_i'']_{\mathbb{G}}, h) \prod_{i\in\mathcal{I}_{\mathbb{H}}} e(g, [c_i'']_{\mathbb{H}}) \prod_{(\mathfrak{l},\mathfrak{r},\mathfrak{e})\in\mathfrak{P}} e([\mathfrak{l}]_{\mathbb{G}}, [\mathfrak{r}]_{\mathbb{H}})^{\mathfrak{e}}$$
$$= e(g,h)^{\mathbf{e}'\mathbf{c}'^{\intercal}+\mathbf{e}''\mathbf{c}''^{\intercal}+\mathbf{s}\mathbf{E}\mathbf{k}^{\intercal}+\mathbf{c}\overline{\mathbf{E}}\mathbf{r}^{\intercal}} = e(g,h)^{c_M}.$$

Notational convention for compiled schemes. To obtain a strong connection between the PES-ISA and compiled ABE scheme, we use notation throughout this paper that, on the one hand, distinguishes PES from ABE, while on the other hand, we use the same indexing strategies used in the PES also in the ABE. As a convention, we use lower-cased letters for the variables and polynomials associated with the PES, and upper-cased letters for key and ciphertext components associated with the compiled ABE scheme. Hence, for key and ciphertext polynomials k_i and c_i, we map to K_i and C_i respectively. For the primed ciphertext polynomials c_i' and c_i'', we map to C_i'. Note that, because we construct $\mathbf{c}''$ from $\mathbf{c}'$, there will be no collisions in the use of indices. For the master-key and semi-common variables α_i and β_i, we map to A_i. Similarly as for the primed ciphertext polynomials, because we construct $\boldsymbol{\beta}$ from $\boldsymbol{\alpha}$, we expect no collisions in the indices. For the non-lone key and ciphertext variables r_j and s_j, we map to $\check{K}_j$ and $\check{C}_j$, respectively.

Security. Security of schemes generated using the compiler follows from the SSSP for the input PES-ISA. In [RVV24a], two results are proven: static security under a q-type assumption, and adaptive security in the generic group model, both in the random oracle model. We review these results in Appendix A.1.

3 The starting point scheme

We give a description of our starting point scheme: the decentralized large-universe CP-ABE scheme that supports negations in a completely unbounded way from [RVV24b], which we denote by RVV24-dCP-NEG. In comparison, existing schemes that support negations [AG23, Ven23] are bounded in the number of key attributes that can be issued per authority [AG23] or the number of re-uses of the same label in the keys [Ven23].

The predicate for the scheme is the multi-authority CP-ABE predicate. Recall that this predicate evaluates true on $(\mathbf{A}, \rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho})$ and $\mathcal{S}$, if, for $\Upsilon = \{j \in [n_1] \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S} \wedge \rho'(j) = 1\}$ and $\overline{\Upsilon} = \{j \in [n_1] \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S} \wedge \rho'(j) = 0 \wedge \exists \mathsf{att} : (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \mathsf{att}) \in \mathcal{S}\}$, there exist $\{\varepsilon_j\}_{j\in\Upsilon\cup\overline{\Upsilon}}$ with $\sum_{j\in\Upsilon\cup\overline{\Upsilon}} \varepsilon_j \mathbf{A}_j = (1, 0, ..., 0)$. This means that, intuitively, Υ and $\overline{\Upsilon}$ denote the sets of rows in $[n_1]$ that are associated with the positive and negative literals, respectively. Some row $j \in [n_1]$ is in Υ when $\rho'(j) = 1$ and the associated pair $(\rho_{\mathsf{lab}}(\mathsf{lab}), \rho(\mathsf{att}))$ is in the set $\mathcal{S}_{\tilde{\rho}(j)}$, where $\mathcal{S}_l = \{(\mathsf{lab}, \mathsf{att}) \mid (l, \mathsf{lab}, \mathsf{att}) \in \mathcal{S}\}$ denotes the set of label-attribute pairs managed by authority l. Conversely, some row $j \in [n_1]$ is in $\overline{\Upsilon}$ when it is negated, i.e., $\rho'(j) = 0$, and the associated pair $(\rho_{\mathsf{lab}}(j), \rho(j))$ is *not* in the set $\mathcal{S}_{\tilde{\rho}(j)}$, and there exists at least one attribute $(\mathsf{lab}, \mathsf{att}) \in \mathcal{S}_{\tilde{\rho}(j)}$ such that $\mathsf{lab} = \rho_{\mathsf{lab}}(j)$ and $\mathsf{att} \neq \rho(j)$.

3.1 High-level description

We first give a high-level description of the scheme that explains the various components. The first component is the access policy, which is represented as an LSSS matrix $\mathbf{A}$

(Definition 1). Then, to support negations, the scheme uses a combination of full-domain hashes and "implicit polynomials in the exponent" to represent attributes. Roughly, to map an authority-label-attribute triple $(l, \mathsf{lab}, \mathsf{att})$ into the group, the scheme computes $B_{l,\mathsf{lab},0} \cdot B_{l,\mathsf{lab},1}^{x_{\mathsf{att}}}$, where $B_{l,\mathsf{lab},0}$ and $B_{l,\mathsf{lab},1}$ are computed using a full-domain hash and $x_{\mathsf{att}} \in \mathbb{Z}_p$ is the integer representation of the attribute string att. Hence, for each authority-label pair, we compute an implicit polynomial of degree 1, and then map the attribute att into the group using that polynomial and by mapping att into $\mathbb{Z}_p$ with a hash.

To support multiple authorities, the scheme uses a similar structure as many existing decentralized ABE schemes [LW11a, RW15, AG21, AG23, Ven23]. These schemes all require a hash to map a global identity GID of a user into the group, which is used as a "linchpin" to tie keys to a single user. Furthermore, each authority has its own "master key", to ensure that we can provide security against corruptions: if several colluding users and corrupted authorities do not satisfy the policy, the message should remain hidden.

Lastly, we observe that the scheme consists of two components: a "positive" and a "negative" part. The positive part is used to encode the positive literals into the ciphertext (and decrypt with the corresponding "positive part" of the key), whereas the negative part is similarly used to encode the negations. We use the same randomizer provided by applying a hash to GID to tie the positive and negative parts together. The positive part is of the following form:

$$\mathsf{MPK}_{l,+} = \left(A_l = h^{\alpha_l}, B_l = h^{b_l}\right),$$

$$\mathsf{SK}_{\mathcal{S}_l,+} = \left(\left\{K_{1,(l,\mathsf{lab},\mathsf{att})} = g^{\alpha_l} \cdot \mathsf{H}(\mathsf{GID})^{b_l} \cdot \left(B_{l,\mathsf{lab},0} \cdot B_{l,\mathsf{lab},1}^{x_{\mathsf{att}}}\right)^{r_{l,\mathsf{att}}}, K_{4,l,\mathsf{att}} = h^{r_{l,\mathsf{att}}}\right\}_{(\mathsf{lab},\mathsf{att})\in\mathcal{S}_l}\right),$$

$$\mathsf{CT}_{\mathbb{A},+} = \left(\left\{C_{1,j} = h^{\mu_j} \cdot B_{\tilde{\rho}(j)}^{s_j}, C_{2,j} = \left(B_{\tilde{\rho}(j),\mathsf{lab},0} \cdot B_{\tilde{\rho}(j),\mathsf{lab},1}^{x_{\mathsf{att}}}\right)^{s_j}, C_{3,j} = h^{\lambda_j} \cdot A_{\tilde{\rho}(j)}^{s_j}, C_{4,j} = h^{s_j}\right\}_{j\in\Psi}\right),$$

where $\mathcal{S}_l$ denotes the label-attribute pairs managed by authority l, Ψ denotes the rows of the matrix associated with positive literals, $\tilde{\rho}$ maps the rows of the matrix to authorities and λ_j and μ_j are shares of the secrets $\tilde{s}$ and 0 with respect to the policy matrix $\mathbf{A}$. Recall that $B_{l,\mathsf{lab},i}$ is produced by using a full-domain hash by using l, lab and i as inputs (among other things). Roughly, the key component $K_{1,(l,\mathsf{lab},\mathsf{att})}$ ties together the ciphertext components $C_{1,j}, C_{2,j}$ and $C_{3,j}$ and to the user with global identifier GID.

For the "negative part", we use the fact that the attributes are mapped into the group with a 1-degree polynomial, which can be reconstructed when two points on the polynomial are given. That is, suppose that $x_{\rho(j)}$ is negated in the policy (and its authority and label are $\tilde{\rho}(j)$ and $\rho_{\mathsf{lab}}(j)$, respectively). Suppose that $C_{2,j} = \left(\bar{B}_{\tilde{\rho}(j),\mathsf{lab},0} \cdot \bar{B}_{\tilde{\rho}(j),\mathsf{lab},1}^{x_{\rho(j)}}\right)^{s_j}$ and h^{s_j} are provided in the ciphertext, and, for each $(l, \mathsf{lab}, \mathsf{att})$ in the set with $\mathsf{att} \neq \rho(j)$ and $\mathsf{lab} = \rho_{\mathsf{lab}}(j), l = \tilde{\rho}(j)$, $K_{3,(l,\mathsf{lab},\mathsf{att})} = \left(\bar{B}_{l,\mathsf{lab},0} \cdot \bar{B}_{l,\mathsf{lab},1}^{x_{\mathsf{att}}}\right)^{\bar{r}_{l,\mathsf{att}}}$ and $h^{\bar{r}_{l,\mathsf{att}}}$ are provided in the key, then pairing $C_{2,j}$ with $h^{\bar{r}_{l,\mathsf{att}}}$ and $K_{3,(l,\mathsf{lab},\mathsf{att})}$ with h^{s_j} allows us to reconstruct any point on the polynomial, including the coefficients "in the exponent", e.g.,

$$e(\bar{B}_{l,\mathsf{lab},1}, h)^{\bar{r}_{l,\mathsf{att}} s_j} = e(K_{3,(l,\mathsf{lab},\mathsf{att})}, h^{s_j})^{-\frac{1}{x_{\rho(j)} - x_{\mathsf{att}}}} \cdot e(C_{2,j}, h^{\bar{r}_{\mathsf{att}}})^{\frac{1}{x_{\rho(j)} - x_{\mathsf{att}}}}$$

To ensure that each attribute att with the same authority l and label lab as the negated attribute $\rho(j)$ is used in the decryption, we tie the keys together with the randomizer $\bar{r}_{l,\mathsf{lab}} = \sum_{(l',\mathsf{lab}',\mathsf{att})\in\mathcal{S}: l'=l, \mathsf{lab}'=\mathsf{lab}} \bar{r}_{l,\mathsf{att}}$, which sums over all attributes with the same authority and label. These keys are tied together via the key component $K_{2,l,\mathsf{lab}} = g^{\alpha_l} \cdot \mathsf{H}(\mathsf{GID})^{\bar{b}_l} \cdot \bar{B}_{l,\mathsf{lab},1}^{\bar{r}_{l,\mathsf{lab}}}$. Similarly as in the positive part of the key, the GID is used to tie the keys to a single user. In sum, the negative part of the keys and ciphertexts are of the form

$$\mathsf{MPK}_{l,-} = \big(A_l = h^{\alpha_l}, \bar{B}_l = h^{\bar{b}_l}\big),$$

$$\mathsf{SK}_{\mathcal{S}_l,-} = \Bigg(\Big\{K_{2,l,\mathsf{lab}} = g^{\alpha_l} \cdot \mathsf{H}(\mathsf{GID})^{\bar{b}_l} \cdot \bar{B}_{l,\mathsf{lab},1}^{\bar{r}_{l,\mathsf{lab}}}\Big\}_{(\mathsf{lab},\cdot)\in\mathcal{S}_l},$$
$$\Big\{K_{3,(l,\mathsf{lab},\mathsf{att})} = \big(\bar{B}_{l,\mathsf{lab},0} \cdot B_{l,\mathsf{lab},1}^{x_{\mathsf{att}}}\big)^{\bar{r}_{l,\mathsf{att}}}\Big\}_{(\cdot,\mathsf{att})\in\mathcal{S}_l}, \big\{K_{5,l,\mathsf{att}} = h^{\bar{r}_{l,\mathsf{att}}}\big\}_{(\mathsf{lab},\mathsf{att})\in\mathcal{S}_l}\Bigg),$$

$$\mathsf{CT}_{\mathbb{A},-} = \left(\Big\{C_{1,j} = h^{\mu_j} \cdot \bar{B}_{\tilde{\rho}(j)}^{s_j}, C_{2,j} = \big(\bar{B}_{\tilde{\rho}(j),\mathsf{lab},0} \cdot \bar{B}_{\tilde{\rho}(j),\mathsf{lab},1}^{x_{\mathsf{att}}}\big)^{s_j}, C_{3,j} = h^{\lambda_j} \cdot A_{\tilde{\rho}(j)}^{s_j}, C_{4,j} = h^{s_j}\Big\}_{j\in\overline{\Psi}}\right),$$

where $\overline{\Psi}$ are the rows of the policy matrix that are associated with negations.

3.2 The underlying pair encoding scheme

We now give a definition of the PES for the starting point scheme.

Definition 5 (Unbounded decentralized large-universe CP-ABE with negations)**.** Let $\mathcal{L}$ denote a universe of labels and let n_{aut} denote a number of authorities. We define a PES-ISA for the predicate $P_{\text{MA-CP-not}} \colon \mathcal{X}_{\text{MA-CP-not}} \times \mathcal{Y}_{\text{MA-CP-not}} \to \{0,1\}$ as follows.

- Param($\mathcal{L}$): Let $\{\mathcal{A}_l\}_{l\in[n_{\text{aut}}]}$ be the authorities. On input the label universe $\mathcal{L}$, we set $n_\alpha = n_{\text{aut}}$ and $n_b = (2+4|\mathcal{L}|)n_{\text{aut}}$, where $\boldsymbol{\alpha} = \{\alpha_l\}_{l\in[n_{\text{aut}}]}$, and $\mathbf{b} = (\{b_l, \bar{b}_l, \{b_{l,\mathsf{lab},0}, b_{l,\mathsf{lab},1}, \bar{b}_{l,\mathsf{lab},0}, \bar{b}_{l,\mathsf{lab},1}\}_{\mathsf{lab}\in\mathcal{L}}\}_{l\in[n_{\text{aut}}]})$.
- EncKey($\mathcal{S}$): We set $m_1 = 2|\mathcal{S}| + 1$, $m_2 = 0$, and

  $$\mathbf{k} = (\{k_{1,(l,\mathsf{lab},\mathsf{att})} = \alpha_l + r_{\mathsf{GID}} b_l + r_{l,\mathsf{att}}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1}),$$
  $$k_{2,l,\mathsf{lab}} = \alpha_l + r_{\mathsf{GID}}\bar{b}_l + \bar{r}_{l,\mathsf{lab}}\bar{b}_{l,\mathsf{lab},1}, k_{3,(l,\mathsf{lab},\mathsf{att})} = \bar{r}_{l,\mathsf{att}}(\bar{b}_{l,\mathsf{lab},0} + x_{\mathsf{att}}\bar{b}_{l,\mathsf{lab},1})\}_{(l,\mathsf{lab},\mathsf{att})\in\mathcal{S}})$$

  where $\bar{r}_{l,\mathsf{lab}} = \sum_{\substack{(l',\mathsf{lab}',\mathsf{att})\in\mathcal{S}\\ :l'=l,\mathsf{lab}'=\mathsf{lab}}} \bar{r}_{l,\mathsf{att}}$ and x_{att} is the representation of att in $\mathbb{Z}_p$.
- EncCt($\mathbf{A}, \rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho}$): We set $w_1 = n_1$, $w_2 = n_2 - 1$, $w_2' = n_2 - 1$, $c_M = \tilde{s}$,

  $$\mathbf{c} = (\{c_{1,j} = \mu_j + s_j b_{\tilde{\rho}(j)}, c_{2,j} = s_j(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\Psi},$$
  $$\{c_{1,j} = \mu_j + s_j\bar{b}_{\tilde{\rho}(j)}, c_{2,j} = s_j(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\overline{\Psi}})$$

  and $\mathbf{c}' = (\{c_j' = \lambda_j + \alpha_{\tilde{\rho}(j)} s_j\}_{j\in[n_1]})$, where $\lambda_j = A_{j,1}\tilde{s} + \sum_{k\in[2,n_2]} A_{j,k}\hat{v}_k$ and $\mu_j = \sum_{k\in[2,n_2]} A_{j,k}\hat{v}_k'$, and $\Psi = \{j \in [n_1] \mid \rho'(j) = 1\}$ and $\overline{\Psi} = [n_1] \setminus \Psi$ (i.e., the set of rows associated with the positive and negative literals, respectively), and $\mathbf{s} = (\{s_j\}_{[n_1]})$.

The Pair algorithm. We show how the Pair algorithm functions by "simulating" decryption. If $(\mathbf{A}, \rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho}) \models (\mathcal{S}, \tilde{\rho})$, this algorithm determines $\Upsilon = \{j \in \Psi \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}\}$, $\overline{\Upsilon} = \{j \in \overline{\Psi} \mid (\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S} \wedge \exists(\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \mathsf{att}) \in \mathcal{S}\}$ and $\{\varepsilon_j \in \mathbb{Z}_p\}_{j\in\Upsilon\cup\overline{\Upsilon}}$ so that $\sum_{j\in\Upsilon\cup\overline{\Upsilon}} \varepsilon_j\lambda_j = \tilde{s}$ (Definition 1), and computes

$$\sum_{j\in\Upsilon\cup\overline{\Upsilon}} \varepsilon_j c_j' - \sum_{j\in\Upsilon} \varepsilon_j s_j k_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))} + \sum_{j\in\Upsilon} \varepsilon_j \left(r_{\mathsf{GID}} c_{1,j} + r_{\tilde{\rho}(j),\rho(j)} c_{2,j}\right)$$
$$- \sum_{j\in\overline{\Upsilon}} \varepsilon_j \left(s_j k_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)} - r_{\mathsf{GID}} c_{1,j}\right) + \sum_{j\in\overline{\Upsilon}} \varepsilon_j \sum_{(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}} \left(\frac{\bar{r}_{\tilde{\rho}(j),\mathsf{att}} c_{2,j} - s_j k_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}}{x_{\rho(j)} - x_{\mathsf{att}}}\right).$$

Instantiating the scheme with the ISABELLA compiler. We give a full-fledged description of the scheme in Figure 1 for the following choices of $\mathfrak{D}$ and $\mathcal{F}$. We set $\mathcal{F}(b_{l,\mathsf{lab},i}) = (4l+i, (\mathcal{A}_l, \mathsf{lab}, 1, i))$ and $\mathcal{F}(\bar{b}_{l,\mathsf{lab},i}) = (4l+i+2, (\mathcal{A}_l, \mathsf{lab}, 0, i))$ for all $l \in [n_{\text{aut}}], i \in \{0,1\}, \mathsf{lab} \in \mathcal{L}$. We further set $\mathcal{F}(r_{\mathsf{GID}}) = (1, \mathsf{GID})$. Note that we can support a more efficient instantiation by moving the $\boldsymbol{\alpha}$ variables to $\boldsymbol{\beta}$ and the $\mathbf{c}'$ polynomials to $\mathbf{c}''$. To optimize the encryption and decryption efficiency, we map $k_{1,(l,\mathsf{lab},\mathsf{att})}, k_{2,l,\mathsf{lab}}, k_{3,(l,\mathsf{lab},\mathsf{att})}, r_{\mathsf{GID}}, c_{2,j}$ (and therefore also $b_{l,\mathsf{lab},i}, \bar{b}_{l,\mathsf{lab},i}$) to $\mathbb{G}$ with $\mathfrak{D}$, and $r_{l,\mathsf{att}}, \bar{r}_{l,\mathsf{att}}, c_{1,j}, c'_j, s_j$ to $\mathbb{H}$. Our choice of distribution is based on the results in ABE Squared [dlPVA22]. Specifically, because hashing in $\mathbb{H}$ is very computationally expensive, we ensure that the hashes need to be computed in $\mathbb{G}$.

Implementing the hashes using domain separation. The above description for the hash inputs are close to the mathematical description and take domain separation into account. However, an authority is represented by both an index in $[n_{\text{aut}}]$ in a short interval (i.e., the number of authorities) and an identifier, which can be any string from an authority identifier space, e.g., $\{0,1\}^{256}$. In practice, we recommend the use of authority identifier rather than indices, so we ensure that our implementation uses identifiers, both as inputs to the schemes and as inputs to the hashes. To this end, we change the behavior of the hash inputs of the scheme so that it is closer to the practical implementation of the scheme. Like the mathematical description, we also apply domain separation, but we replace indices (in small intervals) to identifiers where possible:

PES variable	**Mathematical description**	**Practical description**
r_{GID}	$(1, \mathsf{GID})$	$(\text{“GID”}, \mathsf{GID})$
$b_{l,\mathsf{lab},i}$	$(4l+i, (\mathcal{A}_l, \mathsf{lab}, 1, i))$	$(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“POS”}, i))$
$\bar{b}_{l,\mathsf{lab},i}$	$(4l+i+2, (\mathcal{A}_l, \mathsf{lab}, 1, i))$	$(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“NEG”}, i))$

We assume that “GID”, “AID”, $\mathcal{A}_l$, lab are represented by 256 bits (possibly using a hash function applied to descriptions of $\mathcal{A}_l$ or lab that are longer than 256 bits), and “POS”, “NEG”, $i \in \{0,1\}$ are each represented by 8 bits (i.e., one byte).

Security. We recall the security proof of the starting point scheme in Appendix A.2 since it serves as a basis for the security proofs of our improvements. More specifically, we give the proof for the SSSP which implies static security under a q-type assumption in the random oracle model and adaptive security in the generic group and random oracle model.

4 Improvements to the starting point scheme

We make two types of structural improvements to optimize the starting point scheme. The first type of optimization is to re-use randomness where this is possible. This is more efficient, because the pairing operations required during decryption depend on the number of randomizers used in the scheme. Hence, reducing the number of randomizers also reduces the number of pairing operations we need to decrypt. As it turns out, the current construction does not allow for an optimal re-use of randomness. Therefore, as a second type of structural improvement, we also introduce new randomness-splitting techniques that allow us to change the structure of the scheme to make it more amenable to re-use the randomness. We also show how to benefit from the combined use of randomness re-use and randomness-splitting techniques by rearranging the pairing operations during decryption. In the following subsections, we mainly focus on the structural changes. We discuss the associated effects on the computational costs in Sections 5 and 8.

$\underline{\mathsf{GlobalSetup}}$. Pick a pairing group $\mathcal{PG} \coloneqq (\mathbb{G}, \mathbb{H}, \mathbb{G}_T, p, e, g, h)$ and hash function $\mathsf{H}\colon \{0,1\}^* \to \mathbb{G}$. Define $\mathrm{GP} \coloneqq (\mathcal{PG}, \mathsf{H})$.

$\underline{\mathsf{AuthoritySetup}(\mathrm{GP}, \mathcal{A}_l)}$. Pick $\alpha_l, b_l, \bar{b}_l \in_R \mathbb{Z}_p$. Compute the master public key of authority $\mathcal{A}_l$ as

$$\mathsf{MPK}_{\mathcal{A}_l} \coloneqq (\mathrm{GP}, A_l = h^{\alpha_l}, B_l = h^{b_l}, \bar{B}_l = h^{\bar{b}_l}).$$

Let $\mathsf{MSK}_{\mathcal{A}_l} \coloneqq (\alpha_l, b_l, \bar{b}_l)$ be the master secret key of authority $\mathcal{A}_l$.

$\underline{\mathsf{KeyGen}(\mathcal{A}_l, \mathsf{MSK}_{\mathcal{A}_l}, \mathsf{GID}, \mathcal{S}_{\mathcal{A}_l} \subseteq \mathcal{U}_{\mathcal{A}_l})}$. Pick $r_{l,\mathsf{att}}, \bar{r}_{l,\mathsf{att}} \in_R \mathbb{Z}_p$ for all $(\mathsf{lab}, \mathsf{att}) \in \mathcal{S}_{\mathcal{A}_l}$, set $\bar{r}_{l,\mathsf{lab}} = \sum_{(\mathsf{lab}',\mathsf{att}) \in \mathcal{S}_{\mathcal{A}_l} : \mathsf{lab}' = \mathsf{lab}} \bar{r}_{l,\mathsf{att}}$, then compute for each $(\mathsf{lab}, \mathsf{att}) \in \mathcal{S}_{\mathcal{A}_l}$:

$$K_{1,(l,\mathsf{lab},\mathsf{att})} = g^{\alpha_l} \cdot \mathsf{H}(1, \mathsf{GID})^{b_l} \cdot (\mathsf{H}(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“POS”}, 0) \cdot \mathsf{H}(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“POS”}, 1)^{x_{\mathsf{att}}})^{r_{l,\mathsf{att}}},$$
$$K_{2,l,\mathsf{lab}} = g^{\alpha_l} \cdot \mathsf{H}(1, \mathsf{GID})^{\bar{b}_l} \cdot \mathsf{H}(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“NEG”}, 1)^{\bar{r}_{l,\mathsf{lab}}},$$
$$K_{3,(l,\mathsf{lab},\mathsf{att})} = (\mathsf{H}(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“NEG”}, 0) \cdot \mathsf{H}(\text{“AID”}, \mathcal{A}_l, \mathsf{lab}, \text{“NEG”}, 1)^{x_{\mathsf{att}}})^{\bar{r}_{l,\mathsf{att}}},$$
$$\check{K}_{1,l,\mathsf{att}} = h^{r_{l,\mathsf{att}}}, \qquad \check{K}_{2,l,\mathsf{att}} = h^{\bar{r}_{l,\mathsf{att}}}.$$

Output $\mathsf{SK}_{\mathcal{A}_l,\mathsf{GID}} \coloneqq (\{K_{1,(l,\mathsf{lab},\mathsf{att})}, K_{2,l,\mathsf{lab}}, K_{3,(l,\mathsf{lab},\mathsf{att})}, \check{K}_{1,l,\mathsf{att}}, \check{K}_{2,l,\mathsf{att}}\}_{(\mathsf{lab},\mathsf{att}) \in \mathcal{S}_{\mathcal{A}_l}})$.

$\underline{\mathsf{Encrypt}(\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}\}_{l \in \tilde{\rho}([n_1])}, (\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}))}$. Let $\mathsf{MPK}_{\mathcal{A}_l} = (\mathrm{GP}, A_l, B_l, \bar{B}_l)$. Pick $\tilde{s} \in_R \mathbb{Z}_p$, $\mathbf{s} \in_R \mathbb{Z}_p^{n_1}$, $\mathbf{v}, \mathbf{v}' \in_R \mathbb{Z}_p^{n_2-1}$. For each row $j \in [n_1]$ with $\rho'(j) = 1$, compute

$$C_{1,j} = h^{\mu_j} \cdot B_{\tilde{\rho}(j)}^{s_j}, \qquad C'_j = h^{\lambda_j} \cdot A_{\tilde{\rho}(j)}^{s_j}, \qquad \check{C}_j = h^{s_j}$$
$$C_{2,j} = \left(\mathsf{H}(\text{“AID”}, \mathcal{A}_{\tilde{\rho}(j)}, \rho_{\mathsf{lab}}(j), \text{“POS”}, 0) \cdot \mathsf{H}(\text{“AID”}, \mathcal{A}_{\tilde{\rho}(j)}, \rho_{\mathsf{lab}}(j), \text{“POS”}, 1)^{x_{\rho(j)}}\right)^{s_j},$$

and for each row $j \in [n_1]$ with $\rho'(j) = 0$, compute

$$C_{1,j} = h^{\mu_j} \cdot \bar{B}_{\tilde{\rho}(j)}^{s_j}, \qquad C'_j = h^{\lambda_j} \cdot A_{\tilde{\rho}(j)}^{s_j}, \qquad \check{C}_j = h^{s_j}$$
$$C_{2,j} = \left(\mathsf{H}(\text{“AID”}, \mathcal{A}_{\tilde{\rho}(j)}, \rho_{\mathsf{lab}}(j), \text{“NEG”}, 0) \cdot \mathsf{H}(\text{“AID”}, \mathcal{A}_{\tilde{\rho}(j)}, \rho_{\mathsf{lab}}(j), \text{“NEG”}, 1)^{x_{\rho(j)}}\right)^{s_j},$$

where $\lambda_j = \mathbf{A}_j(\tilde{s} \| \mathbf{v})^\top$ and $\mu_j = \mathbf{A}_j(0 \| \mathbf{v}')^\top$. Further, compute $K \coloneqq e(g,h)^{\tilde{s}}$ and output (K, CT) with $\mathsf{CT} \coloneqq (\{C_{1,j}, C_{2,j}, C'_j, \check{C}_j\}_{j \in [n_1]})$.

$\underline{\mathsf{Decrypt}(\{\mathcal{A}_l, \mathsf{MPK}_{\mathcal{A}_l}, \mathsf{SK}_{\mathcal{A}_l,\mathsf{GID}}\}_l, \mathsf{CT})}$. If $(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \models \mathcal{S}$, this algorithm determines $\Upsilon = \{j \in \Psi \mid (\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_{\tilde{\rho}(j)}\}$, $\overline{\Upsilon} = \{j \in \overline{\Psi} \mid (\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_{\tilde{\rho}(j)} \wedge \exists (\rho_{\mathsf{lab}}(j), \mathsf{att}) \in \mathcal{S}_{\tilde{\rho}(j)}\}$ and $\{\varepsilon_j \in \mathbb{Z}_p\}_{j \in \Upsilon \cup \overline{\Upsilon}}$ so that $\sum_{j \in \Upsilon \cup \overline{\Upsilon}} \varepsilon_j \mathbf{A}_j = (1, 0, ..., 0)$. Let $\mathsf{SK}_{\mathcal{A}_l,\mathsf{GID}} = (\{K_{1,(l,\mathsf{lab},\mathsf{att})}, K_{2,l,\mathsf{lab}}, K_{3,(l,\mathsf{lab},\mathsf{att})}, \check{K}_{1,l,\mathsf{att}}, \check{K}_{2,l,\mathsf{att}}\}_{(\mathsf{lab},\mathsf{att}) \in \mathcal{S}_{\mathcal{A}_l}})$. Output

$$K \coloneqq e\left(g, \prod_{j \in \Upsilon \cup \overline{\Upsilon}} (C'_j)^{\varepsilon_j}\right) \cdot \prod_{j \in \Upsilon} e(K_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j}, \check{C}_j) \cdot e\left(\mathsf{H}(1, \mathsf{GID}), \prod_{j \in \Upsilon \cup \overline{\Upsilon}} C_{1,j}^{\varepsilon_j}\right)$$
$$\cdot \prod_{j \in \Upsilon} e(C_{2,j}^{\varepsilon_j}, \check{K}_{1,\tilde{\rho}(j),\rho(j)}) \cdot \prod_{j \in \overline{\Upsilon}} e(K_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j}, \check{C}_j)$$
$$\cdot \prod_{j \in \overline{\Upsilon}} \left(e\left(C_{2,j}, \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att}) \in \mathcal{S}_{\tilde{\rho}(j)}} \check{K}_{2,\tilde{\rho}(j),\mathsf{att}}^{\frac{\varepsilon_j}{x_{\mathsf{att}} - x_{\rho(j)}}}\right) \cdot e\left(\prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att}) \in \mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}} - x_{\rho(j)}}}, \check{C}_j\right)\right)$$

Figure 1: The decentralized CP-ABE scheme for span programs ($\mathbf{A} \in \mathbb{Z}_p^{n_1 \times n_2}, \rho\colon [n_1] \to \mathcal{U}, \rho'\colon [n_1] \to \{0,1\}, \rho_{\mathsf{lab}}\colon [n_1] \to \mathcal{L}, \tilde{\rho} : [n_1] \to [n_{\mathrm{aut}}]$) based on the PES-ISA in Definition 5, instantiated with the ISABELLA compiler. Note that we have capitalized the key and ciphertext components (where their PES counterparts were lower-cased) while preserving as much of the indexing strategy as possible. The key and ciphertext components associated with non-lone variables $r_{l,\mathsf{att}}$, $\bar{r}_{l,\mathsf{att}}$ and s_j are “checked”, i.e., $\check{K}_{1,l,\mathsf{att}}$, $\check{K}_{2,l,\mathsf{att}}$ and $\check{C}_j$.

4.1 Randomness re-use and optimizing the key randomness

First, we apply the randomness re-use techniques from [AC17] to reduce the number of randomizers in the keys. Roughly, the idea is that we can use the same randomizers for

attributes with different labels. To this end, we introduce the mapping $\iota_l \colon \mathcal{S}_l \to [m_l]$, where m_l denotes the maximum number of attributes att per label managed by the same authority l. In particular, we require that ι_l is injective on each subdomain $\mathcal{S}_{l,\mathsf{lab}} = \{(\mathsf{lab}', \mathsf{att}) \mid (\mathsf{lab}', \mathsf{att}) \in \mathcal{S}_l \wedge \mathsf{lab}' = \mathsf{lab}\}$. As a result, we generally require fewer pairing operations during decryption. For example, if each label is used only once (like in TKN20 [TKN20] and its decentralized variant [Ven23, §5.1]), we require only one randomizer, and therefore only require one pairing for the fourth term in the pairing product in the decryption algorithm. The key polynomials $k_{1,(l,\mathsf{lab},\mathsf{att})}$ would be, using the function ι_l:

$$k_{1,(l,\mathsf{lab},\mathsf{att})} = \alpha_l + r_{\mathsf{GID}} b_l + r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1}).$$

For the key polynomials $k_{3,(l,\mathsf{lab},\mathsf{att})}$, we obtain:

$$k_{3,(l,\mathsf{lab},\mathsf{att})} = \bar{r}_{l,\iota_l(\mathsf{lab},\mathsf{att})}(\bar{b}_{l,\mathsf{lab},0} + x_{\mathsf{att}} \bar{b}_{l,\mathsf{lab},1})$$

so that $\bar{r}_{l,\mathsf{lab}} = \sum_{(\mathsf{lab}',\mathsf{att}) \in \mathcal{S} : \mathsf{lab}' = \mathsf{lab} \wedge \tilde{\rho}_{\mathcal{S}}(\mathsf{att}) = l} \bar{r}_{l,\iota_l(\mathsf{lab},\mathsf{att})}$ as used in $k_{2,l,\mathsf{lab}}$.

Lastly, note that the number of non-lone variables is then also reduced to the number of such variables that we need, i.e., $\mathbf{r} = (\{r_{l,\ell}, \bar{r}_{l,\ell}\}_{\ell \in [m_l]})$. The Pair algorithm is also affected, i.e., instead of pairing $r_{\tilde{\rho}(j),\rho(j)}$ with $c_{2,j}$, we have to pair $r_{\tilde{\rho}(j),\iota_{\tilde{\rho}}(\rho_{\mathsf{lab}}(j),\rho(j))}$ with $c_{2,j}$, and instead of pairing $\bar{r}_{\tilde{\rho}(j),\rho(j)}$ with $c_{2,j}$, we have to pair $\bar{r}_{\tilde{\rho}(j),\iota_{\tilde{\rho}}(\rho_{\mathsf{lab}}(j),\rho(j))}$ with $c_{2,j}$.

4.2 Randomness re-use for the ciphertexts

We also apply randomness re-use in the ciphertexts. We show that, in its current state, the scheme does not have a suitable structure to optimally re-use randomizers. This is because of correctness rather than security. More specifically, the ciphertexts are of the form:

$$\begin{gathered}\mathbf{c} = (\{c_{1,j} = \mu_j + s_j b_{\tilde{\rho}(j)}, c_{2,j} = s_j(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j \in \Psi}, \\ \{c_{1,j} = \mu_j + s_j \bar{b}_{\tilde{\rho}(j)}, c_{2,j} = s_j(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} \bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j \in \overline{\Psi}}), \\ \mathbf{c}' = (\{c'_j = \lambda_j + \alpha_{\tilde{\rho}(j)} s_j\}_{j \in [n_1]}),\end{gathered}$$

where the same randomizer s_j is needed for all components $c_{1,j}$, $c_{2,j}$ and c'_j. This is because decryption requires us to combine s_j with $k_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}$ to cancel out the appropriate terms. The master-key and common variables that occur in

$$k_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))} = \alpha_{\tilde{\rho}(j)} + r_{\mathsf{GID}} b_{\tilde{\rho}(j)} + r_{\tilde{\rho}(j),\rho(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})$$

are $\alpha_{\tilde{\rho}(j)}, b_{\tilde{\rho}(j)}, b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0}$ and $b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1}$. This means that, on the side of the ciphertext polynomials, we also need to combine the same randomizer with these master-key and common variables to ensure that decryption cancels out those terms.

Required randomness. From the security perspective, we require the following amount of randomness for the ciphertexts. For the components $c_{1,j}$ and c'_j, we need a fresh randomizer for each label-attribute pair with the same authority to hide λ_j and μ_j, i.e., we can re-use the same randomizers among different authorities. To this end, we introduce a mapping $\tau_{\tilde{\rho}} \colon [n_1] \to [m_{\tilde{\rho}}]$, where $m_{\tilde{\rho}}$ is the maximum number of rows associated with an authority, i.e., $m_{\tilde{\rho}} = \max_{l \in [n_{\mathrm{aut}}]} |\{j \in [n_1] \mid \tilde{\rho}(j) = l\}|$. The mapping $\tau_{\tilde{\rho}}$ maps each row to different integers in $[m_{\tilde{\rho}}]$ for rows that are associated with the same authority, i.e., it is injective on the subdomain $\{j \in [n_1] \mid \tilde{\rho}(j) = l\}$ for each authority $\mathcal{A}_l$. For the components $c_{2,j}$, we technically require fewer randomizers, because $b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),i}$ provide fresh randomness for each authority-label pair. However, to maintain correctness, we cannot make use of this without additional structural changes, as addressed later with randomness splitting.

Randomness re-use for the starting point scheme. Here, we reduce the number of randomizers using $\tau_{\tilde{\rho}}$. Therefore, the randomness re-use variant of the starting point scheme has ciphertexts of the following form

$$\begin{aligned}\mathbf{c} = (\{c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)}, c_{2,j} = s_{\tau_{\tilde{\rho}}(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\Psi},\\ \{c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)} \bar{b}_{\tilde{\rho}(j)}, c_{2,j} = s_{\tau_{\tilde{\rho}}(j)}(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} \bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\overline{\Psi}}),\\ \mathbf{c}' = (\{c'_j = \lambda_j + \alpha_{\tilde{\rho}(j)} s_{\tau_{\tilde{\rho}}(j)}\}_{j\in[n_1]}).\end{aligned}$$

Note that the number of non-lone variables is then also reduced to the number of such variables that we need, i.e., $\mathbf{s} = (\{s_\ell\}_{\ell\in[m_{\tilde{\rho}}]})$. The Pair algorithm is also affected, i.e., instead of pairing s_j with the key polynomials, we have to use $s_{\tau_{\tilde{\rho}}(j)}$.

4.3 Randomness splitting to improve ciphertext randomness re-use

To further reduce the number of randomizers in the ciphertexts, we also need to make structural changes. Currently, the keys are tied together in such a way that we require the same randomness s_j to be used among all ciphertext components, and we require fresh randomness to ensure that μ_j and λ_j are sufficiently hidden. To enable this, we propose new randomness-splitting techniques that split the randomness used in the key and ciphertext components in such a way that correctness does not require the same randomizer s_j to be used among all ciphertext components.

Further, we introduce the mapping $\tau\colon [n_1] \to [m]$, where $m = \max_{(\mathsf{lab},l)\in\mathcal{L}\times[n_{\mathrm{aut}}]} |\{j \in [n_1] \mid \rho_{\mathsf{lab}}(j) = \mathsf{lab}, \tilde{\rho}(j) = l\}|$ is the maximum number of rows j that are mapped to the same label-authority pair $(\mathsf{lab}, l) \in \mathcal{L} \times [n_{\mathrm{aut}}]$ with ρ_{lab} and $\tilde{\rho}$. The mapping τ is injective on each subdomain $\{j \in [n_1] \mid \rho_{\mathsf{lab}}(j) = \mathsf{lab}, \tilde{\rho}(j) = l\}$ (for each $(\mathsf{lab}, l) \in \mathcal{L} \times [n_{\mathrm{aut}}]$).

Randomness splitting for the "positive parts". We first consider how to split the keys and ciphertexts for the positive literals $j \in \Psi$ in the ciphertext, and the associated part in the keys (i.e., $k_{1,(l,\mathsf{lab},\mathsf{att})}$). We replace $k_{1,(l,\mathsf{lab},\mathsf{att})}$ by

$$k_{1,1,l,\ell} = \alpha_l + r_{\mathsf{GID}} b_l + r_{l,\ell} b'_l, \quad k_{1,2,(l,\mathsf{lab},\mathsf{att})} = r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1})$$

where $\ell \in [m_l]$, and $c_{1,j}$ and $c_{2,j}$ by

$$c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)}, \quad c_{2,j} = s_{\tau_{\tilde{\rho}}(j)} b'_{\tilde{\rho}(j)} + s_{\tau(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1}).$$

Note that, because these new key and ciphertext components use new common variables b'_l, we need to add these to the set $\mathbf{b}$ as well. Furthermore, the number of non-lone ciphertext variables remains the same, i.e., $\mathbf{s} = (\{s_\ell\}_{\ell\in[\max(m_{\tilde{\rho}},m)]})$, where $\max(m_{\tilde{\rho}}, m) = m_{\tilde{\rho}}$. The Pair algorithm is also affected. Instead of pairing $k_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}$ with $s_{\tau_{\tilde{\rho}}(j)}$, we have to pair $k_{1,1,\tilde{\rho}(j),\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}$ with $s_{\tau_{\tilde{\rho}}(j)}$ and $k_{1,2,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}$ with $s_{\tau(j)}$.

Randomness splitting for the "negative parts". For the negative parts, i.e., for $c_{1,j}, c_{2,j}$ with $j \in \overline{\Psi}$ and $k_{2,l,\mathsf{lab}}$ and $k_{3,(l,\mathsf{lab},\mathsf{att})}$, we follow a more involved approach. In particular, we need to ensure that $c_{2,j}$ and $k_{3,(l,\mathsf{lab},\mathsf{att})}$ remain structurally the same while providing randomness re-use techniques, i.e., we can change the randomizers of those but not add more terms. To do this, we use a similar approach as for the positive parts, but twice: once on the key side and once on the ciphertext side. Hence, we replace $k_{2,l,\mathsf{lab}}$ by

$$k_{2,1,l} = \alpha_l + r_{\mathsf{GID}} \bar{b}_l + \bar{r}_l \bar{b}'_l, \quad k_{2,2,l,\mathsf{lab}} = \bar{r}_l \bar{b}'_{l,\mathsf{lab}} + \bar{r}_{l,\mathsf{lab}} \bar{b}_{l,\mathsf{lab},1},$$

where $\bar{r}_{l,\mathsf{lab}} = \sum_{(\mathsf{lab}',\mathsf{att}')\in\mathcal{S}_l | \mathsf{lab}'=\mathsf{lab}} \bar{r}_{l,\iota_l(\mathsf{lab}',\mathsf{att}')}$, and the ciphertexts $c_{1,j}$ and $c_{2,j}$ by

$$c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)} \bar{b}_{\tilde{\rho}(j)}, \quad c_{2,j} = s_{\tau_{\tilde{\rho}}(j)} \bar{b}'_{\tilde{\rho}(j)} + s_{\tau(j)} \bar{b}'_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)},$$

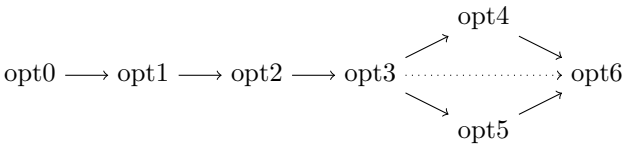


Figure 2: An overview of the optimization candidates implemented and analyzed in this work, and how they build on one another.

$$c_{3,j} = s_{\tau(j)}(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1}).$$

Note that, because these new key and ciphertext components use new common variables $\bar{b}'_l$ and $\bar{b}'_{l,\mathsf{lab}}$, we need to add these to the set $\mathbf{b}$ as well. The variables $\bar{b}'_{l,\mathsf{lab}}$ are generated implicitly using an FDH. Furthermore, the number of non-lone ciphertext variables remains the same, i.e., $\mathbf{s} = (\{s_\ell\}_{\ell\in[\max(m_{\tilde{\rho}},m)]})$, and the number of non-lone key variables is increased, i.e., $\bar{r}_l$ are added. The Pair algorithm is also affected. Instead of pairing $k_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}$ with $s_{\tau_{\tilde{\rho}}(j)}$, we have to pair $k_{2,1,\tilde{\rho}(j)}$ with $s_{\tau_{\tilde{\rho}}(j)}$ and $k_{2,2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}$ with $s_{\tau(j)}$. Further, instead of pairing $c_{2,j}$ with $\bar{r}_{\tilde{\rho}(j),\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}$, we pair $c_{2,j}$ with $\bar{r}_{\tilde{\rho}(j)}$ and $c_{3,j}$ with $\bar{r}_{\tilde{\rho}(j),\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}$. Lastly, instead of pairing $r_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}$ with $s_{\tau_{\tilde{\rho}}(j)}$, we pair it $s_{\tau(j)}$.

Remark 2. In the variants using randomness splitting in the negative parts, we do not have a strict improvement for the required number of pairing operations compared to the variants not using randomness splitting, like in the positive parts. The reason is that we need an extra product involving the pairing operations for $c_{2,j}$ and $\bar{r}_{\tilde{\rho}(j)}$. This product is only efficient if the number of authorities associated with the policy is low.

4.4 Summary of our optimizations

We implement the following versions based on the starting point scheme and our suggested improvements, as summarized below. Their dependencies are depicted in Figure 2.

- **opt0** is the baseline implementation of the starting point scheme (Figure 1)
- **opt1** is the version of opt0 that uses more grouping of the pairing operations
- **opt2** optimizes opt1 by using randomness re-use for the keys
- **opt3** optimizes opt2 by using randomness re-use for the ciphertexts
- **opt4** changes opt3 by using randomness splitting for the positive parts
- **opt5** changes opt3 by using randomness splitting for the negative parts
- **opt6** changes opt3 by using randomness splitting for the positive and negative parts

We also give security proofs for all schemes via the SSSP in Appendix A.3. Specifically, we provide a full proof for opt6 and explain how the proofs can be constructed for the other optimizations.

4.5 Compiling the optimizations

We use similar configurations for the PESs associated with the optimizations as in the starting point scheme, i.e., we have the following choices of $\mathfrak{D}$ and $\mathcal{F}$. We set $\mathcal{F}(b_{l,\mathsf{lab},i}) = (4l+i,(\mathcal{A}_l,\mathsf{lab},1,i))$ and $\mathcal{F}(\bar{b}_{l,\mathsf{lab},i}) = (4l+i+2,(\mathcal{A}_l,\mathsf{lab},0,i))$ for all $l \in [n_{\mathrm{aut}}], i \in \{0,1\}, \mathsf{lab} \in \mathcal{L}$. We further set $\mathcal{F}(r_{\mathsf{GID}}) = (1,\mathsf{GID})$. We also use domain separation to implement the hashes in practice, similarly as for the original scheme. Additionally, for opt5

and opt6, we have additional hashes to implement the group elements associated with $\bar{b}'_{l,\mathsf{lab}}$. For these, we use the practical description ("AID", $\mathcal{A}_l$, lab, "NEG", 2). Note that we can support a more efficient instantiation by moving the $\boldsymbol{\alpha}$ variables to $\boldsymbol{\beta}$ and the $\mathbf{c}'$ polynomials to $\mathbf{c}''$ (without altering the naming of the variables and polynomials). Furthermore, we use the same notational convention for the compiled schemes as in the starting point scheme, using the same indexing strategies as in the associated PES, and distinguishing the key and ciphertext components associated with non-lone variables from those associated with polynomials with accents, e.g., $\check{K}_j$ and $\check{C}_j$. To optimize the encryption and decryption efficiency, use the same distributions of the key and ciphertext components as the original scheme. For the key and ciphertext components that are replaced by multiple components, we use the same group distributions for the replacements as for the original encodings. We have also considered other distributions for these replacements, but the use of full-domain hashes restricts the schemes to the same two distributions as the starting point scheme.

4.6 The decryption strategy

We summarize the decryption strategy for all schemes, so that we can systematically optimize each version fairly. Our high-level approach is as follows: we apply the same order of pairing operations for the pairing products that are the same. For those that differ, we ensure that we group the pairings on the side that is the most likely to be the most efficient. With grouping, we mean that we leverage the bilinearity property in products that have the same argument on one side, e.g., $\prod_{j\in\Upsilon\cup\overline{\Upsilon}} e\left(\mathsf{H}(1,\mathsf{GID}), C_{1,j}^{\varepsilon_j}\right) = e\left(\mathsf{H}(1,\mathsf{GID}), \prod_{j\in\Upsilon\cup\overline{\Upsilon}} C_{1,j}^{\varepsilon_j}\right)$. In this example, we group the ciphertext components $C_{1,j}$, and these are on the "polynomial side", because $C_{1,j}$ has a polynomial in the exponent. For the versions not involving randomness-splitting (i.e., opt0-opt3), this grouping is almost always on the "polynomial side" of the pairing instead of the "singleton variable side". For the versions that involve randomness splitting, we use a combined strategy that we explain in more detail below.

The "fixed" pairings: the first, third and sixth pairing products. First, we consider the pairing products that are the same among optimizations opt1-opt6:

$$e\left(g, \prod_{j\in\Upsilon\cup\overline{\Upsilon}} (C'_j)^{\varepsilon_j}\right) \text{ and } e\left(\mathsf{H}(1,\mathsf{GID}), \prod_{j\in\Upsilon\cup\overline{\Upsilon}} C_{1,j}^{\varepsilon_j}\right).$$

Note that these were already optimized in the original description of the scheme, i.e., opt0, and these will thus be the same in all optimizations. Further, we optimize the sixth pairing product (for opt1-opt2) as follows:

$$\prod_{j\in\overline{\Upsilon}} e\left(C_{2,j}, \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} \check{K}_{2,\tilde{\rho}(j),\mathsf{att}}^{\frac{\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}\right)$$

$$= \prod_{l\in\tilde{\rho}(\overline{\Upsilon}),\mathsf{lab}\in\rho_{\mathsf{lab}}(\overline{\Upsilon}),\mathsf{att}'\in\rho(\overline{\Upsilon})} e\left(\prod_{j\in\overline{\Upsilon}:\tilde{\rho}(j)=l,\rho_{\mathsf{lab}}(j)=\mathsf{lab},\rho(j)=\mathsf{att}'} C_{2,j}, \prod_{(\mathsf{lab},\mathsf{att})\in\mathcal{S}_l} \check{K}_{2,l,\mathsf{att}}^{\frac{\varepsilon_j}{x_{\mathsf{att}}-x_{\mathsf{att}'}}}\right),$$

and similarly for opt3-opt6, we replace the above index att in $\check{K}_{2,l,\mathsf{att}}$ by $\iota_l(\mathsf{lab},\mathsf{att})$:

$$\prod_{l\in\tilde{\rho}(\overline{\Upsilon}),\mathsf{lab}\in\rho_{\mathsf{lab}}(\overline{\Upsilon}),\mathsf{att}'\in\rho(\overline{\Upsilon})} e\left(\prod_{j\in\overline{\Upsilon}:\tilde{\rho}(j)=l,\rho_{\mathsf{lab}}(j)=\mathsf{lab},\rho(j)=\mathsf{att}'} C_{2,j}, \prod_{(\mathsf{lab},\mathsf{att})\in\mathcal{S}_l} \check{K}_{2,l,\iota_l(\mathsf{lab},\mathsf{att})}^{\frac{\varepsilon_j}{x_{\mathsf{att}}-x_{\mathsf{att}'}}}\right),$$

Note that, despite the fact that we have the same $\check{K}_{2,l,\iota_l(\mathsf{lab},\mathsf{att})}$ for multiple attributes, we cannot use existing techniques to benefit from shared-argument pairings, due to the nested loop with exponentiations. Given that this product is roughly half of the pairing operations required for the negative literals, we would expect not more than 50% speedup.

The "fourth" pairing product. The pairing product associated with the fourth pairing product $\prod_{j\in\Upsilon} e(C_{2,j}^{\varepsilon_j}, \check{K}_{1,\tilde{\rho}(j),\rho(j)})$ is optimized in the same way for all variants using randomness re-use that do not use randomness splitting in the negative parts, i.e., opt2-opt4. (For the variants using randomness splitting in the negative parts, we use the same approach, but note that $C_{3,j}$ is used.) In particular, the key components in this pairing are replaced by $\check{K}_{1,\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}$, where $\iota_{\tilde{\rho}(j)}$ maps attributes with different labels to the same set of randomizers. This means that the number of different key components $\check{K}_{1,\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}$ is upper-bounded in the maximum number of attributes with the same authority-label pair. Thus, if we group on the ciphertext side for this pairing product, we upper-bound the number of pairing operations by this maximum as well, i.e., we compute

$$\prod_{j\in\Upsilon} e\left(C_{2,j}^{\varepsilon_j}, \check{K}_{1,\tilde{\rho}(j),\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}\right) = \prod_{l\in\tilde{\rho}(\Upsilon),\ell\in[m_l]} e\left(\prod_{j\in\Upsilon:\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))=\ell} C_{2,j}^{\varepsilon_j}, \check{K}_{1,l,\ell}\right).$$

(We do the same for the variants using randomness splitting in the negative parts, i.e., opt5 and opt6, except using $C_{3,j}$ instead of $C_{2,j}$.) For opt1, we also group on the ciphertext side, i.e., we compute

$$\prod_{l\in\tilde{\rho}(\Upsilon),\mathsf{att}\in\rho(\Upsilon)} e\left(\prod_{j\in\Upsilon:\tilde{\rho}(j)=l\wedge\rho(j)=\mathsf{att}} C_{2,j}^{\varepsilon_j}, \check{K}_{1,l,\mathsf{att}}\right),$$

where the number of pairings grows in the number of distinct authority-attribute pairs.

The "second" pairing product. The pairing products that replace the second pairing product of the starting-point scheme, i.e., $\prod_{j\in\Upsilon} e(K_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j}, \check{C}_j)$ is computed in various ways. For opt1 and opt2, we group on the ciphertext side, i.e., when j is mapped to the same authority-label-attribute triple, i.e.,

$$\prod_{l\in\tilde{\rho}(\Upsilon),\mathsf{lab}\in\rho_{\mathsf{lab}}(\Upsilon),\mathsf{att}\in\rho(\Upsilon)} e\left(K_{1,(l,\mathsf{lab},\mathsf{att})}, \prod_{j\in\Upsilon:\tilde{\rho}(j)=l,\rho_{\mathsf{lab}}(j)=\mathsf{lab},\rho(j)=\mathsf{att}} \check{C}_j^{-\varepsilon_j}\right).$$

Then, we note that opt3 and opt5 compute the following pairing product:

$$\prod_{\ell\in[m_{\tilde{\rho}}]} e\left(\prod_{j\in\Upsilon:\tau_{\tilde{\rho}}(j)=\ell} K_{1,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j}, \check{C}_\ell\right),$$

which reduces the number of pairing operations from the number of matching rows $|\Upsilon|$ to the number of "randomizers" ℓ that are selected with the rows $j \in \Upsilon$ under $\tau_{\tilde{\rho}}$.

For opt4 and opt6, we compute the product in either of two ways. The first option is to do something similar as for opt3 and opt5, and interleave the two pairing products as

$$\prod_{\ell\in[m_{\tilde{\rho}}]} e\left(\prod_{j\in\Upsilon:\tau_{\tilde{\rho}}(j)=\ell} K_{1,1,\tilde{\rho}(j),\iota_{\tilde{\rho}(j)}(\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j} \cdot \prod_{j\in\Upsilon:\tau(j)=\ell} K_{1,2,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j}, \check{C}_\ell\right).$$

which reduces the number of pairing operations from the number of matching rows $|\Upsilon|$ to the number of "randomizers" ℓ that are selected with the rows $j \in \Upsilon$ under $\tau_{\tilde{\rho}}$ or τ. Alternatively, when the number of authorities used in the policy is low, we can compute:

$$\prod_{l\in\tilde{\rho}(\Upsilon),\ell\in[m_l]} e\left(K_{1,1,l,\ell}, \prod_{\substack{j\in\Upsilon:\tilde{\rho}(j)=l \\ \wedge\iota_l(\rho_{\mathsf{lab}}(j),\rho(j))=\ell}} \check{C}_{\tau_{\tilde{\rho}}(j)}^{-\varepsilon_j}\right) \cdot \prod_{\ell\in[m]} e\left(\prod_{j\in\Upsilon:\tau(j)=\ell} K_{1,2,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\rho(j))}^{-\varepsilon_j}, \check{C}_\ell\right),$$

which reduces the number of pairing operations from the number of matching rows $|\Upsilon|$ to the sum of the co-domain sizes of ι_l for all authorities l associated with the matching rows plus the number of "randomizers" ℓ that are selected with the rows $j \in \Upsilon$ under τ. Because we can compute the required number of pairings beforehand, we determine which of the two strategies requires the fewest and compute the pairing product accordingly.

The "extra" pairing product for opt5-opt6. For the variants using randomness splitting in the negative parts, i.e., opt5 and opt6, we have one extra pairing product that is not present in the other variants. This product can be computed more efficiently by grouping on the ciphertext side, i.e., we write

$$\prod_{j\in\overline{\Upsilon}} e\left(C_{2,j}, \check{K}_{3,\tilde{\rho}(j)}\right)^{\varepsilon_j} = \prod_{l\in\tilde{\rho}(\overline{\Upsilon})} e\left(\prod_{j\in\overline{\Upsilon}:\tilde{\rho}(j)=l} C_{2,j}^{\varepsilon_j}, \check{K}_{3,l}\right)$$

so that the number of pairings required is equal to the number of authorities associated with negations that are satisfied by the decrypting user.

The "remaining" pairing products. For the pairing products associated with the fifth and seventh pairing products, we use a similar strategy as for the second pairing product. The most notable difference for the optimizations using randomness-splitting techniques is that, for the first grouping strategy, we interleave three pairing products instead of two. For the second grouping strategy, we interleave two pairing products instead of one. We first note that the fifth and seventh pairing products have a shared argument $\check{C}_j$:

$$\prod_{j\in\overline{\Upsilon}} e(K_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j}, \check{C}_j) \cdot \prod_{j\in\overline{\Upsilon}} e\left(\prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}, \check{C}_j\right),$$

which we can always merge more efficiently by grouping the two products, i.e.,

$$\prod_{j\in\overline{\Upsilon}} e\left(K_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j} \cdot \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}, \check{C}_j\right).$$

For opt1-opt2, we additionally group on the ciphertext side, i.e.,

$$\prod_{l\in\tilde{\rho}(\overline{\Upsilon}),\mathsf{lab}\in\rho_{\mathsf{lab}}(\overline{\Upsilon}),\mathsf{att}'\in\rho(\overline{\Upsilon})} e\left(K_{2,l,\mathsf{lab}}^{-\varepsilon_j} \cdot \prod_{(\mathsf{lab},\mathsf{att})\in\mathcal{S}_l} K_{3,(l,\mathsf{lab},\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\mathsf{att}'}}}, \prod_{j\in\overline{\Upsilon}:\tilde{\rho}(j)=l,\rho_{\mathsf{lab}}(j)=\mathsf{lab},\rho(j)=\mathsf{att}'} \check{C}_j\right).$$

This reduces the number of pairings only in the number of $(l, \mathsf{lab}, \mathsf{att})$ duplicates, so we think it has little effect on the overall efficiency. For opt3-opt4, we can group more efficiently on the key side:

$$\prod_{\ell\in[m_{\tilde{\rho}}]} e\left(\prod_{j\in\overline{\Upsilon}:\tau_{\tilde{\rho}}(j)=\ell} \left(K_{2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j} \cdot \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}\right), \check{C}_\ell\right).$$

For the variants using randomness-splitting for the negative parts, i.e., opt5 and opt6, we compute the products similarly as for the second pairing product, i.e., either

$$\prod_{\ell\in[m_{\tilde{\rho}}]} e\left(\prod_{j\in\overline{\Upsilon}:\tau_{\tilde{\rho}}(j)} K_{2,1,\tilde{\rho}(j)}^{-\varepsilon_j} \cdot \prod_{j\in\overline{\Upsilon}:\tau(j)=\ell} \left(K_{2,2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j} \cdot \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}\right), \check{C}_\ell\right),$$

or, when the number of authorities is small, we compute

$$\prod_{l\in\tilde{\rho}(\Upsilon)} e\left(K_{2,1,l}, \prod_{j\in\Upsilon:\tilde{\rho}(j)=l} \check{C}_{\tau_{\tilde{\rho}}(j)}^{-\varepsilon_j}\right)$$
$$\cdot \prod_{\ell\in[m_{\tilde{\rho}}]} e\left(\prod_{j\in\overline{\Upsilon}:\tau(j)=\ell}\left(K_{2,2,\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}^{-\varepsilon_j}\cdot \prod_{(\rho_{\mathsf{lab}}(j),\mathsf{att})\in\mathcal{S}_{\tilde{\rho}(j)}} K_{3,(\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),\mathsf{att})}^{\frac{-\varepsilon_j}{x_{\mathsf{att}}-x_{\rho(j)}}}\right), \check{C}_\ell\right).$$

Note that, again, we precompute how many pairings are required for either method and, based on the minimum, we compute the product accordingly.

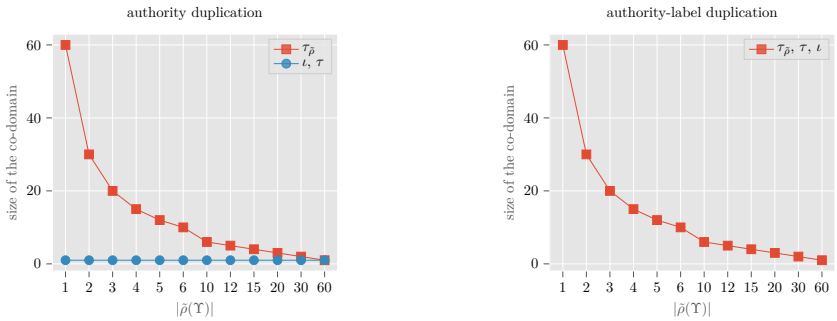
Remark 3. The randomness-splitting techniques allow us to benefit from the thresholdized approach between the grouping strategies. This is not possible in the versions without randomness splitting due to their structure.

4.7 Analyzing the behavior of the deduplication mappings

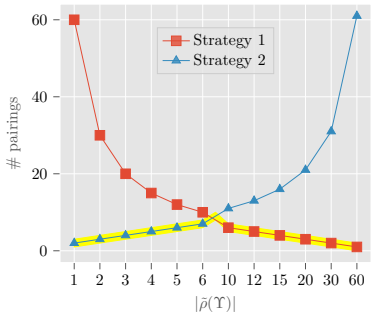
To understand why the second grouping strategy for the second and remaining pairing products can be faster in certain (practically motivated) situations, we analyze the behavior of the deduplication mappings τ, $\tau_{\tilde{\rho}}$ and ι_l. Specifically, we analyze their dependency on the total numbers of authorities used in the policies and sets. Although we have explicitly defined the deduplication mappings to depend on the maximum number of duplicates of either the authorities (in the case of $\tau_{\tilde{\rho}}$) or the authority-label pairs (in the case of τ and ι_l), they are indirectly also dependent on the total number of authorities. That is, for long policies and small numbers of authorities, we naturally expect many authority duplicates, meaning that the size of the co-domain of $\tau_{\tilde{\rho}}$ is large.

Example. To illustrate this with an example, suppose that we fix the policy length to 60. Then, having three authorities in the policy yields at least $\frac{60}{3} = 20$ authority duplicates, and thus, the size of the co-domain of $\tau_{\tilde{\rho}}$ is at least 20. In turn, the number of pairing operations required in the first grouping strategy for the second and remaining pairing products would be 20. Conversely, having only three authorities in the policy means that the second grouping strategy may require only 3 pairing operations in one of the partial products (depending on the size of the associated co-domains of ι_l), and $|\tau(\Upsilon)|$ in the other. Since τ and ι_l depend only on the number of authority-label duplicates, its co-domains could be much smaller than that of $\tau_{\tilde{\rho}}$. In practice, it is likely that the number of authorities is comparatively smaller than the number of labels, which is in turn smaller than the number of attribute values. In the best case, we have no label duplicates at all, in which case τ would map to only one randomizer, and thus, we require only one pairing for the other partial product. This amounts to a total of 4 pairing operations instead of 20, being a factor 5 faster. In the worst case, each label occurs 60 times, in which case τ maps to the same co-domain as $\tau_{\tilde{\rho}}$, meaning that the first grouping strategy is more efficient.

Systematic analysis. We analyze the effect of various duplication cases more systematically by expanding the example. In particular, we consider the cases where the authorities are duplicated (for varying numbers of duplicates) without duplicating the labels, and the cases where both the authorities and the labels are duplicated. This gives us a good overview of both the best-case and worst-case scenarios for the efficiency. For simplicity, we consider the divisors $1, 2, 3, 4, 5, 6, 10, 12, 15, 20, 30$ and 60 itself for our analysis, which will denote the total number of unique authorities $|\tilde{\rho}(\Upsilon)|$ (or authority-label pairs) that occur in the policy. We assume that each authority (or authority-label pair) occurs equally many times, meaning that the (minimum) number of duplicates is $\frac{60}{|\tilde{\rho}(\Upsilon)|}$. For the authority-label



(a) Behavior of the deduplication mappings.



(b) The costs for the second pairing product, assuming authority-only duplications.

Figure 3: Analysis of the behavior of the deduplication mappings, i.e., the relationship between the sizes of the co-domain and the total number of authorities associated with the policy $|\tilde{\rho}(\Upsilon)|$, and their effect on the total number of pairings required in the second pairing product. Note that the yellow highlighted costs reflect those that can be achieved by the variants with randommness splitting (in the positive parts).

duplication case, we assume that the duplication happens for the pairs, so that we obtain a clean comparison among all mappings.

Evaluation of the deduplication mappings. As Figure 3a shows, the behavior of the mappings τ and ι_l differ strongly from $\tau_{\tilde{\rho}}$ for the cases where the authorties are duplicated but not the labels. In particular, we observe that τ and ι_l map all attributes to the same value for any number of authorities in the authority-duplication case, whereas $\tau_{\tilde{\rho}}$ maps all attributes to different integers in the worst case. It also shows that, for the authority-label duplication case, τ and ι_l behave the same as $\tau_{\tilde{\rho}}$, meaning that the second strategy is not more efficient in this case.

To illustrate the advantage in the authority-duplication case, Figure 3b shows the total number of pairings that we require for the first and second strategy for the second pairing product, which require $|\tau_{\tilde{\rho}}(\Upsilon)|$ and $|\tau(\Upsilon)| + m_{\mathsf{aut}\text{-}\iota}$ pairing operations, respectively. Here, $m_{\mathsf{aut}\text{-}\iota}$ is the sum of the co-domain sizes of ι_l for each authority l associated with the matching rows, i.e., $l \in \tilde{\rho}(\Upsilon)$. Note that the line associated with $|\tau_{\tilde{\rho}}(\Upsilon)|$ are the pairing costs for the randomness re-use variants that do not use randomness splitting, whereas the yellow-highlighted costs are those for our variants with randomness splitting. Effectively, we are able to reduce the number of pairing operations with randomness splitting.

Analyzing and evaluating the behavior of the second and remaining products. To summarize the effect of randomness splitting, we analyze the pairing costs of the second and remaining products (including the extra product for the variants using randomness splitting)

Table 1: Analysis of the pairing operations for various decryption strategies (available for the variants using randomness re-use only (R) or randomness splitting (S)), for the authority duplication cases without label duplicates. The costs associated with the positive literals Υ are related to the second pairing product, and the negative literals $\overline{\Upsilon}$ are related to the remaining products. For the variants with randomness splitting, we also add the extra pairing product to the costs. We indicate the variant requiring the lowest pairing costs with "–", boldfacing the associated costs, and, for each other variant, we compute the slowdown factor by dividing their costs by the lowest costs. We use the notation $m_{\mathsf{aut}\text{-}\iota}$ to indicate sum of the co-domain sizes of ι_l for each authority l associated with the matching rows, i.e., $l \in \tilde{\rho}(\Upsilon)$.

		Number of authorities ($\lvert\tilde{\rho}(\Upsilon)\rvert$ or $\lvert\tilde{\rho}(\overline{\Upsilon})\rvert$)											
		1		2		3		4		5		6	
Pairing costs	*Var*	#	SlF	#	SlF	#	SlF	#	SlF	#	SlF	#	SlF
$\lvert\tau_{\tilde{\rho}}(\Upsilon)\rvert$	R,S	60	30.0	30	10.0	20	5.0	15	3.0	12	2.0	10	1.43
$\lvert\tau(\Upsilon)\rvert + m_{\mathsf{aut}\text{-}\iota}$	S	**2**	–	**3**	–	4	–	**5**	–	**6**	–	**7**	–
$\lvert\tau_{\tilde{\rho}}(\overline{\Upsilon})\rvert$	R	60	20.0	30	6.0	20	2.86	15	1.67	12	1.09	**10**	–
$\lvert\tau_{\tilde{\rho}}(\overline{\Upsilon})\rvert + \lvert\tilde{\rho}(\overline{\Upsilon})\rvert$	S	61	20.33	32	6.4	23	3.29	19	2.11	17	1.55	16	1.6
$\lvert\tau(\overline{\Upsilon})\rvert + 2\lvert\tilde{\rho}(\overline{\Upsilon})\rvert$	S	**3**	–	**5**	–	**7**	–	**9**	–	**11**	–	13	1.3

(a) Number of authorities from 1 to 6.

		Number of authorities ($\lvert\tilde{\rho}(\Upsilon)\rvert$ or $\lvert\tilde{\rho}(\overline{\Upsilon})\rvert$)											
		10		12		15		20		30		60	
Pairing costs	*Var*	#	SlF	#	SlF	#	SlF	#	SlF	#	SlF	#	SlF
$\lvert\tau_{\tilde{\rho}}(\Upsilon)\rvert$	R,S	**6**	–	**5**	–	**4**	–	**3**	–	**2**	–	**1**	–
$\lvert\tau(\Upsilon)\rvert + m_{\mathsf{aut}\text{-}\iota}$	S	11	1.83	13	2.6	16	4.0	21	7.0	31	15.5	61	61.0
$\lvert\tau_{\tilde{\rho}}(\overline{\Upsilon})\rvert$	R	**6**	–	**5**	–	**4**	–	**3**	–	**2**	–	**1**	–
$\lvert\tau_{\tilde{\rho}}(\overline{\Upsilon})\rvert + \lvert\tilde{\rho}(\overline{\Upsilon})\rvert$	S	16	2.67	17	3.4	19	4.75	23	7.67	32	16.0	61	61.0
$\lvert\tau(\overline{\Upsilon})\rvert + 2\lvert\tilde{\rho}(\overline{\Upsilon})\rvert$	S	21	3.5	25	5.0	31	7.75	41	13.67	61	30.5	121	121.0

(b) Number of authorities from 10 to 60.

based on the various mappings and in the two duplication cases. Table 1 demonstrates that, for small numbers of authorities, the variants using randomness splitting allow a more efficient decryption strategy than the variants using randomness re-use only. Concretely, for positive literals, we can reduce the number of pairings by a factor of up to 30 with the second grouping strategy, whereas we can always use the first grouping strategy, should that be more efficient. For negative literals, we can reduce the number of pairings by a factor of up to 20. Unfortunately, the number of pairing operations increases faster than those for the positive literals, requiring a factor of up to 60 more pairing operations than the variants with randomness re-use only in the worst case. Hence, depending on how many authorities we expect in the system, we may choose either the variant with or without randomness splitting.

5 Theoretical performance analysis

We want to formally compare the base scheme and the different optimizations. We start with the computational costs and then analyze storage costs. Throughout this section, we will use additional variables to express the costs. These are defined as follows:

- m_{lab}: number of distinct labels in the set $\mathcal{S}_l$, therefore $m_{\mathsf{lab}} \leq |\mathcal{S}_l|$.
- m_{att}: number of distinct attributes in the set $\mathcal{S}_l$, therefore $m_{\mathsf{att}} \leq |\mathcal{S}_l|$.

- m_l: maximum number of attributes per label in the set $\mathcal{S}_l$ (cf. mapping ι_l), therefore $m_l \leq m_{\mathsf{att}}$.
- n_1: number of literals, i.e., rows in the LSSS matrix.
- n_1^+ resp. n_1^-: number of positive literals (rows with $\rho'(j) = 1$) resp. negative literals (rows with $\rho'(j) = 0$) used in encryption, therefore $n_1^+ + n_1^- = n_1$.
- m_{aut} resp. $\overline{m}_{\mathsf{aut}}$: number of unique authorities across positive resp. negative literals, denoted earlier as $|\tilde{\rho}(\Upsilon)|$ and $|\tilde{\rho}(\overline{\Upsilon})|$, respectively .
- $m_{\mathsf{aut\text{-}att}}$: number of unique authority-attribute pairs across positive literals.
- $m_{\mathsf{aut\text{-}lab}}$ resp. $\overline{m}_{\mathsf{aut\text{-}lab}}$: maximum number of authority-label duplicates across positive resp. negative literals, denoted earlier as $|\tau(\Upsilon)|$ and $|\tau(\overline{\Upsilon})|$, respectively .
- m_{trip} resp. $\overline{m}_{\mathsf{trip}}$: number of unique authority-label-attribute triples across positive resp. negative literals.
- $m_{\tilde{\rho}}^+$ resp. $\overline{m}_{\tilde{\rho}}$: maximum number of positive resp. negative literals mapped to an authority (cf. mapping $\tau_{\tilde{\rho}}$), denoted earlier as $|\tau_{\tilde{\rho}}(\Upsilon)|$ and $|\tau_{\tilde{\rho}}(\overline{\Upsilon})|$, respectively .
- $m_{\mathsf{aut}\text{-}\iota}$: sum of the maximum number of attributes per label across positive literals for each authority (cf. mapping ι_l), therefore $m_{\mathsf{aut}\text{-}\iota} \leq \sum_l m_l$.
- $\overline{d}$ ("negation degree"): number of alternatives to the negated attribute.

5.1 Computational costs

We summarize the cost for key generation, encryption and decryption in Table 2. We investigate the computational costs by focusing only on computation-heavy operations such as exponentiations, hashes in the curve and pairings, and not on occasional multiplications and squarings. In addition to standard exponentiations, we will also count multi-base exponentiations (MBE), where we write k-MBE to denote an exponentiation of the form $\prod_{i\in[k]} g_i^{x_i}$ where $x_i \in \mathbb{Z}_p$ and $g_i \in \mathbb{G}$ for all i or $g_i \in \mathbb{H}$ for all i (cf. also Section 6). Further, we assume that master secret keys which are used as group elements in "the other group" than specified in the public key are pre-computed, e.g., g^{α_l}.

Key generation. Randomness re-use for keys (opt2-opt6) allows to reduce exponentiations in $\mathbb{H}$ from m_{att} to m_l. Randomness splitting on the positive parts (opt4) adds m_l exponentiations in $\mathbb{G}$, whereas it adds $m_{\mathsf{lab}} + 1$ exponentations in $\mathbb{G}$ and 1 exponentation in $\mathbb{H}$ when used on the negative parts (opt5). In the combined scheme (opt6) they add up. Ignoring constant added costs, we see a slowdown for opt4-opt6, but the cost is still dominated by exponentiations in $\mathbb{H}$ and 2-MBEs in $\mathbb{G}$.

Encryption. Randomness re-use for the ciphertexts (opt3-opt6) allows to reduce the number of exponentiations in $\mathbb{H}$ from $5n_1$ to $4n_1 + m_{\tilde{\rho}}$. Randomness splitting for the positive parts (opt4,opt6) adds n_1^+ exponentiations in $\mathbb{G}$ to that, whereas splitting for the negative parts (opt5,opt6) adds n_1^- 2-MBEs in $\mathbb{G}$. Hence, for all optimizations, we expect a slowdown of at most a factor 1.2 compared to opt1. The concrete value however depends on the actual number of randomizers required and may therefore be nontrivial in practice.

Decryption. As the table indicates, the decryption costs depend on many different variables, making it hard to draw any qualitative conclusions about their expected improvements. In general, all optimizations opt2-opt6 reduce the number of pairings compared to the starting point scheme (here, we use opt1 as reference which is similarly optimized and thus more fairly comparable). Schemes opt4-opt6 allow different ways to decrypt so that they can perform better in certain cases. One can see that opt4 requires less pairings than

Table 2: Computational costs for key generation and encryption (upper table) and decryption cost (lower table), counting exponentiations ("Exp"), multi-base exponentiations with $k \in \{2, \overline{d}\}$ bases ("k-MBE") in groups $\mathbb{G}$ and $\mathbb{H}$, as well as pairings ("Pair"). For a description of the variables, refer to the list at the beginning of Section 5. Colored entries highlight the differences to opt1.

	Key generation			*Encryption*		
	$\mathbb{G}$		$\mathbb{H}$	$\mathbb{G}$		$\mathbb{H}$
Scheme	Exp	2-MBE	Exp	Exp	2-MBE	Exp
opt0	$2 + m_{\mathsf{lab}}$	$2\lvert\mathcal{S}_l\rvert$	$2m_{\mathsf{att}}$	-	n_1	$5n_1$
opt1	$2 + m_{\mathsf{lab}}$	$2\lvert\mathcal{S}_l\rvert$	$2m_{\mathsf{att}}$	-	n_1	$5n_1$
opt2	$2 + m_{\mathsf{lab}}$	$2\lvert\mathcal{S}_l\rvert$	$2m_l$	-	n_1	$5n_1$
opt3	$2 + m_{\mathsf{lab}}$	$2\lvert\mathcal{S}_l\rvert$	$2m_l$	-	n_1	$4n_1 + m_{\tilde{\rho}}$
opt4	$2 + m_{\mathsf{lab}} + m_l$	$2\lvert\mathcal{S}_l\rvert$	$2m_l$	n_1^+	n_1	$4n_1 + m_{\tilde{\rho}}$
opt5	$3 + 2m_{\mathsf{lab}}$	$2\lvert\mathcal{S}_l\rvert$	$2m_l + 1$	-	$n_1 + n_1^-$	$4n_1 + m_{\tilde{\rho}}$
opt6	$3 + 2m_{\mathsf{lab}} + m_l$	$2\lvert\mathcal{S}_l\rvert$	$2m_l + 1$	n_1^+	$n_1 + n_1^-$	$4n_1 + m_{\tilde{\rho}}$

(a) Key generation and encryption cost: Schemes opt0-opt4 additionally perform $1 + 4m_{\mathsf{lab}}$ hashes into $\mathbb{G}$ during key generation and $2(m_{\mathsf{aut\text{-}lab}} + \overline{m}_{\mathsf{aut\text{-}lab}})$ hashes into $\mathbb{G}$ during encryption. Schemes opt5-opt6 add another m_{lab} hashes during key generation and $\overline{m}_{\mathsf{aut\text{-}lab}}$ during encryption.

	Decryption: Pairing operations				
Scheme	"fixed"	"fourth"	"second"	"extra"	"remaining"
opt0	$2 + \lvert\overline{\Upsilon}\rvert$	$\lvert\Upsilon\rvert$	$\lvert\Upsilon\rvert$	-	$2\lvert\overline{\Upsilon}\rvert$
opt1	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut\text{-}att}}$	m_{trip}	-	$\overline{m}_{\mathsf{trip}}$
opt2	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut}\text{-}\iota}$	m_{trip}	-	$\overline{m}_{\mathsf{trip}}$
opt3	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut}\text{-}\iota}$	$m_{\tilde{\rho}}^+$	-	$\overline{m}_{\tilde{\rho}}$
opt4	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut}\text{-}\iota}$	$\min\{m_{\tilde{\rho}}^+, m_{\mathsf{aut}\text{-}\iota} + m_{\mathsf{aut\text{-}lab}}\}$	-	$\overline{m}_{\tilde{\rho}}$
opt5	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut}\text{-}\iota}$	$m_{\tilde{\rho}}^+$	$\overline{m}_{\mathsf{aut}}$	$\min\{\overline{m}_{\tilde{\rho}}, \overline{m}_{\mathsf{aut}} + \overline{m}_{\mathsf{aut\text{-}lab}}\}$
opt6	$2 + \overline{m}_{\mathsf{trip}}$	$m_{\mathsf{aut}\text{-}\iota}$	$\min\{m_{\tilde{\rho}}^+, m_{\mathsf{aut}\text{-}\iota} + m_{\mathsf{aut\text{-}lab}}\}$	$\overline{m}_{\mathsf{aut}}$	$\min\{\overline{m}_{\tilde{\rho}}, \overline{m}_{\mathsf{aut}} + \overline{m}_{\mathsf{aut\text{-}lab}}\}$

(b) Decryption cost: All schemes additionally perform one hash into $\mathbb{G}$. Scheme opt0 performs $m_{\overline{\Upsilon}}$ $\overline{d}$-MBEs in $\mathbb{G}$ and $\mathbb{H}$. Schemes opt1-opt6 perform $\overline{m}_{\mathsf{trip}}$ $\overline{d}$-MBEs in $\mathbb{G}$ and $\mathbb{H}$.

opt3 when $m_{\mathsf{aut}\text{-}\iota} + m_{\mathsf{aut\text{-}lab}} < m_{\tilde{\rho}}^+$, and that opt5 requires less pairings than opt3 when $2\overline{m}_{\mathsf{aut}} + \overline{m}_{\mathsf{aut\text{-}lab}} < \overline{m}_{\tilde{\rho}}$, which we also see in the analysis in Section 4.7. However, we also perceive a significant dependency on the negation degree, and these costs are the same for all optimizations (opt1-opt6, see caption of Table 2b). There is a strict trade-off between encryption and decryption: when the decryption costs decrease compared to the case without duplicates, the encryption costs increase, and vice versa.

5.2 Storage costs

In Table 3 we summarize the effects on the sizes of master public keys, secret keys and ciphertexts. We note that, for common group choices, group elements in $\mathbb{H}$ are about twice the size compared to group elements in $\mathbb{G}$. Overall, the storage costs do not increase much and we do not expect it to be an issue in common real-world settings, e.g., Portunus [LVV+23]. We describe the storage costs of the different optimizations below.

Master public keys. We first observe that randomness splitting that is used in opt4-opt6 adds 1 resp. 2 group elements in $\mathbb{G}$ to the MPK of each authority.

Table 3: Storage costs for master public keys (for each authority), secret keys and ciphertexts in terms of group elements in $\mathbb{G}$ and $\mathbb{H}$. Colored entries highlight the differences to opt1.

	MPK		*Secret Key*		*Ciphertext*	
Scheme	$\mathbb{G}$	$\mathbb{H}$	$\mathbb{G}$	$\mathbb{H}$	$\mathbb{G}$	$\mathbb{H}$
opt0	-	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert$	$2m_{\mathsf{att}}$	n_1	$3n_1$
opt1	-	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert$	$2m_{\mathsf{att}}$	n_1	$3n_1$
opt2	-	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert$	$2m_l$	n_1	$3n_1$
opt3	-	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert$	$2m_l$	n_1	$2n_1 + m_{\tilde{\rho}}$
opt4	1	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert + m_l$	$2m_l$	n_1	$2n_1 + m_{\tilde{\rho}}$
opt5	1	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert + 1$	$2m_l + 1$	$n_1 + n_1^-$	$2n_1 + m_{\tilde{\rho}}$
opt6	2	3	$m_{\mathsf{lab}} + 2\lvert\mathcal{S}_l\rvert + m_l + 1$	$2m_l + 1$	$n_1 + n_1^-$	$2n_1 + m_{\tilde{\rho}}$

Secret keys. Randomness splitting for the positive parts (opt4,opt6) increases the secret key size by m_l group elements in $\mathbb{G}$, and randomness splitting on the negative parts (opt5,opt6) additionally increases the secret key size by another group element in $\mathbb{G}$. At the same time, randomness re-use decreases the size from m_{att} to m_l. This is similar to what we have observed for the computation cost, since each additional exponentiation comes from adding one group element. Since $m_l \leq m_{\mathsf{lab}} \leq |\mathcal{S}_l|$ and one element in $\mathbb{G}$ is about half the size of an element in $\mathbb{H}$, the increase in size (of opt4 and opt6) compared to opt0 is by a factor at most 1.2, while all other optimizations lead to a decrease in size, unless $m_{\mathsf{att}} = m_l$, which means that randomness re-use is not possible in the first place.

Ciphertexts. The ciphertext size is only influenced by randomness re-use. This decreases the storage cost for opt3-opt6 from $3n_1$ to $2n_1 + m_{\tilde{\rho}}$ group elements in $\mathbb{H}$ plus n_1 group elements in $\mathbb{G}$, where the latter are present in all schemes. For opt4 and opt6, we get an additional n_1^- group elements in $\mathbb{G}$. Hence, we see a substantial improvement for opt3 and opt4,when the maximum number of rows associated with an authority is small, whereas for opt5 and opt6 this depends on the number of negated rows.

6 Implementation

We implement all nine optimization variants of the scheme in Rust based on the Arkworks [ac22] ecosystem. It provides a variable-time implementation of the BLS12-381 pairing-friendly curve [BLS02, Bow17, WB19] including the necessary group and pairing operations. We exclusively consider BSL12-381 for the evaluation, as it provides a security level of roughly 128 bits [Gui20, SKSW22] and clearly outperforms other pairing-friendly curves at the same security level [dlPVA22, AFG24]. However, the implementation is curve-agnostic and can be used with any curve provided by Arkworks, which supports the necessary group and pairing operations. The measurements are taken on an AMD server running an Ubuntu 22.04 virtual machine with 8 physical and 16 virtual cores and 32GB of RAM. The benchmarks are done with the Criterion [AH14] benchmarking framework.

6.1 Group and pairing operations

We use the following group operations provided by Arkworks. Note that we do not use precomputation to speed up any algorithms, as Arkworks provides limited support for this.

- **Variable-base exponentiation (VBE):** an exponentiation of the form g^x, in which the variable base g varies in each execution of the algorithm. Note that Arkworks refers to these as scalar multiplications;

- **Multiple-base exponentiation (MBE):** a product of multiple exponentiations [Möl01], typically of the form $\prod_{i \in \mathcal{I}} g_i^{x_i}$ such that g_i are bases and x_i are exponents for each $i \in \mathcal{I}$ with $|\mathcal{I}| \geq 2$. Note that Arkworks refers to these as multi-scalar multiplications (MSM);
- **Hashing into the group:** a mapping from the set of arbitrary-length strings $\{0,1\}^*$ to a group.

Hashing into the group (which we refer to as *full-domain hashes*, like ISABELLA [RVV24a]) is implemented by extending Arkworks with the SwiftEC algorithm [CRT25] for BLS12-381, following the version specialized to curves with $b = 0$ given in [AHST23]. In this approach, an arbitrary byte sequence is first expanded to a pseudorandom byte sequence using a cryptographically secure hash function, SHA256 in this case. The latter sequence is then mapped to the underlying field, after which it is encoded to a point on the elliptic curve through a map f. SwiftEC constructs the encoding map f as a parameterized family of functions, such that $f_{h_2(m)}(h_1(m))$ is indifferentiable from a random oracle if hash functions h_1 and h_2 are indifferentiable as well. Clearing the co-factor eventually yields an element of the desired subgroup. This idea saves one evaluation of the encoding function f in comparison to the *simplified Shallue-van de Woestijne-Ulas* (SWU) [WB19] approach used in Arkworks, and significantly speeds up hashing. This implementation can be considered of independent interest. Lastly, we locally cache repeated computations of a full-domain hash for the same inputs (e.g., authority-label pairs or identities) in the implementation of the schemes to avoid expensive recomputations.

6.2 Policy parser and secret sharing

A crucial part of an ABE implementation is the implementation of a suitable policy parser. In our schemes, we require both a representation of the policy as well as a secret-sharing scheme (Definition 1). Our policy parser is based on the code from [FHK19, LVV+23] and supports conjunctions, disjunctions, negations and parentheses. It parses a policy as a left-associative, binary tree and propagates negations downwards to the leaves using De Morgan's laws. Each literal has the form `x.y:z`, where `x` is an authority, `y` is a label and `z` is a value. This representation yields a natural, unambigious and implicit encoding of functions like ρ or ρ_{lab} without requiring further input. The parsing time is not included in the benchmarks.

During encryption, the secret is split into shares as introduced by Lewko and Waters in [LW11b]. Their secret-sharing scheme works for all Boolean formulas represented as a tree. Roughly, it starts at the root of the tree and propagates shares of a secret down to the leaves. For every node in the tree that represents a conjunction, it splits a secret s into two secrets s' and $s - s'$ with $s' \in_R \mathbb{Z}_p$ and propagates the shares to the children. For every disjunction, the secret in the node is propagated to the children. This ensures that we have $\varepsilon_j = 1$ if $j \in \Upsilon \cup \overline{\Upsilon}$ during decryption. The advantage is that we can replace many exponentiations with branches: if $\varepsilon_j = 1$, then we include the product, and if $\varepsilon_j = 0$, we do not. Although branching is often considered insecure due to vulnerability against timing attacks, it is not a problem for our implementation. In the first place, our implementation does not aim to be constant-time, because the implementations of the operations, provided by Arkworks, are not constant-time. In the second place, the policy is public, so a potential attacker could already learn that the decrypting user satisfies the policy.

During decryption we determine $\Upsilon \cup \overline{\Upsilon}$ by recursively traversing the policy tree. In case of disjunctions, we favor the *smaller* subexpression by estimating the number of literals required in each case. For negations, we instead consider the number of alternative attributes with the same label, as these will be used to reconstruct the missing literal. This ultimately reduces the number of operations during decryption for practical use-cases.

The secret-sharing and secret-reconstruction routines for encryption and decryption are included in the benchmarks. They are identical for all scheme variants.

6.3 Constructing "deduplication" mappings

Several optimization variants require auxiliary "deduplication" mappings ι_l, $\tau_{\tilde{\rho}}$ and τ. We call them deduplication mappings, because they assign each occurrence of the same tuple a unique integer, effectively deduplicating those tuples. These functions depend on the concrete set of user attributes (ι_l) or policy ($\tau_{\tilde{\rho}}$ and τ) and are derived at runtime. For the implementation, we deviate slightly from mathematical notations and define

$$\begin{gathered}\iota_l(\mathsf{lab}, \mathsf{att}) \leftrightarrow \mathcal{I}_\iota(l, \mathsf{lab}, \mathsf{att}) \\ \tau_{\tilde{\rho}}(j) \leftrightarrow \mathcal{I}_{\tau_{\tilde{\rho}}}(\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)) \text{ and} \\ \tau(j) \leftrightarrow \mathcal{I}_\tau(\tilde{\rho}(j), \rho_{\mathsf{lab}}(j), \rho(j)).\end{gathered}$$

This yields a uniform and generic interface $\mathcal{I}_R$ for all deduplication mappings, which solely depends on the respective *injectivity restriction* R. The row mappings $(\rho, \rho_{\mathsf{lab}}, \tilde{\rho})$ are not necessarily injective and may thus cause two different rows j to be mapped to the same $(l, \mathsf{lab}, \mathsf{att})$ triple. This is not problematic, since the set of user attributes and the rows of the policy are represented as *lists* of triples and may thus contain duplicates.

The purpose of the ι and τ functions is to indicate in which cases a randomizer may be reused without compromising security. Conversely, they also indicate when two $(l, \mathsf{lab}, \mathsf{att})$ triples must not be associated with the same randomizer, namely if they belong to the same subdomain for which we require injectivity. So each "deduplication" mapping is essentially defined by how the subdomains are formed. All elements in one subdomain can simply be mapped to consecutive integers starting from 1.

We use this observation to construct a deduplication mapping for a list of $(l, \mathsf{lab}, \mathsf{att})$ triples as follows. First, we blind each component of each entry that is not part of the injectivity restriction. The resulting duplicate entries in our list belong to the same subdomain and must be mapped to different integers. By simply associating each entry of the list with "how many times it has appeared so far" the list is *deduplicated*.

For instance, we require ι_l to be injective on each subdomain $\mathcal{S}_{l,\mathsf{lab}} = \{(\mathsf{lab}', \mathsf{att}) \mid (\mathsf{lab}', \mathsf{att}) \in \mathcal{S}_l \wedge \mathsf{lab}' = \mathsf{lab}\}$, i.e., each label occurrence (in the set $\mathcal{S}_l$) needs to be mapped to a unique integer. By blinding each attribute and numbering the entries we obtain

$$\begin{aligned}&\mathrm{blind}((l_1, \mathsf{lab}_1, \mathsf{att}_1), (l_1, \mathsf{lab}_1, \mathsf{att}_2), (l_2, \mathsf{lab}_1, \mathsf{att}_1), (l_1, \mathsf{lab}_2, \mathsf{att}_1), (l_2, \mathsf{lab}_2, \mathsf{att}_1)) \\ &= \underbrace{(l_1, \mathsf{lab}_1, *)}_{1}, \underbrace{(l_1, \mathsf{lab}_1, *)}_{2}, \underbrace{(l_2, \mathsf{lab}_1, *)}_{1}, \underbrace{(l_1, \mathsf{lab}_2, *)}_{1}, \underbrace{(l_2, \mathsf{lab}_2, *)}_{1}.\end{aligned}$$

In our implementation in Rust we use `HashMap`s to store the derived function instances. They are passed to `keygen`, `encrypt` and `decrypt` as additional parameters, even if not required for a particular optimization. Like in the case of constructing the policies and sets, the time required to construct a deduplication mapping is excluded from the benchmarks.

6.4 Generating test cases

The correctness of all implementations is tested with hand-crafted and auto-generated tests. The former are exhaustive for small and simple policies. They cover all possible causes of decryption success as well as decryption failures. This ensures that we can test all subroutines within our implementation. The latter enable easy correctness checks with hundreds of (significantly larger) policies and attribute sets which are unfeasable to construct and oversee manually. By generating random test inputs which are larger then the sets of authorities, labels and attributes we sample from, we automatically enforce

duplications in the test cases. Some of the decryption routines use a different grouping strategy based on the policy and user attribute set. We make sure to test the correctness of all possible execution paths, irrespectively of which one is actually taken during the benchmarks. We achieve this by including flags in the code that allow us to manually force the decryption procedure to follow specific subroutines. Because of the equivalence of the pairing products (Section 4.6), this is not a problem.

7 Our advanced benchmarking extensions

We devise a new benchmarking strategy that allows us to evaluate the schemes' efficiency more fairly. Since most previous schemes had reasonably simpler representations of the policies and sets, the benchmarks only varied the policy and set sizes in the benchmarks. For a fair comparison, the policies were restricted to the sole use of ANDs, and each attribute was used only once. In this way, it could not be the case that one scheme's benchmarks could decrypt with, on average, fewer matches between the set and policy attributes than another scheme's benchmarks.

The computational costs of the schemes we consider depend on additional variables, therefore, we need to extend that benchmarking strategy. In particular, the computational costs are heavily influenced by the amount or randomness we can re-use. That is, the more randomness can be re-used, the fewer pairings we need during decryption. The amount of randomness that we can re-use is in turn dependent on the uniqueness of the $(l, \mathsf{lab}, \mathsf{att})$ triples. For example, the scheme could require more randomness (and thus more pairings during decryption) when the same authority-label pair is used for multiple attributes.

7.1 Shortcomings of ABE Squared

To apply the same fairness principles as put forth in ABE Squared, we also need to find a good strategy to measure the influence of any such duplicates. One could argue that ABE Squared contained a gap in this respect: the AC17(-LU) schemes outperformed the Wat11(-LU) schemes due to the use of randomness re-use, but the authors did not benchmark AC17(-LU) for policies in which the same attribute occurs multiple times, to illustrate the effect. In the most extreme case (in which the policy consists only of the same attribute), the efficiency of AC17(-LU) would have likely collided with that of Wat11.

Because it is not very likely that policies consisting of only the same copy occur in practice, we do not consider it a significant gap in the case of Wat11(-LU) and AC17(-LU). Furthermore, from the structure, it is also clear that the AC17(-LU) costs approach the Wat11(-LU) costs when the same attribute is re-used many times, meaning that we do not have to run experiments to confirm our suspicions. In our schemes, however, each attribute is represented by a triple, and some types of duplicates would actually occur in practice. For example, if there are few authorities in the system, we would expect many authority duplicates in the policies and sets. Hence, we need a more fine-grained approach than ABE Squared.

7.2 Extending the benchmarking strategy

To bridge this gap, we extend the benchmarking strategy of ABE Squared as follows.

The first phase. We start similarly, by benchmarking the schemes with policies consisting of only ANDs. Each attribute in the policy is globally different (i.e., each authority, label and attribute value is unique). We do the same for the sets, and in particular, each set contains exactly those attributes that are also in the policy. We benchmark the schemes for variable policy lengths and set sizes to obtain a baseline on the computational costs.

Because our schemes also support negations, we do this twice: once for policies with positive instances for all attributes and once for policies with negative instances for all attributes. Then, after setting the baselines, we fix the lengths and sizes of the policies and sets to some large enough value, e.g., to 60, to "filter out" the overhead and constant costs from the algorithms.

The second phase. In the next phase of our benchmarks, we want to obtain a fine-grained efficiency analysis on the effect of using duplicates in the policies and sets (whether that be authorities, labels or attribute values). To this end, we extend the strategy outlined in our analysis of the deduplication mappings in Section 4.7. In particular, we consider every (strictly smaller) subset of {authority, label, attribute} entries of the policies and sets. (Note that we do not consider the setting in which the whole triple is duplicated, as motivated earlier.) For each subset, we replace those corresponding entries of the policies and sets by duplicates. We start by using only one unique value for each entry, then by two unique values, then by three, and repeat this (with divisors of 60, excluding 15 and 20) until we reach the original setting with 60 different values. By applying this benchmarking strategy for every subset of {authority, label, attribute}, we can measure the effect of using any type of duplicates in the policies and sets. By considering divisors of 60, we obtain a smooth comparison, as it cannot be the case that one triple occurs more often than another. We have excluded 15 and 20 from our benchmarks, because we anticipate that they add little extra insight to the results compared to the smallest values $(1, 2, 3, 4, 5, 6, 10, 12)$, and 30. Note that 30 divides 60 exactly in half, allowing us to measure the effect of having two duplicates (which we consider more valuable than considering 3 or 4 duplicates), whereas considering small values allows us to consider cases where we have few, e.g., numbers of authorities, i.e., $1, 2, 3, 4, 5, 6, 10, 12$, which we believe to be realistic scenarios.

As an example, consider a fixed policy length and set size of 4 attributes, where 4 has divisors 1, 2 and 4. If we want to measure the effect of using attribute duplicates, then we benchmark the following "attributes" (i.e., the policies are constructed by applying an AND to the sets and the sets are simply the sets):

1. $(\mathrm{aut}_1, \mathrm{lab}_1, \mathrm{att}_1), (\mathrm{aut}_2, \mathrm{lab}_2, \mathrm{att}_1), (\mathrm{aut}_3, \mathrm{lab}_3, \mathrm{att}_1), (\mathrm{aut}_4, \mathrm{lab}_4, \mathrm{att}_1)$
2. $(\mathrm{aut}_1, \mathrm{lab}_1, \mathrm{att}_1), (\mathrm{aut}_2, \mathrm{lab}_2, \mathrm{att}_1), (\mathrm{aut}_3, \mathrm{lab}_3, \mathrm{att}_2), (\mathrm{aut}_4, \mathrm{lab}_4, \mathrm{att}_2)$
4. $(\mathrm{aut}_1, \mathrm{lab}_1, \mathrm{att}_1), (\mathrm{aut}_2, \mathrm{lab}_2, \mathrm{att}_2), (\mathrm{aut}_3, \mathrm{lab}_3, \mathrm{att}_3), (\mathrm{aut}_4, \mathrm{lab}_4, \mathrm{att}_4)$

Similarly, we can measure the effect of having, e.g., authority-label duplicates.

7.3 Benchmarking negations

As our theoretical analysis (and even the construction of our schemes) suggests, the schemes perform significantly differently when using positive literals than when negations are used. Thus, we run all the benchmarks for the case in which all attributes in the policy have positive literals, and for the case in which all attributes have negative literals. In this way, we can ascribe certain perceived behavior to the specific case that we consider (i.e., positive or negative literals). Furthermore, if we benchmark both at the same time, they might cancel out each other's influence on the efficiency.

For negations, we also have another variable that influences the efficiency: the number of attributes in the key set that has the same authority and label as the negated policy attribute. We call this number of alternatives to the negated attribute the negation degree. Because the computational costs depend heavily on the negation degree, we also benchmark the schemes for various negation degrees. In the first phase of our benchmarking strategy, this means that we fix the policy length to 50. Then we benchmark the schemes by increasing the negation degree with small steps, i.e., $1, 2, ..., 10$, which gives a fine-grained

illustration of the influence of the negation degree on a small interval. After that, we take larger steps, i.e., $20, 30$, to analyze the efficiency for significantly higher negation degrees. Furthermore, we run all the Strategy 2 benchmarks with negation degrees $1, 4, 7$, so that we can analyze any potential dependencies between duplications and the negation degree.

7.4 Naming convention for our benchmarking strategy

Our benchmarking strategy consists of several phases. At a high level, we consider two phases: the first phase covers the "regular" benchmarks, which consider various policy lengths, distinction between positive and negative literals and various negation degrees. Because this phase consists of benchmarks "in two dimensions", this phase consists of two subphases. In the first subphase, we fix the measurements for various negation degrees and use the policy length as the variable. In the second subphase, we fix the policy length and use the negation degree as the variable. These are called "Strategy 1(a)" and "Strategy 1(b)", respectively. In the second phase of the benchmarking strategy, we fix all the "variables" from the first phase (i.e., policy lengths and negation degrees) and consider the different duplication cases. For each duplication case, we use the number of duplicates as the variable. These are called the "Strategy 2" benchmarks.

The negation degree. In our benchmarks, we have slightly abused the definition of the negation degree to cover both expressions for positive and negative literals with one variable. For a degree $d \geq 1$, we simply consider negations (with d alternatives with the same authority-label pair in the attribute set). For a degree $d = 0$, we do not consider a negation with 0 alternatives, but positive literals. This is simply a naming convention in the benchmarking utilities and scripts and enables simpler processing of data.

Subtypes for our "Strategy 2" benchmarks. We summarize each type of "duplication case" in the naming of the strategy and the benchmarks. For example, for varying numbers of (l, lab) duplicates, we refer to the strategy as `vary_auth_lbl`. We consider the following six duplication types: `vary_attr`, `vary_lbl`, `vary_auth`, `vary_attr_lbl`, `vary_attr_auth`, `vary_lbl_auth`.

7.5 Applicability to other schemes

Although our new benchmarking strategy is mostly applied to the type of schemes that we aim to analyze (i.e., in which the attributes are triples and the literals can be positive or negative), we note that our methodology applies to any scheme that is "in between" the ones we analyze and those implemented in ABE Squared (i.e., single-authority CP-ABE for monotone Boolean formulas), as well as any key-policy analogs of those schemes. For example, single-authority schemes supporting negations (using labeled attributes) [TKN20, AT20, Ven23] and all multi-authority schemes [LW11a, RW15, AG21, AG23, Ven23] could not be properly benchmarked with ABE Squared but can be properly benchmarked with ABE Cubed. For those, the duplication cases that we consider can be adjusted to those that are relevant.

8 Evaluation of the benchmarks

In this section, we evaluate the benchmarks for the various optimizations that we have implemented. Because of the large number of timings collected during our performance analysis (we have obtained a data set of over 18,000 timings!), we strive to summarize the most important takeaways from our benchmarks. At the same time, we aim to give a complete analysis, considering both the benefits and trade-offs of the optimizations. To do

this, we have created several Jupyter notebooks, which contain scripts that analyze the data sets and print tables accordingly. In this way, each table provided in this paper can be carefully reconstructed and its contents can be verified. These notebooks as well as other plots that visualize our timings can be found in our GitHub repository [AVR+].

8.1 Strategy 1(a) benchmarks

In the first phase of our benchmarking strategy, we analyze the computational costs of the setup, key generation, encryption and decryption for various policy lengths. In these policies, each authority-label-attribute triple consists of three unique strings. As the figures and tables in our data sets show, all algorithms of all the schemes grow linearly in policy sizes of this kind, and they thus compare similarly at larger sizes. In all cases, the setup requires, per authority, less than a millisecond, and is not affected by any other variables.

We list the costs for all schemes for key generation, encryption and decryption with policy length 100 in Table 4, for positive literals and for negative literals with negation degree 1. We also determine, for each algorithm and positive/negative cases, the fastest optimization, and compute slowdown factors for each other optimization comparative to the fastest. With slowdown factor, we mean that we compute $\max / \min$, where min denotes the minimum costs (i.e., of the fastest candidate) and max denotes the costs of the scheme for which we want to compute the slowdown factor. Computing it as such ensures that we obtain values ≥ 1, effectively normalizing all timings with respect to the fastest. In general, the higher the number, the worse the scheme compares to the fastest.

As the table suggests, all algorithms cost at most 6 milliseconds per attribute. Thus, all algorithms perform in under a second even for large policies, making our implementations practical for those set and policy sizes. Furthermore, the table illustrates that optimizations opt3-opt6 outperform opt0-opt1 in the decryption by a factor 1.5-2, whereas the key generation and encryption algorithms never perform worse than 40% slower. Also note that the decryption costs of opt3-opt6 are within margin of error (of 3%) from one another for the positive literals. Therefore, they do not significantly distinguish themselves from each other in the positive literals. However, the variants using randomness splitting for the negative parts, i.e., opt5-opt6, are significantly slower for policies with negative literals than those that do not use randomness splitting in the negative parts, i.e., opt3-opt4.

We show in the next subsections that they compare differently among various duplication cases. In particular, we show that the current improvements provided by our structural optimizations are not as clearly visible in these benchmarks as they would be in more practically relevant scenarios, e.g., where the number of authorities is much lower compared to the number of labels and attributes. That is, the speedup factor is "only" 2 for the positive literals now, because the fourth pairing product is now linear in the policy length (because it is linear in the number of authorities associated with the policy).

8.2 Strategy 1(b) benchmarks

In the second step, we analyze the influence of the negation degree on the costs. We do this by fixing the policy length to 50 and by measuring the timings for negation degrees $\{1, 2, ..., 10, 20, 30\}$. As our tables in the notebooks show, encryption and decryption both perform in under a second for all negation degrees, and can thus be called practical.

Furthermore, the negation degree does not affect the encryption efficiency, and thus, all costs compare similarly as for negation degree 1. The key generation costs increase by a factor in the degree, because the number of keys generated depends directly on the degree. This means that the algorithms compare similarly for negation degrees larger than 1, and thus, the comparative results provided by Strategy 1(a) are also representative for larger negation degrees.

Table 4: Costs for key generation, encryption and decryption with policy length 100 for positive literals (pos) and negative literals with negation degree 1 (neg), the latter highlighted in gray. The cost is measured in milliseconds (ms) and the slowdown factor (SlF) is with respect to the lowest cost (i.e., fastest timing, indicated by "–") for the same algorithm, but separately for positive and negative literals. Hence, all timings are normalized with respect to the fastest and a higher slowdown factor means higher cost. We highlight in boldface the factors that are within a 3% error margin.

	Key generation		*Encryption*		*Decryption*	
Scheme	ms	SlF	ms	SlF	ms	SlF
opt0 (pos)	**420.7**	–	465.5	1.196	320.9	1.959
opt0 (neg)	**421.5**	1.002	465.7	1.191	610.9	2.063
opt1 (pos)	**420.9**	1.000	464.6	1.194	320.2	1.954
opt1 (neg)	**420.8**	–	464.3	1.188	452.3	1.528
opt2 (pos)	**420.9**	1.001	465.3	1.196	320.5	1.956
opt2 (neg)	**421.0**	1.000	465.4	1.191	453.1	1.530
opt3 (pos)	**421.1**	1.001	**390.1**	1.003	**163.8**	–
opt3 (neg)	**421.0**	1.000	**390.9**	–	**296.1**	–
opt4 (pos)	444.8	1.057	413.7	1.063	**163.9**	1.000
opt4 (neg)	444.7	1.057	**391.5**	1.002	**296.1**	1.000
opt5 (pos)	561.7	1.335	**389.1**	–	**164.0**	1.001
opt5 (neg)	561.2	1.333	442.3	1.131	454.7	1.536
opt6 (pos)	585.4	1.392	414.8	1.066	**164.2**	1.002
opt6 (neg)	585.2	1.390	446.4	1.142	454.9	1.536

Lastly, for the decryption costs, we see that the slowdown factor of opt0-opt2 compared to opt3-opt6 decreases. This means that, even though our optimizations opt3-opt4 still significantly outperform opt0-opt2 in decryption, the benefits of the reduced number of pairings obtained by the randomness re-use techniques diminish as the degree grows. The reason that we see this decrease in effectiveness is that increasing the negation degree only affects the number of exponentiations, and not the number of pairings (as shown in Section 5). To improve further on these numbers, we require new techniques, which we leave for future work.

8.3 Strategy 2 benchmarks

In the second phase, we analyze the computational costs for various duplications types. Because we have to fix many variables, i.e., the policy size, whether the literals are positive or negative (and if negative, the negation degree), and the duplication strategy, we obtain many timings. To separate the interesting results from less insightful findings, we first perform some preliminary analyses on our data sets.

Step 1: measuring the influence of the different duplication cases. First, we measure the influence of the different duplication cases on the costs compared to the baseline. With baseline, we mean the case measured in Strategy 1(a), i.e., all triples are unique. Our approach computes, for every optimization and every duplication case, slowdown factors compared to the baseline. We do this for all divisors $\{1, 2, 3, 4, 5, 6, 10, 12, 30, 60\}$, i.e., the number of "unique triples"[2] that are used in the policy. If, for any of these divisors, the slowdown factor is significant, we consider the duplication case to be relevant for the optimization in question. Otherwise, the optimization performs with similar efficiency as the baseline, in which case, the Strategy 1(a) benchmarks apply.

[2]Note that we consider only the uniqueness of the sub-tuple considered in the duplication case, e.g., `var_auth` considers $1, 2, 3, ..., 30, 60$ different authorities.

Table 5: Influence of the various duplication cases on the optimizations, where we use no duplications (`no_dup`) as the baseline (–). Upward arrows ↑ mean that the respective duplication case results in an increase of the cost compared to the baseline, and downward arrows ↓ stand for a decrease. A circle indicates that there is no substantial change.

	`no_dup`			`auth`			`lbl`			`attr`			`auth_lbl`			`auth_attr`			`lbl_attr`		
Scheme	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D
opt0	–	–	–	↓	∘	∘	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	∘	∘	∘	∘
opt1	–	–	–	↓	∘	∘	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	↓	∘	∘	∘
opt2	–	–	–	↓	∘	↓	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	↓	∘	∘	∘
opt3	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↓	∘	∘	∘
opt4	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↓	∘	∘	∘
opt5	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↓	∘	∘	∘
opt6	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↓	∘	∘	∘

(a) Positive literals

	`no_dup`			`auth`			`lbl`			`attr`			`auth_lbl`			`auth_attr`			`lbl_attr`		
Scheme	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D	KG	E	D
opt0	–	–	–	↓	∘	∘	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	∘	∘	∘	∘
opt1	–	–	–	↓	∘	∘	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	∘	∘	∘	∘
opt2	–	–	–	↓	∘	∘	∘	∘	∘	∘	∘	∘	↓	↓	∘	↓	∘	∘	∘	∘	∘
opt3	–	–	–	↓	↑	↑	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↑	∘	∘	∘
opt4	–	–	–	↓	↑	↑	∘	∘	∘	∘	∘	∘	↓	↓	↑	↓	↑	↑	∘	∘	∘
opt5	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↓	↓	↑	↓	∘	∘	∘
opt6	–	–	–	↓	↑	↓	∘	∘	∘	∘	∘	∘	↓	↓	↓	↓	↑	↓	∘	∘	∘

(b) Negative literals with negation degree 1

We summarize the influence of the various duplication cases on the optimizations in Table 5. The table shows that only the duplication of authorities, of authority-label pairs and of authority-attribute pairs influences the efficiency of the optimizations. The other duplication types do not affect the baseline efficiency, and we can therefore rule these out in our further analysis. The table also suggests that, for positive literals, there is a strict trade-off between encryption and decryption: when the decryption costs decrease compared to the case without duplicates, the encryption costs increase, and vice versa. For the policies with only negative literals, this is not the case.

Step 2: the interaction between duplication and the negation degrees. Second, we argue that the duplication cases and the negation degrees are mostly independent. With mostly independent, we mean that the costs among optimizations do not compare much differently for the various duplication cases compared to the baseline than for negation degree 1. That is, where we noticed increases in costs for certain duplication cases compared to the baseline for negation degree 1, we see similar such increases in costs for higher negation degrees (see the tables in the notebook). Furthermore, much like in the Strategy 1(b) benchmarks, we do see that the speedup obtained from the our randomness re-use techniques (as used in opt3-opt6) is getting less pronounced as the negation degree increases. We therefore choose to focus on only negation degree 1, which simplifies the evaluation. Nevertheless, we emphasize that the optimizations that outperform the starting point scheme do so in all negation degrees across all duplication cases considered, and we expect this trend to show also for other negation degrees.

Step 3: singling out the most significant results. Finally, we compare the computational costs among the optimizations for the relevant duplication cases, to illustrate the decryption speedups as well as the key generation/encryption slowdowns provided by our

new optimization techniques. Because we have three such duplication cases, the distinction between positive and negative literals, three algorithms of interest, and two tables for each combination of those (to fit the results for 10 divisors), we have 36 tables with relevant results. These can all be found in the notebook (and we will summarize them later this section). In this paragraph, we would like to highlight the most significant results, which are the decryption costs for the authority-duplication case and the five or seven smallest divisors (for the positive and negative literals, respectively). Table 6 shows that opt4 and opt6 outperform the starting point scheme (opt0-opt1) by a factor 21 in the best case, i.e., for 1 unique authority and all positive literals. Even for 5 different authorities, this speedup remains convincing: a factor 9 compared to the starting point scheme, and even for 30 authorities, the speedup is still a factor 3.5.

Decryption improvements owing to randomness splitting for the positive parts. The reason why we see this large speedup compared to all other duplication cases (including the baseline) is because of the randomness splitting in the positive parts. As we have shown in our analysis of the decryption (Section 4.6), the split version of the keys allows us to compute the pairings in an order that we cannot perform for the schemes without randomness splitting. The reason why this product is so efficient in this duplication case is twofold: on the one hand, the number of different authorities is small (meaning that one of the pairing products is small). On the other hand, the label-attributes in this duplication cases are still unique, ensuring that ι_l and τ map to one single element. On the other hand, for $\tau_{\tilde{\rho}}$, we have a stronger injectivity restriction, which needs to map each occurrence of the same authority to another randomizer. Hence, $\tau_{\tilde{\rho}}$ maps to a large domain.

Decryption trade-off for randomness splitting in the negative parts. For the case with negative literals, we see a much less pronounced effect: the optimizations using randomness splitting in the negative parts perform significantly faster than all other schemes when the number of authorities is small (i.e., between 50-100%), but much less pronounced. Furthermore, for higher numbers of authorities, the variants using randomness splitting, i.e., opt5 and opt6, are outperformed by the variants using only randomness re-use techniques, i.e., opt3 and opt4. As the table shows, the threshold is at roughly 6 authorities. This means that, in settings in which we can expect that, on average, the number of authorities is at most one per ten attributes in the policy, we can expect a speedup for the variants using randomness splitting compared to those only using randomness re-use.

The reason why we see this trade-off is because of the extra pairing product (see Section 4.6), which grows in the total number of authorities associated with the policy. Even though the "remaining products" of opt5-opt6 should perform (roughly) at least as efficient as those of opt3-opt4, the extra product outweighs the decrease in costs after the threshold for the number of authorities has been reached. Furthermore, in general, we see a less significant speedup for opt5-opt6, even for the low numbers of authorities, because the pairing costs are only a relatively small portion of the decryption costs. Performing the products for negations also requires two exponentiations (for degree 1) per literal, one of which is in the more expensive group $\mathbb{H}$, and the sixth pairing product is still linear in the policy length. Because our optimizations only address reducing the number of pairings and do not address reducing those exponentiations or extra pairings, we do not see such a pronounced improvement in the negations.

The improvements are practical(ly motivated). In summary, our optimizations using randomness re-use have a significant advantage over the schemes not using randomness re-use, and for the positive parts, the randomness-splitting techniques have proven to be very successful. In the best cases, decryption is a factor 11 faster than for the starting point scheme, but even in the more regular or worse cases, we still see significant improvements.

Table 6: Cost of decryption for auth-style duplications and policies of length 60. The cost is measured in milliseconds (ms) and the slowdown factor (SlF) is with respect to the lowest cost (i.e., fastest timing, indicated by "–") in the same column. We vary the number of different authorities used in the policy (m_{aut} resp. $\overline{m}_{\mathsf{aut}}$) from 1 to 5, such that each authority has the same frequency of occurring.

	$m_{\mathsf{aut}} = 1$		$m_{\mathsf{aut}} = 2$		$m_{\mathsf{aut}} = 3$		$m_{\mathsf{aut}} = 4$		$m_{\mathsf{aut}} = 5$	
Scheme	ms	SlF	ms	SlF	ms	SlF	ms	SlF	ms	SlF
opt0	193.610	21.786	193.490	16.089	193.502	12.753	193.487	10.554	193.450	9.012
opt1	193.463	21.769	193.602	16.098	193.615	12.760	193.576	10.559	193.517	9.015
opt2	100.432	11.301	101.953	8.477	103.560	6.825	105.163	5.736	106.772	4.974
opt3	100.160	11.270	54.432	4.526	40.230	2.651	33.907	1.850	30.752	1.433
opt4	**8.887**	–	**12.027**	–	**15.173**	–	**18.332**	–	**21.466**	–
opt5	100.370	11.294	54.513	4.533	40.287	2.655	33.980	1.854	30.818	1.436
opt6	**8.894**	1.001	**12.043**	1.001	**15.196**	1.001	**18.357**	1.001	**21.499**	1.002

(a) Positive literals

	$m_{\mathsf{aut}} = 1$		$m_{\mathsf{aut}} = 2$		$m_{\mathsf{aut}} = 3$		$m_{\mathsf{aut}} = 4$		$m_{\mathsf{aut}} = 5$		$m_{\mathsf{aut}} = 6$		$m_{\mathsf{aut}} = 10$	
Scheme	ms	SlF	ms	SlF	ms	SlF	ms	SlF	ms	SlF	ms	SlF	ms	SlF
opt0	367.5	2.01	367.4	1.97	367.3	1.94	367.5	1.91	367.5	1.88	367.6	1.90	367.5	1.96
opt1	273.1	1.49	273.0	1.47	273.1	1.44	273.0	1.42	273.1	1.40	273.1	1.41	273.1	1.46
opt2	273.4	1.49	273.2	1.47	273.3	1.44	273.3	1.42	273.3	1.40	273.3	1.41	273.3	1.47
opt3	273.3	1.49	225.8	1.21	209.9	1.11	202.0	1.05	**197.3**	1.01	**194.1**	1.01	**187.8**	1.00
opt4	273.0	1.49	225.7	1.21	209.8	1.11	201.8	1.05	**197.1**	1.01	**194.0**	–	**187.8**	–
opt5	**183.2**	1.00	**186.4**	1.00	**189.5**	1.00	**192.6**	–	**196.0**	1.00	**199.0**	1.03	203.7	1.09
opt6	**183.1**	–	**186.2**	–	**189.3**	–	**192.6**	1.00	**195.7**	–	**198.9**	1.03	203.5	1.08

(b) Negative literals with negation degree 1

Also note that these improvements are practically motivated: in practice, we would probably see very often that few authorities are deployed in a system, whereas we might see longer policies consisting of various label-attribute pairs. Thus, even these most significant improvements are practically relevant.

8.4 Summarizing evaluation

Finally, we would like to summarize our results and findings in one table. So far, we have focused mainly on analyzing the algorithm that we aim to optimize: decryption. Furthermore, we have analyzed the impact of our structural changes to the starting point scheme to the key generation and encryption efficiency to some extent, but we have not discussed them in detail for all duplication cases. Therefore, we would like to conclude this section with a table that illustrates the trade-offs.

Overview of the trade-offs. The aim is to give an overview of the decryption improvements (provided by our optimizations) compared to the starting point scheme, while also showing that potential slowdowns of the key generation and encryption algorithms are much less pronounced. We do this by computing the maximum slowdown factor, for every optimization, among all different dimensions (i.e., regular/duplication cases and number of divisors), for both the case with all positive literals and all negative literals, for key generation and encryption. (We do fix the policy size to be 60, also for the "regular" duplication case.) We compute these slowdown factors compared to the opt1 costs (for the same configurations). By taking the maximum of all factors, we can ensure that key generation and encryption do not perform worse compared to opt1 than listed in the table. We compare with opt1, because it is the version of the starting point scheme

Table 7: Trade-offs between the key generation, encryption and decryption efficiency. All results are for policies of length 60, and (neg) rows use negation degree 1. For key generation (KG) and encryption (E), we report the maximum slowdown factor (MSlF) with respect to opt1, where we compare (pos) rows and (neg) rows separately. Note that slowdown factors < 1 indicate that a scheme performs better than opt1. For decryption, we report the minimal cost (in milliseconds, *ms*) and the speedup factor (range) (SpF(R)) with respect to opt1 for different duplication styles. The ranges are computed over all divisors of 60. We highlight the two schemes with the best overall efficiency compared to the others in yellow. For those two schemes, we list the significant differences in bold.

	KG	E	D							
			`no_dup`		`auth`		`auth_lbl`		`auth_attr`	
Scheme	MSlF	MSlF	*ms*	SpF	*ms*	SpFR	*ms*	SpFR	*ms*	SpFR
opt1 (pos)	–	–	193.2	–	193.5	–	193.4	–	100.3	–
opt1 (neg)	–	–	272.6	–	273.0	–	272.5	–	269.3	–
opt2 (pos)	1.003	1.008	193.6	0.998	100.4	0.997-1.926	193.7	0.997-0.999	100.4	0.997-0.999
opt2 (neg)	1.002	1.006	272.9	0.999	273.2	0.997-0.999	273.0	0.997-1.000	269.7	0.997-0.999
opt3 (pos)	**1.002**	**1.002**	100.2	1.929	29.2	1.927-6.633	100.1	1.000-1.931	29.2	1.928-3.926
opt3 (neg)	**1.002**	**1.003**	179.5	1.519	179.7	0.998-1.520	179.8	0.999-1.519	179.8	1.013-1.519
opt4 (pos)	1.060	1.055	100.2	1.929	8.9	1.928-**21.769**	100.2	0.999-1.930	8.9	1.927-**11.284**
opt4 (neg)	1.061	1.003	179.6	1.518	179.8	0.999-1.519	179.8	1.000-1.519	179.7	1.013-1.519
opt5 (pos)	1.337	1.005	100.3	1.925	29.2	1.924-6.622	100.3	0.998-1.928	29.2	1.925-3.919
opt5 (neg)	1.338	1.137	274.9	0.992	183.2	0.992-1.490	203.6	0.992-1.342	179.1	0.992-1.504
opt6 (pos)	1.394	1.056	100.6	1.921	8.9	1.924-21.753	100.4	0.998-1.926	8.9	1.925-11.277
opt6 (neg)	1.393	1.137	274.9	0.992	183.1	0.992-1.492	203.5	0.992-1.342	178.8	0.992-1.506

that is optimized with respect to the same optimization strategies as the other schemes. On the other hand, for decryption, we consider the relevant duplication cases (including the regular one) separately, showing the minimum costs for 60 attributes, as well as the speedup factor for every divisor (compared to the opt1 costs for the same configurations). We visualize the speedup factors 10 divisors by giving a range, rather than a maximum.

Recommendations. Table 7 illustrates the trade-offs between the key generation/encryption and decryption efficiency. Concretely, the table shows that the key generation and encryption are minimally impacted for opt2-opt4: at most 6% slower than opt1 (among all optimizations). For decryption, we can see that opt4 is the most consistent winner. However, we also note that opt4 has slightly larger key generation and encryption penalties than opt3 (at the cost of less impressive speedups for decryption). Hence, our recommendation would be to use opt3 if key generation and encryption efficiency is more important to optimize, and to use opt4 if the performance penalty of up to 6% is not a problem.

9 Future work and conclusion

Finally, we would like to reflect on our goal: extending the benchmarking strategies of ABE Squared to obtain a more complete efficiency analysis, and ultimately, a fairer comparison. We argued earlier that the ABE Squared benchmarking strategy compares schemes only in one dimension. To demonstrate this, we covered the ABE Squared strategy in our Strategy 1(a) benchmarks, which indeed showed that it was insufficient to fairly compare more complex schemes such as those considered in this paper. As our evaluation reveals, the Strategy 1(a) benchmarks illustrate that the decryption of opt3-opt4 performs a factor 1.53-1.96 better than the starting point scheme (opt1), and that decryption of opt3-opt4 perform similarly efficient, not allowing us to pick a clear winner.

In contrast, our analysis based on the Strategy 2 benchmarks give a much broader overview of the computational costs, uncovering the influence of the different variables

on the decryption efficiency. It illustrates that opt4 can have much better decryption improvements when the number of authorities is small compared to the total number of attributes. It is therefore a logical choice of scheme in settings in which an efficiency penalty of at most 6% (for key generation and encryption) is acceptable.

Modularizing randomness re-use and splitting. We also introduced new decentralized large-universe CP-ABE schemes supporting negations, built using (manually applied) randomness re-use and splitting techniques. Although we needed to prove the resulting schemes secure, our proofs build incrementally on the original proof. In future work, it could be interesting to define transformations that modularize the techniques, as well as their security proofs. Additionally, we showed that, in general, the randomness re-use techniques provide good decryption speedups. To enable better speedups, the randomness-splitting techniques have proven to be effective for positive literals.

Improving decryption for negative literals using other structural techniques. Unfortunately, we were not able to see such impressive speedups for decryption with negative literals. One of the reasons is that, to preserve correctness, we need to apply randomness splitting twice, which results in an extra pairing product in decryption. (This extra pairing product ultimately increases the decryption costs compared to the randomness re-use variants that are not using randomness splitting for large numbers of authorities. In a sense, this is the reason why we do not achieve the "best of both worlds" property that we have for the positive literals.) It is possible that better improvements can be made with a slightly different starting-point scheme, e.g., like the more bounded decentralized CP-ABE scheme supporting negations in [Ven23]. This scheme would first need to be adapted to the completely unbounded setting, requiring a completely new security proof. Alternatively, a similar trick as introduced in [VA23] could improve the pairing costs (and possibly, also for the (sixth) pairing product that we were not able to improve).

Improving decryption for negative literals via other operational orders or curves. A more prominent reason why the decryption speedups are less impressive for the negative literals (even in the best cases) is that only the pairing costs of one of the two large products are reduced, and not the (large) number of exponentiations or the other large pairing product. Although the number of pairings (of the remaining products) are reduced by several factors for small numbers of authorities (see Section 4.7), they amount to less than half of the total decryption costs. Perhaps, this other product and the number of exponentiations can be reduced by reordering the operations and using different pairing-friendly curves. To investigate this, we need to obtain a better view on these possible orders in which we can compute the operations and whether we can use pairing-friendly curves that enable better efficiency for those orders. To the best of our knowledge, no works have analyzed this, and thus, this is an interesting, unexplored line of research.

Acknowledgments. The authors would like to thank the anonymous reviewers for their helpful comments. Sven Argo and Tim Güneysu have been supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) under Germany's Excellence Strategy - EXC 2092 CASA - 390781972. Diego Aranha has been supported by the Danish Independent Research Council under project number 1026-00350B (RENAIS).

References

[ABGW17] M. Ambrona, G. Barthe, R. Gay, and H. Wee. Attribute-based encryption in the generic group model: Automated proofs and new constructions. In

B. M. Thuraisingham, D. Evans, T. Malkin, and D. Xu, editors, *CCS*, pages 647–664. ACM, 2017.

[AC17] S. Agrawal and M. Chase. Simplifying design and analysis of complex predicate encryption schemes. In J.-S. Coron and J. B. Nielsen, editors, *EUROCRYPT*, volume 10210 of *LNCS*, pages 627–656. Springer, 2017.

[ac22] arkworks contributors. `arkworks` zkSNARK ecosystem, 2022.

[AFG24] D. F. Aranha, G. Fotiadis, and A. Guillevic. A short-list of pairing-friendly curves resistant to the special TNFS algorithm at the 192-bit security level. *IACR Communications in Cryptology*, 1(3), 2024.

[AG21] M. Ambrona and R. Gay. Multi-authority ABE, revisited. Cryptology ePrint Archive, Paper 2021/1381, 2021.

[AG23] M. Ambrona and R. Gay. Multi-authority ABE for non-monotonic access structures. In A. Boldyreva and V. Kolesnikov, editors, *PKC*, volume 13941 of *LNCS*, pages 306–335. Springer, 2023.

[AGM+13] J. A. Akinyele, C. Garman, I. Miers, M. W. Pagano, M. Rushanan, M. Green, and A. D. Rubin. Charm: a framework for rapidly prototyping cryptosystems. *J. Cryptogr. Eng.*, 3(2):111–128, 2013.

[AH14] Jorge Aparicio and Brook Heisler. `criterion` statistics-driven micro-benchmarking in Rust, 2014.

[AHST23] D. F. Aranha, B. Salling Hvass, B. Spitters, and M. Tibouchi. Faster constant-time evaluation of the Kronecker symbol with application to elliptic curve hashing. In *CCS*, pages 3228–3238. ACM, 2023.

[AT20] N. Attrapadung and J. Tomida. Unbounded dynamic predicate compositions in ABE from standard assumptions. In *ASIACRYPT*, pages 405–436. Springer, 2020.

[Att14] N. Attrapadung. Dual system encryption via doubly selective security: Framework, fully secure functional encryption for regular languages, and more. In P. Q. Nguyen and E. Oswald, editors, *EUROCRYPT*, volume 8441 of *LNCS*, pages 557–577. Springer, 2014.

[Att19] N. Attrapadung. Unbounded dynamic predicate compositions in attribute-based encryption. In Y. Ishai and V. Rijmen, editors, *EUROCRYPT*, volume 11476 of *LNCS*, pages 34–67. Springer, 2019.

[AVR+] S. Argo, M. Venema, D. Riepel, T. Güneysu, and D. F. Aranha. ABE Cubed: Advanced Benchmarking Extensions for ABE Squared. https://github.com/lincolncryptools/ABE_Cubed.

[Bei96] A. Beimel. *Secure Schemes for Secret Sharing and Key Distribution*. Phd thesis, Ben Gurion University, 1996.

[BLS02] P S. L. M. Barreto, B. Lynn, and M. Scott. Constructing elliptic curves with prescribed embedding degrees. In S. Cimato, C. Galdi, and G. Persiano, editors, *SCN*, volume 2576 of *LNCS*, pages 257–267. Springer, 2002.

[Bow17] S. Bowe. BLS12-381: New zk-SNARK elliptic curve construction. https://blog.z.cash/new-snark-curve/, 2017.

[BSW07] J. Bethencourt, A. Sahai, and B. Waters. Ciphertext-policy attribute-based encryption. In *S&P*, pages 321–334. IEEE, 2007.

[CC09] M. Chase and S. S. M. Chow. Improving privacy and security in multi-authority attribute-based encryption. In E. Al-Shaer, S. Jha, and A. D. Keromytis, editors, *CCS*, pages 121–130. ACM, 2009.

[Cha07] M. Chase. Multi-authority attribute-based encryption. In S. P. Vadhan, editor, *TCC*, volume 4392 of *LNCS*, pages 515–534. Springer, 2007.

[CRT25] J. Chávez-Saab, F. Rodríguez-Henríquez, and M. Tibouchi. SwiftEC: Shallue-van de Woestijne indifferentiable function to elliptic curves. *J. Cryptol.*, 38(1):3, 2025.

[dlPVA22] A. de la Piedra, M. Venema, and G. Alpár. ABE squared: Accurately benchmarking efficiency of attribute-based encryption. *TCHES*, 2022(2):192–239, 2022.

[EHK+13] A. Escala, G. Herold, E. Kiltz, C. Ràfols, and J. L. Villar. An algebraic framework for Diffie-Hellman assumptions. In R. Canetti and J. A. Garay, editors, *CRYPTO*, volume 8043 of *LNCS*, pages 129–147. Springer, 2013.

[ETS18] ETSI. ETSI TS 103 458 (V1.1.1). Technical specification, European Telecommunications Standards Institute (ETSI), 2018.

[FEN] The FENTEC project. https://github.com/fentec-project.

[FHK19] Armando Faz-Hernandez and Kris Kwiatkowski. *Introducing CIRCL: An Advanced Cryptographic Library*. Cloudflare, June 2019. Available at https://github.com/cloudflare/circl. v1.4.0 Accessed Aug, 2024.

[GPSW06a] V. Goyal, O. Pandey, A. Sahai, and B. Waters. Attribute-based encryption for fine-grained access control of encrypted data. In A. Juels, R. N. Wright, and S. De Capitani di Vimercati, editors, *CCS*. ACM, 2006.

[GPSW06b] V. Goyal, O. Pandey, A. Sahai, and B. Waters. Attribute-based encryption for fine-grained access control of encrypted data. Cryptology ePrint Archive, Report 2006/309, 2006.

[Gui20] A. Guillevic. Pairing-friendly curves. https://members.loria.fr/AGuillevic/pairing-friendly-curves/, 2020.

[KL10] S. Kamara and K. E. Lauter. Cryptographic cloud storage. In R. Sion, R. Curtmola, S. Dietrich, A. Kiayias, J. M. Miret, K. Sako, and F. Sebé, editors, *RLCPS*, volume 6054 of *LNCS*, pages 136–149. Springer, 2010.

[LSW10] A. Lewko, A. Sahai, and B. Waters. Revocation systems with very small private keys. In *IEEE S & P*, pages 273–285, 2010.

[LVV+23] W. Ladd, T. Verma, M. Venema, A. Faz-Hérnandez, B. McMillion, A. Wildani, and N. Sullivan. Portunus: Re-imagining access control in distributed systems. In J. Lawall and D. Williams, editors, *USENIX ATC*, pages 35–52. USENIX Association, 2023.

[LW11a] A. Lewko and B. Waters. Decentralizing attribute-based encryption. In *EUROCRYPT*, pages 568–588. Springer, 2011.

[LW11b] A. B. Lewko and B. Waters. Unbounded HIBE and attribute-based encryption. In K. G. Paterson, editor, *EUROCRYPT*, volume 6632 of *LNCS*, pages 547–567. Springer, 2011.

[Möl01] B. Möller. Algorithms for multi-exponentiation. In S. Vaudenay and A. M. Youssef, editors, *SAC*, volume 2259 of *LNCS*, pages 165–180. Springer, 2001.

[Nat23] National Institute of Standards and Technology. NIST IR 8214C – NIST First Call for Multi-Party Threshold Schemes. https://csrc.nist.gov/pubs/ir/8214/c/ipd, 2023.

[RVV24a] D. Riepel, M. Venema, and T. Verma. ISABELLA: Improving structures of attribute-based encryption leveraging linear algebra. In B. Luo, X. Liao, J. Xu, E. Kirda, and D. Lie, editors, *CCS 2024*, pages 4628–4642. ACM, 2024.

[RVV24b] D. Riepel, M. Venema, and T. Verma. ISABELLA: Improving structures of attribute-based encryption leveraging linear algebra. Cryptology ePrint Archive, Paper 2024/1352, 2024.

[RW15] Y. Rouselakis and B. Waters. Efficient statically-secure large-universe multi-authority attribute-based encryption. In R. Böhme and T. Okamoto, editors, *FC*, volume 8975 of *LNCS*, pages 315–332. Springer, 2015.

[RW22] D. Riepel and H. Wee. FABEO: fast attribute-based encryption with optimal security. In H. Yin, A. Stavrou, C. Cremers, and E. Shi, editors, *CCS*, pages 2491–2504. ACM, 2022.

[SKSW22] Yumi Sakemi, Tetsutaro Kobayashi, Tsunekazu Saito, and Riad S. Wahby. Pairing-Friendly Curves. Internet-Draft draft-irtf-cfrg-pairing-friendly-curves-11, Internet Engineering Task Force, November 2022. Work in Progress.

[SRGS12] N. Santos, R. Rodrigues, K. P. Gummadi, and S. Saroiu. Policy-sealed data: A new abstraction for building trusted cloud services. In *USENIX Security Symposium*, pages 175–188. USENIX Association, 2012.

[SW05] A. Sahai and B. Waters. Fuzzy identity-based encryption. In R. Cramer, editor, *EUROCRYPT*, volume 3494 of *LNCS*, pages 457–473. Springer, 2005.

[TKN20] J. Tomida, Y. Kawahara, and R. Nishimaki. Fast, compact, and expressive attribute-based encryption. In A. Kiayias, M. Kohlweiss, P. Wallden, and V. Zikas, editors, *PKC*, volume 12110 of *LNCS*, pages 3–33. Springer, 2020.

[VA22] M. Venema and G. Alpár. TinyABE: Unrestricted ciphertext-policy attribute-based encryption for embedded devices and low-quality networks. In L. Batina and J. Daemen, editors, *AFRICACRYPT*, volume 13503 of *LNCS*, pages 103–129. Springer, 2022.

[VA23] M. Venema and G. Alpár. GLUE: generalizing unbounded attribute-based encryption for flexible efficiency trade-offs. In A. Boldyreva and V. Kolesnikov, editors, *PKC*, volume 13940 of *LNCS*, pages 652–682. Springer, 2023.

[VAH23] M. Venema, G. Alpár, and J.-H. Hoepman. Systematizing core properties of pairing-based attribute-based encryption to uncover remaining challenges in enforcing access control in practice. *Des. Codes Cryptogr.*, 91(1):165–220, 2023.

[Ven23] M. Venema. A practical compiler for attribute-based encryption: New decentralized constructions and more. In M. Rosulek, editor, *CT-RSA*, volume 13871 of *LNCS*, pages 132–159. Springer, 2023.

[Wat11] B. Waters. Ciphertext-policy attribute-based encryption - an expressive, efficient, and provably secure realization. In D. Catalano, N. Fazio, R. Gennaro, and A. Nicolosi, editors, *PKC*, volume 6571 of *LNCS*, pages 53–70. Springer, 2011.

[WB19] R. S. Wahby and D. Boneh. Fast and simple constant-time hashing to the BLS12-381 elliptic curve. *TCHES*, 2019(4):154–179, 2019.

[WMZV16] F. Wang, J. Mickens, N. Zeldovich, and V. Vaikuntanathan. Sieve: Cryptographically enforced access control for user data in untrusted clouds. In Katerina J. Argyraki and Rebecca Isaacs, editors, *NSDI*, pages 611–626. USENIX Association, 2016.

[YAHK14] S. Yamada, N. Attrapadung, G. Hanaoka, and N. Kunihiro. A framework and compact constructions for non-monotonic attribute-based encryption. In H. Krawczyk, editor, *PKC*, volume 8383 of *Lecture Notes in Computer Science*, pages 275–292. Springer, 2014.

[Zeu20] Zeutro. The OpenABE library - open source cryptographic library with attribute-based encryption implementations in C/C++. https://github.com/zeutro/openabe, 2020.

A Security proofs

A.1 Security statements from ISABELLA

Theorem 1 ([RVV24b], informal)**.** *Let* Γ *be a PES-ISA scheme,* $\mathfrak{D}$ *and* $\mathcal{F}$ *be compatible distributions and* SPM *be a split-predicate mold for* Γ*. Let* $\mathsf{ABE} = \mathbf{Comp}[\mathsf{PGGen}, \Gamma, \mathsf{SPM}, \mathfrak{D}, \mathcal{F}]$ *be the compiled scheme. If* Γ *satisfies SSSP, then* ABE *is statically secure under the strong parallel DBDH assumption and adaptively secure in the generic group model.*

The remaining subsections therefore focus on proving SSSP for the different variants.

A.2 SSSP for the starting point scheme

The SSSP proof for the scheme is given in [RVV24b]. We recall it here, because we use it as a starting point for the security proofs for our modified schemes.

Lemma 1. *The PES-ISA in Definition 5 satisfies SSSP.*

Proof. We set $d_1 = n_1$ and $d_2 = ((n_1+2)|\rho_{\mathsf{lab}}([n_1])|+1)n_2+1$. Let $\mathfrak{C} \subseteq [n_{\mathrm{aut}}]$ denote some set of corrupted authorities. We also use a special notation from e.g., [VA22, Ven23, VA23] for the column indices that makes the relationship between the input indices and the columns of the substitution matrices clearer. In particular, we map the tuples $(1,k)$, $(2,j,k,\mathsf{lab})$, $(3,k,\mathsf{lab})$, $(4,1)$, $(5,k,\mathsf{lab})$ injectively in the interval $[d_2]$, where $j \in [n_1]$ are the rows of the policy matrix, $k \in [n_2]$ are the columns of the policy matrix, and $\mathsf{lab} \in \rho_{\mathsf{lab}}([n_1])$ are the labels that occur in the ciphertext policy.

- $\mathrm{EncB}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \to (\{\mathbf{a}_l, \mathbf{B}_l, \bar{\mathbf{B}}_l\}_{l\in[n_{\mathrm{aut}}]}, \{\mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i}\}_{\mathsf{lab}\in\mathcal{L},i\in\{0,1\}}\}_{l\in[n_{\mathrm{aut}}]})$, where $\mathbf{a}_l = \mathbf{0}^{d_1}$ and $\mathbf{B}_l, \bar{\mathbf{B}}_l, \mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i} = \mathbf{0}^{d_1\times d_2}$ for all $l \in \mathfrak{C}$ and $i \in \{0,1\}$, and let

$\mathbf{v} \in \mathbb{Z}_p^{n_2}$ (with $v_1 = 1$) be the vector orthogonal to each row $j \in \tilde{\rho}^{-1}(\mathfrak{C})$ associated with a corrupted authority. For all $l \in [n_{\text{aut}}] \setminus \mathfrak{C}$, we set:

$$\mathbf{a}_l = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \\ k \in [n_2]}} A_{j,k} v_k \mathbf{1}_j^{d_1} ,$$

$$\mathbf{B}_l = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \\ k \in [2,n_2]}} A_{j,k} \left(\mathbf{1}_{j,(1,k)}^{d_1 \times d_2} + v_k \mathbf{1}_{j,(1,1)}^{d_1 \times d_2} \right) - \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \\ k \in [n_2]}} A_{j,k} v_k \mathbf{1}_{j,(4,1)}^{d_1 \times d_2} ,$$

$$\bar{\mathbf{B}}_l = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \\ k \in [2,n_2]}} A_{j,k} \left(\mathbf{1}_{j,(1,k)}^{d_1 \times d_2} + v_k \mathbf{1}_{j,(1,1)}^{d_1 \times d_2} \right) - \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \\ k \in [n_2]}} A_{j,k} v_k \mathbf{1}_{j,(4,1)}^{d_1 \times d_2} ,$$

$$\mathbf{B}_{l,\mathsf{lab},0} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \cap \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}_{j,(2,j,k,\mathsf{lab})}^{d_1 \times d_2} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \setminus \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}_{j,(3,k,\mathsf{lab})}^{d_1 \times d_2} ,$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},0} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \cap \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}_{j,(2,j,k,\mathsf{lab})}^{d_1 \times d_2} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}_{j,(5,k,\mathsf{lab})}^{d_1 \times d_2} ,$$

$$\mathbf{B}_{l,\mathsf{lab},1} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \cap \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}_{j,(2,j,k,\mathsf{lab})}^{d_1 \times d_2} ,$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},1} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \cap \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}_{j,(2,j,k,\mathsf{lab})}^{d_1 \times d_2} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}_{j,(3,k,\mathsf{lab})}^{d_1 \times d_2} ,$$

where $\Psi_0 = \{j \in [n_1] \mid \rho'(j) = 0\}$ and $\Psi_1 = \{j \in [n_1] \mid \rho'(j) = 1\}$.

- $\mathrm{EncR}((\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}), \mathcal{S})$
  $\to (\mathbf{r}_{\mathsf{GID}}, \{\mathbf{r}_{l,\mathsf{att}}, \bar{\mathbf{r}}_{l,\mathsf{att}}\}_{(l,\mathsf{lab},\mathsf{att}) \in \mathcal{S}})$, where

$$\mathbf{r}_{\mathsf{GID}} = \overline{\mathbf{1}}_{(4,1)}^{d_2} - \sum_{k \in [n_2]} w_k \overline{\mathbf{1}}_{(1,k)}^{d_2} ,$$

$$\mathbf{r}_{l,\mathsf{att}} = \sum_{\mathsf{lab}' \in \chi_{l,\mathsf{att}}} \left(\sum_{\substack{j \in \overline{\Upsilon}_{l,1} \cap \rho_{\mathsf{lab}}^{-1}(\mathsf{lab}') \\ k \in [n_2]}} \frac{w_k \overline{\mathbf{1}}_{(2,j,k,\mathsf{lab}')}^{d_2}}{x_{\mathsf{att}} - x_{\rho(j)}} + \sum_{k \in [n_2]} w_k \overline{\mathbf{1}}_{(3,k,\mathsf{lab}')}^{d_2} \right) ,$$

$$\bar{\mathbf{r}}_{l,\mathsf{att}} = \sum_{\substack{\mathsf{lab}' \in \chi_{l,\mathsf{att}} \\ k \in [n_2]}} w_k \left(\overline{\mathbf{1}}_{(3,k,\mathsf{lab}')}^{d_2} - x_{\mathsf{att}} \overline{\mathbf{1}}_{(5,k,\mathsf{lab}')}^{d_2} + \sum_{j \in \overline{\Upsilon}_{l,0} \cap \rho^{-1}(\mathsf{att})} \overline{\mathbf{1}}_{(2,j,k,\mathsf{lab}')}^{d_2} \right) ,$$

  and $\chi_{l,\mathsf{att}} = \{\mathsf{lab} \mid (\mathsf{lab}, \mathsf{att}) \in \mathcal{S}_l\}$, $\overline{\Upsilon}_{l,1} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 1 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_l\}$, and $\overline{\Upsilon}_{l,0} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 0 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_l\}$, and the vector $\mathbf{w}$ is such that $(\mathbf{w})_1 = -1$ and $\mathbf{A}_j \mathbf{w}^\intercal = 0$ for all j with $\rho'(j) = 1$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_{\tilde{\rho}(j)}$ or $\rho'(j) = 0$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_{\tilde{\rho}(j)}$.
- $\mathrm{EncS}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \to (\{\mathbf{s}_j\}_{j \in [n_1]}, \{\hat{\mathbf{v}}_k, \hat{\mathbf{v}}'_k\}_{k \in [2,n_2]}, \tilde{\mathbf{s}})$, where

$$\tilde{\mathbf{s}} = 1, \qquad \mathbf{s}_j = -\mathbf{1}_j^{d_1}, \qquad \hat{\mathbf{v}}_k = v_k, \qquad \hat{\mathbf{v}}'_k = \overline{\mathbf{1}}_{(1,k)}^{d_2} + v_k \overline{\mathbf{1}}_{(1,1)}^{d_2}.$$

In [RVV24b], it is shown that these proofs verify correctly. □

A.3 SSSP for the optimizations

We also provide security proofs for the optimizations obtained by modifying the starting point scheme, in the ISABELLA framework [RVV24b]. Because the security property, i.e., SSSP (Definition 3), is purely algebraic, we can prove security by adapting the original proof rather than providing a proof completely from scratch. This simplifies the proof verification considerably, because we can assume that the original proof verifies correctly and use that to verify our proofs more efficiently. Furthermore, it could also provide more insights on how to prove similar modifications to other schemes secure. To illustrate that our proof strategy works, we give a complete proof of SSSP, along with verifications.

Security proof for the randomness re-use versions: opt2 and opt3. To prove security of opt2 and opt3, we adapt the security proof of the starting point scheme (which constitutes a security proof for opt1 as well). The proof below is for opt3, but can be easily extended to opt2 by taking trivial mappings $\tau_{\tilde{\rho}}$. The proof is similar to the proof for the starting point scheme, with the exception that the row entries of the matrices (and subsequently entries of the non-lone ciphertext-variable substitutions) are changed from j to $\tau_{\tilde{\rho}}(j)$ and the substitution for the non-lone key variables are layering with respect to ι_l now. We set $d_1 = m_{\tilde{\rho}}$ and $d_2 = ((n_1+2)|\rho_{\mathsf{lab}}([n_1])| + 1)n_2 + 1$. Let $\mathfrak{C} \subseteq [n_{\text{aut}}]$ denote some set of corrupted authorities.

- $\text{EncB}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \to (\{\mathbf{a}_l, \mathbf{B}_l, \bar{\mathbf{B}}_l\}_{l\in[n_{\text{aut}}]}, \{\mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i}\}_{\mathsf{lab}\in\mathcal{L},i\in\{0,1\}}\}_{l\in[n_{\text{aut}}]})$, where $\mathbf{a}_l = \mathbf{0}^{d_1}$ and $\mathbf{B}_l, \bar{\mathbf{B}}_l, \mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i} = \mathbf{0}^{d_1\times d_2}$ for all $l \in \mathfrak{C}$ and $i \in \{0,1\}$, and let $\mathbf{v} \in \mathbb{Z}_p^{n_2}$ (with $v_1 = 1$) be the vector orthogonal to each row $j \in \tilde{\rho}^{-1}(\mathfrak{C})$ associated with a corrupted authority. For all $l \in [n_{\text{aut}}] \setminus \mathfrak{C}$, we set:

$$\mathbf{a}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}_{\tau_{\tilde{\rho}}(j)}{}^{d_1},$$

$$\mathbf{B}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)},$$

$$\bar{\mathbf{B}}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)},$$

$$\mathbf{B}_{l,\mathsf{lab},0} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(3,k,\mathsf{lab})},$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},0} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(5,k,\mathsf{lab})},$$

$$\mathbf{B}_{l,\mathsf{lab},1} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(2,j,k,\mathsf{lab})},$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},1} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(3,k,\mathsf{lab})},$$

where $\Psi_0 = \{j \in [n_1] \mid \rho'(j) = 0\}$ and $\Psi_1 = \{j \in [n_1] \mid \rho'(j) = 1\}$.

- $\mathrm{EncR}((\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}), \mathcal{S})) \to (\mathbf{r}_{\mathsf{GID}}, \{\mathbf{r}_{l,\ell}, \bar{\mathbf{r}}_{l,\ell}\}_{l,\ell\in[m_l]})$, where

$$\mathbf{r}_{\mathsf{GID}} = \overline{\mathbf{1}}^{d_2}_{(4,1)} - \sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(1,k)} ,$$

$$\mathbf{r}_{l,\ell} = \sum_{(\mathsf{lab},\mathsf{att})\in\iota_l^{-1}(\ell)} \left(\sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}) \\ k\in[n_2]}} \frac{w_k \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})}}{x_{\mathsf{att}} - x_{\rho(j)}} + \sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab})} \right) ,$$

$$\bar{\mathbf{r}}_{l,\ell} = \sum_{\substack{(\mathsf{lab},\mathsf{att})\in\iota_l^{-1}(\ell), \\ k\in[n_2]}} w_k \left(\overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab})} - x_{\mathsf{att}} \overline{\mathbf{1}}^{d_2}_{(5,k,\mathsf{lab})} + \sum_{j\in\overline{\Upsilon}_{l,0}\cap\rho^{-1}(\mathsf{att})} \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})} \right) ,$$

  and $\overline{\Upsilon}_{l,1} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 1 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_l\}$, and $\overline{\Upsilon}_{l,0} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 0 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_l\}$, and the vector $\mathbf{w}$ is such that $(\mathbf{w})_1 = -1$ and $\mathbf{A}_j \mathbf{w}^\intercal = 0$ for all j with $\rho'(j) = 1$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_{\tilde{\rho}(j)}$ or $\rho'(j) = 0$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_{\tilde{\rho}(j)}$.
- $\mathrm{EncS}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \to (\{\mathbf{s}_\ell\}_{\ell\in[m_{\tilde{\rho}}]}, \{\hat{\mathbf{v}}_k, \hat{\mathbf{v}}'_k\}_{k\in[2,n_2]}, \tilde{\mathbf{s}})$, where

$$\tilde{\mathbf{s}} = 1, \qquad \mathbf{s}_\ell = -\mathbf{1}^{d_1}_\ell, \qquad \hat{\mathbf{v}}_k = v_k, \qquad \hat{\mathbf{v}}'_k = \overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k \overline{\mathbf{1}}^{d_2}_{(1,1)}.$$

Note that the verification of the proof is similar to that in [RVV24b]. In particular, for the ciphertext polynomials, we obtain the same values by substituting, e.g., $s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)}$, as the original monomial $s_j b_{\tilde{\rho}(j)}$ because of how the proofs are constructed, e.g., $\mathbf{1}_j$ multiplied by a matrix with j in the first entry yields the associated row, and $\mathbf{1}_{\tau_{\tilde{\rho}}(j)}$ multiplied by a matrix with $\tau_{\tilde{\rho}}(j)$ in the first entry yields then that same row. Furthermore, for the key polynomials, evaluating $r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1})$ in $k_{1,(l,\mathsf{lab},\mathsf{att})}$ yields the same values as in the original proof. The reason is that $r_{l,\ell}$ contains all substitution vectors for $(\mathsf{lab}, \mathsf{att})$ pairs that are mapped to ℓ with ι_l. The injectivity restriction ensures that each lab may occur at most once. This label lab is used in the substitution vectors, and it ensures that multiplying by, e.g., $b_{l,\mathsf{lab}',0}$, selects the layer in the non-lone variable substitution that matches lab'. Since the rest of the substitution vectors and matrices are not affected, we know that all polynomials evaluate to $\mathbf{0}$.

Security proof for the randomness-splitting technique (positive parts): opt4, opt6. To prove security of opt4 and opt6, we adapt the security proof of the starting point scheme *for the positive parts* as follows. In particular, we replace the original substitutions for b_l for new substitutions for b_l and b'_l, and we change the row indices of the substitutions for $b_{l,\mathsf{lab},0}$ and $b_{l,\mathsf{lab},1}$ from $\tau_{\tilde{\rho}}(j)$ to $\tau(j)$.

To find suitable substitutions for b_l and b'_l, we again use the verifications of the relevant keys and ciphertexts in the ISABELLA framework. Specifically, we use that $\alpha_l + r_{\mathsf{GID}} b_l$ is substituted by $-\sum_{j\in\overline{\Upsilon}_{l,1},k\in[n_2]} A_{j,k} w_k \mathbf{1}^{d_1}_j$ (which is in this case replaced by $-\sum_{j\in\overline{\Upsilon}_{l,1},k\in[n_2]} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}$ due to the randomness re-use techniques), and is canceled by the substitution for $r_{l,\mathsf{att}}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1})$. Now, we need to ensure that $\alpha_l + r_{\mathsf{GID}} b_l$ is canceled out by $r_{l,\mathsf{att}} b'_l$ and that $r_{l,\mathsf{att}}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}} b_{l,\mathsf{lab},1})$ evaluates to 0 on its own. At the same time, we also need to make sure that the ciphertext polynomials $c_{2,j}$ evaluate to $\overline{\mathbf{0}}^{d_2}$. To ensure this, we encode the "remainder" in $b_{l,\mathsf{lab},0}$ and b'_l, so that they cancel out the "remainder" in keys and one another in the evaluation of $c_{2,j}$. (Note that this strategy is similar to what we applied to b_l and b'_l in the proof for the first randomness-splitting

strategy.) In sum, for all $l \in [n_{\text{aut}}] \setminus \mathfrak{C}$, we set:

$$\mathbf{B}'_l = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1, \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau_{\tilde{\rho}}(j),(6,k)},$$

$$\mathbf{B}_{l,\mathsf{lab},0} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(3,k,\mathsf{lab})}$$
$$- \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}), \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(6,k)},$$

$$\mathbf{B}_{l,\mathsf{lab},1} = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_1 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(2,j,k,\mathsf{lab})},$$

and lastly, we set, for all $\ell \in [m_l]$,

$$\mathbf{r}_{l,\ell} = \sum_{k \in [n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(6,k)} + \sum_{(\mathsf{lab},\mathsf{att}) \in \iota_l^{-1}(\ell)} \left(\sum_{\substack{j \in \overline{\Upsilon}_{l,1} \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}') \\ k \in [n_2]}} \frac{w_k \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab}')}}{x_{\mathsf{att}} - x_{\rho(j)}} + \sum_{k \in [n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab}')} \right).$$

The rest of the substitutions is the same. Note that, by how the substitution vectors and matrices are designed from the original proof, the polynomials evaluate to $\mathbf{0}$ upon substitution.

Security proof for the randomness-splitting technique (negative parts): opt5, opt6. For the proofs for opt5 and opt6, we adapt the security proofs of the versions that they are building on *for the negative parts* as follows. In particular, we replace the substitutions for $\bar{b}_l$ (in the proofs for opt3) for new substitutions for common variables $\bar{b}'_l$, and $\bar{b}'_{l,\mathsf{lab}}$, and non-lone key variables $\bar{r}_l$ and $\bar{r}_{l',\iota_{l'}(\mathsf{lab}',\mathsf{att}')}$.

Like for our other optimizations, we use the verifications of the key and ciphertext substitutions (for opt3) to find suitable substitutions. Specifically, for the original keys $k_{2,l,\mathsf{lab}}$, it is shown that the part $\alpha_l + r_{\mathsf{GID}} \bar{b}_l$ is substituted by $-\sum_{j \in \overline{\Upsilon}_{l,0}, k \in [n_2]} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}$, and is canceled by the substitution for $\bar{r}_{l,\mathsf{lab}} \bar{b}_{l,\mathsf{lab},1}$. To ensure that $k_{2,1,l}$ evaluates to $\mathbf{0}$, we need to substitute $\bar{r}_l$ and $\bar{b}'_l$ so that $\bar{r}_l \bar{b}'_l$ evaluates to $-\sum_{j \in \overline{\Upsilon}_{l,0}, k \in [n_2]} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}$. Additionally, we need to substitute $\bar{r}_l$ and $\bar{b}'_{l,\mathsf{lab}}$ so that $\bar{r}_l \bar{b}'_{l,\mathsf{lab}}$ evaluates to $\sum_{j \in \overline{\Upsilon}_{l,0}, k \in [n_2]} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}$. In this way, $k_{2,2,l,\mathsf{lab}}$ also evaluates to $\mathbf{0}$.

At the same time, the substitutions for $\bar{b}_l$, $\bar{b}'_l$, and $\bar{b}'_{l,\mathsf{lab}}$ need to ensure that the ciphertext polynomials $c_{2,j}$ and $c_{3,j}$ evaluate to $\mathbf{0}$. (Note that, because $c_{1,j}$ is the same as in opt3, and $\bar{b}_l$ is not modified, we already know it evaluates to $\mathbf{0}$.) To ensure that $c_{3,j}$ evaluates to $\mathbf{0}$, we only have to change the row indices of $\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1}$ and $\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0}$ from $\tau_{\tilde{\rho}}(j)$ to $\tau(j)$. For $c_{2,j}$, we note that it consists of common variables that were not present in opt3. So, if we substitute them by matrices so that (modulo the row indices), their entries are the same, then we can ensure that $c_{2,j}$ evaluates to $\mathbf{0}$. To this end, we make the following substitutions for all $l \in [n_{\text{aut}}] \setminus \mathfrak{C}$:

$$\bar{\mathbf{B}}'_l = \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0, \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau_{\tilde{\rho}}(j),(7,k)}$$

$$\bar{\mathbf{B}}'_{l,\mathsf{lab}} = - \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0, \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(7,k)}$$

$$\bar{\mathbf{r}}'_l = \sum_{k\in[n_2]} w_k \bar{\mathbf{1}}^{d_2}_{(7,k)}$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},0} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)}A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(5,k,\mathsf{lab})},$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},1} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(3,k,\mathsf{lab})}$$

The other substitutions are the same as those for opt3 and opt4, depending on whether we use randomness splitting in the positive parts (opt4) or not (opt3).

A.4 Complete description and security proof for opt6

We give a complete description and security proof for opt6. We first give a definition of the PES for opt6.

Definition 6 (Unbounded decentralized large-universe CP-ABE with negations with randomness splitting and re-use)**.** Let $\mathcal{L}$ denote a universe of labels and let n_{aut} denote a number of authorities. We define a PES-ISA for the predicate $P_{\text{MA-CP-not}}\colon \mathcal{X}_{\text{MA-CP-not}} \times \mathcal{Y}_{\text{MA-CP-not}} \to \{0,1\}$ as follows.

- Param($\mathcal{L}$): Let $\{\mathcal{A}_l\}_{l\in[n_{\text{aut}}]}$ be the authorities. On input the label universe $\mathcal{L}$, we set $n_\alpha = n_{\text{aut}}$ and $n_b = (2+4|\mathcal{L}|)n_{\text{aut}}$, where $\boldsymbol{\alpha} = \{\alpha_l\}_{l\in[n_{\text{aut}}]}$, and $\mathbf{b} = (\{b_l, \bar{b}_l, \{b_{l,\mathsf{lab},0}, b_{l,\mathsf{lab},1}, \bar{b}_{l,\mathsf{lab},0}, \bar{b}_{l,\mathsf{lab},1}\}_{\mathsf{lab}\in\mathcal{L}}\}_{l\in[n_{\text{aut}}]})$.
- EncKey($\mathcal{S}$): We set $m_1 = 2|\mathcal{S}|+1$, $m_2 = 0$, and

$$\mathbf{k} = (\{k_{1,1,l,\ell} = \alpha_l + r_{\mathsf{GID}}b_l + r_{l,\ell}b'_l,\quad k_{1,2,(l,\mathsf{lab},\mathsf{att})} = r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}}b_{l,\mathsf{lab},1}),$$
$$k_{2,1,l} = \alpha_l + r_{\mathsf{GID}}\bar{b}_l + \bar{r}_l\bar{b}'_l,\quad k_{2,2,l,\mathsf{lab}} = \bar{r}_l\bar{b}'_{l,\mathsf{lab}} + \bar{r}_{l,\mathsf{lab}}\bar{b}_{l,\mathsf{lab},1},$$
$$k_{3,(l,\mathsf{lab},\mathsf{att})} = \bar{r}_{l,\mathsf{att}}(\bar{b}_{l,\mathsf{lab},0} + x_{\mathsf{att}}\bar{b}_{l,\mathsf{lab},1})\}_{(l,\mathsf{lab},\mathsf{att})\in\mathcal{S}})$$

  where $\bar{r}_{l,\mathsf{lab}} = \sum_{(\mathsf{lab}',\mathsf{att}')\in\mathcal{S}_l|\mathsf{lab}'=\mathsf{lab}} \bar{r}_{l',\iota_{l'}(\mathsf{lab}',\mathsf{att}')}$ and x_{att} is the representation of att in $\mathbb{Z}_p$. Additionally, the mapping $\iota_l\colon \mathcal{S}_l \to [m_l]$ is such that m_l denotes the maximum number of attributes att per label managed by the same authority l, i.e., in $\mathcal{S}_l$, and ι_l is injective on each subdomain $\mathcal{S}_{l,\mathsf{lab}} = \{(\mathsf{lab}',\mathsf{att}) \mid (\mathsf{lab}',\mathsf{att}) \in \mathcal{S}_l \wedge \mathsf{lab}' = \mathsf{lab}\}$.
- EncCt($\mathbf{A}, \rho, \rho', \rho_{\mathsf{lab}}, \tilde{\rho}$): We set $w_1 = n_1$, $w_2 = n_2 - 1$, $w'_2 = n_2 - 1$, $c_M = \tilde{s}$,

$$\mathbf{c} = (\{c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)}b_{\tilde{\rho}(j)},$$
$$c_{2,j} = s_{\tau_{\tilde{\rho}}(j)}b'_{\tilde{\rho}(j)} + s_{\tau(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\Psi},$$
$$\{c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)}\bar{b}_{\tilde{\rho}(j)},\quad c_{2,j} = s_{\tau_{\tilde{\rho}}(j)}\bar{b}'_{\tilde{\rho}(j)} + s_{\tau(j)}\bar{b}'_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)},$$
$$c_{3,j} = s_{\tau(j)}(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})\}_{j\in\overline{\Psi}})$$

  and $\mathbf{c}' = (\{c'_j = \lambda_j + \alpha_{\tilde{\rho}(j)}s_{\tau_{\tilde{\rho}}(j)}\}_{j\in[n_1]})$, where $\lambda_j = A_{j,1}\tilde{s} + \sum_{k\in[2,n_2]} A_{j,k}\hat{v}_k$ and $\mu_j = \sum_{k\in[2,n_2]} A_{j,k}\hat{v}'_k$, and $\Psi = \{j\in[n_1] \mid \rho'(j) = 1\}$ and $\overline{\Psi} = [n_1]\setminus\Psi$ (i.e., the set of rows associated with the non-negated and negated attributes, respectively), and $\mathbf{s} = (\{s_j\}_{[n_1]})$. Lastly, the mapping $\tau_{\tilde{\rho}}\colon [n_1] \to [m_{\tilde{\rho}}]$ is such that $m_{\tilde{\rho}}$ is the maximum number of rows associated with an authority, i.e., $m_{\tilde{\rho}} = \max_{l\in[n_{\text{aut}}]} |\{j\in[n_1] \mid \tilde{\rho}(j) = l\}|$, and it is injective on the subdomain $\{j\in[n_1] \mid \tilde{\rho}(j) = l\}$ for each authority $\mathcal{A}_l$. Furthermore, the mapping $\tau\colon [n_1] \to [m]$ is such that $m = \max_{(\mathsf{lab},l)\in\mathcal{L}\times[n_{\text{aut}}]} |\{j\in[n_1] \mid \rho_{\mathsf{lab}}(j) = \mathsf{lab}, \tilde{\rho}(j) = l\}|$ is the maximum number of rows j that are mapped to the same label-authority pair $(\mathsf{lab}, l) \in \mathcal{L}\times[n_{\text{aut}}]$ with ρ_{lab} and $\tilde{\rho}$, and τ is injective on each subdomain $\{j\in[n_1] \mid \rho_{\mathsf{lab}}(j) = \mathsf{lab}, \tilde{\rho}(j) = l\}$ (for each $(\mathsf{lab}, l) \in \mathcal{L}\times[n_{\text{aut}}]$).

We show that the following lemma holds. For this proof, we also show that the proofs verify correctly upon substitution in the key and ciphertext polynomials.

Lemma 2. *The PES-ISA for opt6 (Definition 6) satisfies SSSP.*

Proof. We set $d_1 = m_{\tilde{\rho}}$ and $d_2 = ((n_1+2)|\rho_{\mathsf{lab}}([n_1])| + 3)\, n_2 + 1$. Let $\mathfrak{C} \subseteq [n_{\mathrm{aut}}]$ denote some set of corrupted authorities. We also use a special notation from e.g., [VA22, Ven23, VA23] for the column indices that makes the relationship between the input indices and the columns of the substitution matrices clearer. In particular, we map the tuples $(1,k)$, $(2,j,k,\mathsf{lab})$, $(3,k,\mathsf{lab})$, $(4,1)$, $(5,k,\mathsf{lab})$, $(6,k)$ and $(7,k)$ injectively in the interval $[d_2]$, where $j \in [n_1]$ are the rows of the policy matrix, $k \in [n_2]$ are the columns of the policy matrix, and $\mathsf{lab} \in \rho_{\mathsf{lab}}([n_1])$ are the labels that occur in the ciphertext policy. Let $\mathfrak{C} \subseteq [n_{\mathrm{aut}}]$ denote some set of corrupted authorities.

- $\mathrm{EncB}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}})$
  $\to (\{\mathbf{a}_l, \mathbf{B}_l, \bar{\mathbf{B}}_l, \bar{\mathbf{B}}'_l\}_{l\in[n_{\mathrm{aut}}]}, \{\mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i}, \mathbf{B}'_{l,\mathsf{lab}}\}_{\mathsf{lab}\in\mathcal{L}, i\in\{0,1\}}\}_{l\in[n_{\mathrm{aut}}]})$, where $\mathbf{a}_l = \mathbf{0}^{d_1}$ and $\mathbf{B}_l, \bar{\mathbf{B}}_l, \mathbf{B}_{l,\mathsf{lab},i}, \bar{\mathbf{B}}_{l,\mathsf{lab},i} = \mathbf{0}^{d_1\times d_2}$ for all $l \in \mathfrak{C}$ and $i \in \{0,1\}$, and let $\mathbf{v} \in \mathbb{Z}_p^{n_2}$ (with $v_1 = 1$) be the vector orthogonal to each row $\mathbf{A}_j$ with $j \in \tilde{\rho}^{-1}(\mathfrak{C})$ associated with a corrupted authority. For all $l \in [n_{\mathrm{aut}}] \setminus \mathfrak{C}$, we set:

$$\mathbf{a}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}\,,$$

$$\mathbf{B}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)}\,,$$

$$\bar{\mathbf{B}}_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)}\,,$$

$$\mathbf{B}'_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1,\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(6,k)},$$

$$\bar{\mathbf{B}}'_l = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(7,k)}$$

$$\bar{\mathbf{B}}'_{l,\mathsf{lab}} = -\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(7,k)}$$

$$\mathbf{B}_{l,\mathsf{lab},0} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\setminus\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(3,k,\mathsf{lab})}$$
$$- \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab}),\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(6,k)}\,,$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},0} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(5,k,\mathsf{lab})}\,,$$

$$\mathbf{B}_{l,\mathsf{lab},1} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})}\,,$$

$$\bar{\mathbf{B}}_{l,\mathsf{lab},1} = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(3,k,\mathsf{lab})} ,$$

where $\Psi_0 = \{j \in [n_1] \mid \rho'(j) = 0\}$ and $\Psi_1 = \{j \in [n_1] \mid \rho'(j) = 1\}$.

- $\mathrm{EncR}((\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}), \mathcal{S})) \to (\mathbf{r}_{\mathsf{GID}}, \{\bar{\mathbf{r}}_l, \mathbf{r}_{l,\ell}, \bar{\mathbf{r}}_{l,\ell}\}_{l,\ell\in[m_l]})$, where

$$\mathbf{r}_{\mathsf{GID}} = \overline{\mathbf{1}}^{d_2}_{(4,1)} - \sum_{k\in[n_2]} w_k\overline{\mathbf{1}}^{d_2}_{(1,k)} ,$$

$$\bar{\mathbf{r}}'_l = \sum_{k\in[n_2]} w_k\overline{\mathbf{1}}^{d_2}_{(7,k)}$$

$$\mathbf{r}_{l,\ell} = \sum_{k\in[n_2]} w_k\overline{\mathbf{1}}^{d_2}_{(6,k)} + \sum_{(\mathsf{lab},\mathsf{att})\in\iota_l^{-1}(\ell)} \left(\sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} \frac{w_k\overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})}}{x_{\mathsf{att}} - x_{\rho(j)}} + \sum_{k\in[n_2]} w_k\overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab})} \right),$$

$$\bar{\mathbf{r}}_{l,\ell} = \sum_{\substack{(\mathsf{lab},\mathsf{att})\in\iota_l^{-1}(\ell),\\ k\in[n_2]}} w_k \left(\overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab})} - x_{\mathsf{att}}\overline{\mathbf{1}}^{d_2}_{(5,k,\mathsf{lab})} + \sum_{j\in\overline{\Upsilon}_{l,0}\cap\rho^{-1}(\mathsf{att})} \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})} \right) ,$$

and $\overline{\Upsilon}_{l,1} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 1 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_l\}$, and $\overline{\Upsilon}_{l,0} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 0 \wedge (\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_l\}$, and the vector $\mathbf{w}$ is such that $(\mathbf{w})_1 = -1$ and $\mathbf{A}_j\mathbf{w}^\intercal = 0$ for all j with $\rho'(j) = 1$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \in \mathcal{S}_{\tilde{\rho}(j)}$ or $\rho'(j) = 0$ and $(\rho_{\mathsf{lab}}(j), \rho(j)) \notin \mathcal{S}_{\tilde{\rho}(j)}$.

- $\mathrm{EncS}(\mathbf{A}, \rho, \tilde{\rho}, \rho', \rho_{\mathsf{lab}}) \to (\{\mathbf{s}_\ell\}_{\ell\in[m_{\tilde{\rho}}]}, \{\hat{\mathbf{v}}_k, \hat{\mathbf{v}}'_k\}_{k\in[2,n_2]}, \tilde{\mathbf{s}})$, where

$$\tilde{\mathbf{s}} = 1, \qquad \mathbf{s}_\ell = -\mathbf{1}^{d_1}_\ell, \qquad \hat{\mathbf{v}}_k = v_k, \qquad \hat{\mathbf{v}}'_k = \overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k\overline{\mathbf{1}}^{d_2}_{(1,1)}.$$

We now verify that all polynomials evaluate to $\mathbf{0}$ when the variables are substituted as indicated above. We start with the key polynomials for $l \notin \mathfrak{C}$. (For the corrupted authorities $l \in \mathfrak{C}$, all master-key and (semi-)common variables are substituted by all-zero vectors and matrices, and thus, the associated key polynomials trivially evaluate to $\mathbf{0}$.) Let $\Upsilon_{l,0} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 0 \wedge (\rho_{\mathsf{lab}}, \rho(j)) \notin \mathcal{S}_l\}$ and $\Upsilon_{l,1} = \{j \in [n_1] \mid \tilde{\rho}(j) = l \wedge \rho'(j) = 1 \wedge (\rho_{\mathsf{lab}}, \rho(j)) \in \mathcal{S}_l\}$.

For $k_{1,1,l,\ell}$, we first show that $\alpha_l + r_{\mathsf{GID}}b_l$ evaluates to

$$\begin{aligned}
\alpha_l + r_{\mathsf{GID}}b_l \;:\; & \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\\ k\in[n_2]}} A_{j,k}v_k\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \\
& + \left(\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[n_2]}} A_{j,k}v_k\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)} \right) \\
& \cdot \left(\overline{\mathbf{1}}^{d_2}_{(4,1)} - \sum_{k\in[n_2]} w_k\overline{\mathbf{1}}^{d_2}_{(1,k)} \right) \\
= & \sum_{\substack{j\in\Psi_{l,0}\\ k\in[n_2]}} A_{j,k}v_k\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} + \sum_{\substack{j\in\Psi_{l,1}\\ k\in[n_2]}} A_{j,k}v_k\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}w_k\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}
\end{aligned}$$

$$
\begin{aligned}
&- \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}v_k w_1 \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[n_2]}} A_{j,k}v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)}\\
&= - \sum_{j\in\Psi_{l,1}} A_{j,1}v_1 w_1 \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[2,n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}\\
&= \underbrace{- \sum_{\substack{j\in\Upsilon_{l,1}\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}}_{=0} - \sum_{\substack{j\in\overline{\Upsilon}_{l,1}\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} = - \sum_{\substack{j\in\overline{\Upsilon}_{l,1}\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)},
\end{aligned}
$$

which is canceled out by $r_{l,\ell}b'_l$, because

$$
\begin{aligned}
r_{l,\ell}b'_l \;:\; &\left(\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1,\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(6,k)} \right)\\
&\cdot \left(\sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(6,k)} + \sum_{(\mathsf{lab},\mathsf{att})\in\iota_l^{-1}(\ell)} \left(\sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} \frac{w_k \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})}}{x_{\mathsf{att}} - x_{\rho(j)}} + \sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab})} \right) \right)\\
&= \underbrace{\sum_{\substack{j\in\Upsilon_{l,1},\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}}_{=0} + \sum_{\substack{j\in\overline{\Upsilon}_{l,1},\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} = \sum_{\substack{j\in\overline{\Upsilon}_{l,1},\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}.
\end{aligned}
$$

Then, we have that the key polynomial $k_{1,2,(l,\mathsf{lab},\mathsf{att})} = r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}}b_{l,\mathsf{lab},1})$ evaluates to $\mathbf{0}$, because

$$
\begin{aligned}
&r_{l,\iota_l(\mathsf{lab},\mathsf{att})}(b_{l,\mathsf{lab},0} + x_{\mathsf{att}}b_{l,\mathsf{lab},1})\\
&: \left(\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j)}A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\setminus\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(3,k,\mathsf{lab})} \right.\\
&\quad \left. - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}),\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(6,k)} + x_{\mathsf{att}} \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} A_{j,k}\mathbf{1}^{d_1\times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} \right)\\
&\cdot \left(\sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(6,k)} + \sum_{(\mathsf{lab}',\mathsf{att}')\in\iota_l^{-1}(\iota_l(\mathsf{lab},\mathsf{att}))} \left(\sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}')\\ k\in[n_2]}} \frac{w_k \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab}')}}{x_{\mathsf{att}'} - x_{\rho(j)}} + \sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab}')} \right) \right)\\
&= - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}),\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau(j)} + \sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} \frac{x_{\mathsf{att}} - x_{\rho(j)}}{x_{\mathsf{att}} - x_{\rho(j)}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau(j)}\\
&= \underbrace{- \sum_{\substack{j\in\Upsilon_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}),\\ k\in[n_2]}} A_{j,k}w_k \mathbf{1}^{d_1}_{\tau(j)}}_{=0}
\end{aligned}
$$

$$- \sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab}),\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)} + \sum_{\substack{j\in\overline{\Upsilon}_{l,1}\cap\rho_{\mathsf{lab}}^{-1}(\mathsf{lab})\\ k\in[n_2]}} \frac{x_{\mathsf{att}} - x_{\rho(j)}}{x_{\mathsf{att}} - x_{\rho(j)}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)} = \mathbf{0}^{d_1},$$

because there exists exactly one att' with the label lab that is mapped to $\iota_l(\mathsf{lab},\mathsf{att})$ with ι_l and that is att. (This follows from the injectivity restriction of ι_l.) Note that this also ensures that there exist no $j \in \tilde{\rho}^{-1}(l)$ with $\rho_{\mathsf{lab}}(j) \neq \mathsf{lab}$ and $\iota_l(\rho_{\mathsf{lab}}(j), \rho(j)) = \iota_l(\mathsf{lab},\mathsf{att})$.

Then, $k_{2,1,l} = \alpha_l + r_{\mathsf{GID}}\bar{b}_l + \bar{r}_l\bar{b}'_l$ evaluates to $\mathbf{0}$, because for $\alpha_l + r_{\mathsf{GID}}\bar{b}_l$, we have

$$\begin{aligned}
\alpha_l + r_{\mathsf{GID}}\bar{b}_l \;:\; & \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \\
& + \left(\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(1,1)}\right) - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)} \right) \\
& \cdot \left(\overline{\mathbf{1}}^{d_2}_{(4,1)} - \sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(1,k)} \right) \\
= & \sum_{\substack{j\in\Psi_{l,1}\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} + \sum_{\substack{j\in\Psi_{l,0}\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \\
& - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k} v_k w_1 \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[n_2]}} A_{j,k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(4,1)} \\
= & - \sum_{j\in\Psi_{l,0}} A_{j,1} v_1 w_1 \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} - \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \\
= & \underbrace{- \sum_{\substack{j\in\Upsilon_{l,0}\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}}_{=0} - \sum_{\substack{j\in\overline{\Upsilon}_{l,0}\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} = - \sum_{\substack{j\in\overline{\Upsilon}_{l,0}\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)},
\end{aligned}$$

which is canceled out by $\bar{r}_l\bar{b}'_l$, because

$$\begin{aligned}
\bar{r}_l\bar{b}'_l \;:\; & \left(\sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j),(7,k)} \right) \cdot \left(\sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(7,k)} \right) = \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \\
& = \underbrace{\sum_{\substack{j\in\Upsilon_{l,0},\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}}_{=0} + \sum_{\substack{j\in\overline{\Upsilon}_{l,0},\\ k\in[n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}
\end{aligned}$$

Then, verification of $k_{2,2,l,\mathsf{lab}} = \bar{r}_l\bar{b}'_{l,\mathsf{lab}} + \bar{r}_{l,\mathsf{lab}}\bar{b}_{l,\mathsf{lab},1}$ is split again in two parts:

$$\bar{r}_l\bar{b}'_{l,\mathsf{lab}} \;:\; \left(- \sum_{\substack{j\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j,k} \mathbf{1}^{d_1\times d_2}_{\tau(j),(7,k)} \right) \cdot \left(\sum_{k\in[n_2]} w_k \overline{\mathbf{1}}^{d_2}_{(7,k)} \right)$$

$$= - \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0, \\ k \in [n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)} = \underbrace{- \sum_{\substack{j \in \Upsilon_{l,0}, \\ k \in [n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)}}_{=0} - \sum_{\substack{j \in \overline{\Upsilon}_{l,0}, \\ k \in [n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)},$$

which is canceled by

$$\bar{r}_{l,\mathsf{lab}} \bar{b}_{l,\mathsf{lab},1} = \left(\sum_{(\mathsf{lab}',\mathsf{att}') \in \mathcal{S}_l \mid \mathsf{lab}' = \mathsf{lab}} \bar{r}_{l',\iota_{l'}(\mathsf{lab}',\mathsf{att}')} \right) \bar{b}_{l,\mathsf{lab},1}$$

$$\cdot \left(\sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(3,k,\mathsf{lab})} \right)$$

$$\cdot \left(\sum_{\substack{(\mathsf{lab}',\mathsf{att}') \in \mathcal{S}_l \mid \mathsf{lab}' = \mathsf{lab} \\ (\mathsf{lab}'',\mathsf{att}'') \in \iota_l^{-1}(\iota_l(\mathsf{lab}',\mathsf{att}')) \\ k \in [n_2]}} w_k \left(\overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab}'')} - x_{\mathsf{att}'} \overline{\mathbf{1}}^{d_2}_{(5,k,\mathsf{lab}'')} + \sum_{j \in \overline{\Upsilon}_{l,0} \cap \rho^{-1}(\mathsf{att}')} \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab}'')} \right) \right)$$

$$= \underbrace{\sum_{\substack{j \in \Upsilon_{l,0} \\ k \in [n_2]}} A_{j,k} w_k \mathbf{1}^{d_1 \times d_2}_{\tau(j)}}_{=0} + \sum_{\substack{j \in \overline{\Upsilon}_{l,0} \\ k \in [n_2]}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)},$$

because there exists exactly one att' with the label lab that is mapped to $\iota_l(\mathsf{lab}, \mathsf{att})$ with ι_l and that is att itself. (This follows from the injectivity restriction of ι_l.)

For the verification of $k_{3,(l,\mathsf{lab},\mathsf{att})} = \bar{r}_{l,\iota_l(\mathsf{lab},\mathsf{att})}(\bar{b}_{l,\mathsf{lab},0} + x_{\mathsf{att}} \bar{b}_{l,\mathsf{lab},1})$, we compute

$$\bar{r}_{l,\mathsf{att}}(\bar{b}_{l,\mathsf{lab},0} + x_{\mathsf{att}} \bar{b}_{l,\mathsf{lab},1})$$

$$\cdot \left(\sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} -x_{\rho(j)} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(5,k,\mathsf{lab})} \right.$$

$$\left. + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} x_{\mathsf{att}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(2,j,k,\mathsf{lab})} + \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} x_{\mathsf{att}} A_{j,k} \mathbf{1}^{d_1 \times d_2}_{\tau(j),(3,k,\mathsf{lab})} \right)$$

$$\cdot \left(\sum_{\substack{(\mathsf{lab}',\mathsf{att}') \in \iota_l^{-1}(\iota_l(\mathsf{lab},\mathsf{att})), \\ k \in [n_2]}} w_k \left(\overline{\mathbf{1}}^{d_2}_{(3,k,\mathsf{lab}')} - x_{\mathsf{att}'} \overline{\mathbf{1}}^{d_2}_{(5,k,\mathsf{lab}')} + \sum_{j \in \overline{\Upsilon}_{l,0} \cap \rho^{-1}(\mathsf{att}')} \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab}')} \right) \right)$$

$$= \sum_{\substack{j \in \overline{\Upsilon}_{l,0} \cap \rho^{-1}(\mathsf{att}) \cap \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} \underbrace{\left(x_{\mathsf{att}} - x_{\rho(j)}\right)}_{=0} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)} - \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} x_{\mathsf{att}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)}$$

$$+ \sum_{\substack{j \in \tilde{\rho}^{-1}(l) \cap \Psi_0 \setminus \rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k \in [n_2]}} x_{\mathsf{att}} A_{j,k} w_k \mathbf{1}^{d_1}_{\tau(j)} = \mathbf{0}^{d_1},$$

because there exists exactly one att' with the label lab that is mapped to $\iota_l(\mathsf{lab}, \mathsf{att})$ with ι_l and that is att itself. (This follows from the injectivity restriction of ι_l.) Note that this also ensures that there exist no $j \in \tilde{\rho}^{-1}(l)$ with $\rho_{\mathsf{lab}}(j) \neq \mathsf{lab}$ and $\iota_l(\rho_{\mathsf{lab}}(j), \rho(j)) = \iota_l(\mathsf{lab}, \mathsf{att})$.

For the verification of the ciphertext polynomials $c'_j = \lambda_j + \alpha_{\tilde{\rho}(j)} s_{\tau_{\tilde{\rho}}(j)}$, we compute

$$\begin{aligned}
\lambda_j + \alpha_{\tilde{\rho}(j)} s_{\tau_{\tilde{\rho}}(j)} &= A_{j,1}\tilde{s} + \sum_{k\in[2,n_2]} A_{j,k}\hat{v}_k + \alpha_{\tilde{\rho}(j)} s_{\tau_{\tilde{\rho}}(j)} \\
&: \; A_{j,1}\cdot 1 + \sum_{k\in[2,n_2]} A_{j,k} v_k - \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \sum_{\substack{j'\in\tilde{\rho}^{-1}(\tilde{\rho}(j)) \\ k\in[n_2]}} A_{j',k} v_k \mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j')} \\
&= \sum_{k\in[n_2]} A_{j,k} v_k - \sum_{k\in[n_2]} A_{j,k} v_k = 0.
\end{aligned}$$

We now verify the ciphertext polynomials with $j \in \Psi_1$. We first verify $c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)}$, i.e.,

$$\begin{aligned}
\mu_j + s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)} &= \sum_{k\in[2,n_2]} A_{j,k}\hat{v}'_k + s_{\tau_{\tilde{\rho}}(j)} b_{\tilde{\rho}(j)} \\
&: \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k \overline{\mathbf{1}}^{d_2}_{(1,1)}\right) \\
-\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} &\left(\sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1 \\ k\in[2,n_2]}} A_{j',k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}('j),(1,k)} + v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(1,1)}\right) - \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0 \\ k\in[n_2]}} A_{j',k} v_k \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(4,1)} \right) \\
&= \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k \overline{\mathbf{1}}^{d_2}_{(1,1)}\right) - \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k \overline{\mathbf{1}}^{d_2}_{(1,1)}\right) = \overline{\mathbf{0}}^{d_2}.
\end{aligned}$$

Then, we verify $c_{2,j} = s_{\tau_{\tilde{\rho}}(j)} b'_{\tilde{\rho}(j)} + s_{\tau(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})$ by showing that

$$s_{\tau_{\tilde{\rho}}(j)} b'_{\tilde{\rho}(j)} \; : \; -\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1, \\ k\in[n_2]}} A_{j',k} \mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(6,k)} = \sum_{k\in[n_2]} A_{j,k} \overline{\mathbf{1}}^{d_2}_{(6,k)}$$

is canceled by

$$\begin{aligned}
& s_{\tau(j)}(b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)} b_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1}) \\
: \; -\mathbf{1}^{d_1}_{\tau(j)} &\left(\sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k\in[n_2]}} -x_{\rho(j)} A_{j',k} \mathbf{1}^{d_1\times d_2}_{\tau(j'),(2,j',k,\mathsf{lab})} + \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1\setminus\rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k\in[n_2]}} A_{j',k} \mathbf{1}^{d_1\times d_2}_{\tau(j'),(3,k,\mathsf{lab})} \right. \\
& \left. - \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab}), \\ k\in[n_2]}} A_{j',k} \mathbf{1}^{d_1\times d_2}_{\tau(j'),(6,k)} + x_{\rho(j)} \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab}) \\ k\in[n_2]}} A_{j',k} \mathbf{1}^{d_1\times d_2}_{\tau(j'),(2,j',k,\mathsf{lab})} \right) \\
&= \sum_{k\in[n_2]} \left(x_{\rho(j)} - x_{\rho(j)}\right) A_{j,k} \overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})} + \sum_{k\in[n_2]} A_{j,k} \overline{\mathbf{1}}^{d_2}_{(6,k)} = \sum_{k\in[n_2]} A_{j,k} \overline{\mathbf{1}}^{d_2}_{(6,k)}.
\end{aligned}$$

We then verify the ciphertext polynomials with $j \in \Psi_0$. For $c_{1,j} = \mu_j + s_{\tau_{\tilde{\rho}}(j)}\bar{b}_{\tilde{\rho}(j)}$, we compute

$$\mu_j + s_{\tau_{\tilde{\rho}}(j)}\bar{b}_{\tilde{\rho}(j)} = \sum_{k\in[2,n_2]} A_{j,k}\hat{v}'_k + s_{\tau_{\tilde{\rho}}(j)}\bar{b}_{\tilde{\rho}(j)}$$
$$: \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k\overline{\mathbf{1}}^{d_2}_{(1,1)}\right)$$
$$-\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)}\left(\sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0\\ k\in[2,n_2]}} A_{j',k}\left(\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}('j),(1,k)} + v_k\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(1,1)}\right) - \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1\\ k\in[n_2]}} A_{j',k}v_k\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(4,1)}\right)$$
$$= \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k\overline{\mathbf{1}}^{d_2}_{(1,1)}\right) - \sum_{k\in[2,n_2]} A_{j,k}\left(\overline{\mathbf{1}}^{d_2}_{(1,k)} + v_k\overline{\mathbf{1}}^{d_2}_{(1,1)}\right) = \overline{\mathbf{0}}^{d_2}.$$

Subsequently, we verify $c_{2,j} = s_{\tau_{\tilde{\rho}}(j)}\bar{b}'_{\tilde{\rho}(j)} + s_{\tau(j)}\bar{b}'_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)}$ by showing that

$$s_{\tau_{\tilde{\rho}}(j)}\bar{b}'_{\tilde{\rho}(j)} \;:\; -\mathbf{1}^{d_1}_{\tau_{\tilde{\rho}}(j)} \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_1,\\ k\in[n_2]}} A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau_{\tilde{\rho}}(j'),(7,k)} = -\sum_{k\in[n_2]} A_{j,k}\overline{\mathbf{1}}^{d_2}_{(7,k)}$$

is canceled by

$$s_{\tau(j)}\bar{b}'_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j)} \;:\; -\mathbf{1}^{d_1}_{\tau(j)} \cdot - \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0,\\ k\in[n_2]}} A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau(j'),(7,k)} = \sum_{k\in[n_2]} A_{j,k}\overline{\mathbf{1}}^{d_2}_{(7,k)}.$$

Lastly, we have that $c_{3,j} = s_{\tau(j)}(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})$ evaluates to $\overline{\mathbf{0}}^{d_2}$, because

$$s_{\tau(j)}(\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),0} + x_{\rho(j)}\bar{b}_{\tilde{\rho}(j),\rho_{\mathsf{lab}}(j),1})$$
$$: \; -\mathbf{1}^{d_1}_{\tau(j)}\left(\sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} -x_{\rho(j')}A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau(j'),(2,j',k,\mathsf{lab})} + \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau(j'),(5,k,\mathsf{lab})}\right.$$
$$\left. + x_{\rho(j)}\left(\sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0\cap\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau(j'),(2,j',k,\mathsf{lab})} + \sum_{\substack{j'\in\tilde{\rho}^{-1}(l)\cap\Psi_0\setminus\rho^{-1}_{\mathsf{lab}}(\mathsf{lab})\\ k\in[n_2]}} A_{j',k}\mathbf{1}^{d_1\times d_2}_{\tau(j'),(3,k,\mathsf{lab})}\right)\right)$$
$$= -\sum_{k\in[n_2]} \left(x_{\rho(j)} - x_{\rho(j)}\right) A_{j,k}\overline{\mathbf{1}}^{d_2}_{(2,j,k,\mathsf{lab})} = \overline{\mathbf{0}}^{d_2}.$$

□